

CS 6290 — Georgia Tech OMSCS

High Performance Computer Architecture

Lecture Notes

kinginu

Fall 2026

Contents

1	Introduction	1
1.1	What is computer architecture?	1
1.2	Technology trends	1
1.3	Speed, power and cost	2
1.4	Power consumption	2
1.5	Fabrication cost	3
2	Metrics and Evaluation	4
2.1	Performance	4
2.2	Comparing performance	4
2.3	Measuring performance	4
2.4	Summarizing performance	5
2.5	The iron law of performance	5
2.6	Amdahl's law	6
3	Pipelining	8
3.1	The idea of pipelining	8
3.2	Pipeline CPI	8
3.3	Stalls and flushes	9
3.4	Control dependences	9
3.5	Data dependences	10
3.6	Dependences and hazards	10
3.7	Handling hazards	10
3.8	How many stages?	11
4	Branches	12
4.1	Branches in a pipeline	12
4.2	Prediction accuracy and performance	13
4.3	Predict not taken	13
4.4	The need for better prediction	14
4.5	The branch target buffer	14
4.6	Direction prediction	15
4.7	The two-bit predictor	16
4.8	History-based predictors	18
4.9	Shared counters: PShare and GShare	19
4.10	Tournament and hierarchical predictors	20
4.11	The return address stack	22
5	Predication	24
5.1	Prediction and predication	24
5.2	If-conversion	25
5.3	Conditional move instructions	26
5.4	Performance of if-conversion	27
5.5	Benefits and costs of conditional moves	28
5.6	Full predication	28
6	ILP	30
6.1	All instructions in the same cycle	30
6.2	Dependences in same-cycle execution	30
6.3	Removing false dependences	31
6.4	Register renaming	31
6.5	Dependences after renaming	32
6.6	Instruction-level parallelism	33
6.7	Structural and control dependences	34
6.8	ILP and IPC	35

7	Instruction Scheduling	37
7.1	Improving IPC	37
7.2	Tomasulo's algorithm	37
7.3	Organization	38
7.4	Issue	39
7.5	Dispatch	40
7.6	Write result	40
7.7	The steps in one cycle	41
7.8	Load and store instructions	42
7.9	A complete example	43
7.10	Timing without tracing the structures	44
8	ReOrder Buffer	46
8.1	Exceptions in out-of-order execution	46
8.2	Branch mispredictions	46
8.3	Correct out-of-order execution	47
8.4	The reorder buffer	48
8.5	Branch misprediction recovery	50
8.6	Exceptions with a reorder buffer	50
8.7	RAT updates on commit	51
8.8	A reorder buffer example	51
8.9	Timing with a reorder buffer	53
8.10	Unified reservation stations	54
8.11	Superscalar processors	54
8.12	Terminology	55
8.13	What is out of order	55
9	Memory Ordering	56
9.1	Dependences through memory	56
9.2	When memory is written	56
9.3	The load-store queue	57
9.4	Out-of-order and in-order load-store execution	58
9.5	Store-to-load forwarding	59
9.6	A load-store queue example	60
9.7	The LSQ, the ROB and the reservation stations	61
10	Compiler ILP	62
10.1	How the compiler can raise IPC	62
10.2	Tree height reduction	63
10.3	Instruction scheduling	64
10.4	Scheduling and if-conversion	66
10.5	Loop unrolling	66
10.6	Function call inlining	68
10.7	Other compiler techniques	70
11	VLIW	71
11.1	Superscalar and VLIW processors	71
11.2	Advantages and disadvantages	72
11.3	VLIW instructions	73
11.4	VLIW processors in practice	74
12	Cache Review	75
12.1	The locality principle	75
12.2	Caches	76
12.3	Cache performance	76
12.4	Blocks and lines	77
12.5	Block number, tag and valid bit	78
12.6	Where a block can be placed	79
12.7	Direct-mapped caches	79

12.8 Set-associative caches	80
12.9 Fully associative caches	80
12.10 Cache replacement	81
12.11 Write policy	82
12.12 Putting it together	83
13 Virtual Memory	86
13.1 Why virtual memory	86
13.2 Physical memory and virtual address spaces	86
13.3 Pages and page tables	87
13.4 Page faults	88
13.5 Address translation	88
13.6 The size of a flat page table	89
13.7 Multi-level page tables	90
13.8 Choosing the page size	91
13.9 Memory access time with translation	92
13.10 The translation lookaside buffer	93
13.11 TLB organization and performance	94
14 Advanced Caches	95
14.1 Improving cache performance	95
14.2 Pipelined caches	95
14.3 Address translation and the hit time	96
14.4 Aliasing	97
14.5 Way prediction	99
14.6 Replacement policy and hit time	100
14.7 Causes of misses	101
14.8 Larger cache blocks	102
14.9 Prefetching	102
14.10 Loop interchange	104
14.11 Overlapping misses	104
14.12 Cache hierarchies	105
14.13 Local and global miss rates	106
14.14 The inclusion property	107
15 Memory	109
15.1 Memory technologies	109
15.2 The SRAM cell	109
15.3 The DRAM cell	110
15.4 Memory chip organization	111
15.5 Refresh	113
15.6 Address multiplexing	113
15.7 Fast page mode	113
15.8 Connecting DRAM to the processor	114
Bibliography	116
Index	118

Introduction

Lecture	Lesson 1, videos 1–19
Textbook	H&P [1] §1.1–1.6
Keywords	computer architecture, Moore’s law, memory wall, dynamic power, static power, fabrication cost, yield

This lesson defines what computer architecture is, explains why it is a discipline in its own right, and introduces the technology trends and the three quantities—performance, power and cost—that constrain every design decision in the rest of the course.

1.1 What is computer architecture?

The discipline of designing a computer that is well suited to its purpose, using the fabrication technology that is available, is called *computer architecture*. Two points in this definition deserve emphasis.

- *Well suited to its purpose*. A mobile phone, a laptop and a data-center server are all computers, but they optimize for different things: battery life, size and weight in one case; raw performance and throughput per dollar in another. Architecture is therefore about *trade-offs*, not about a single best design.
- *Available technology*. The architect does not design transistors, but must know what the current process can deliver—how many transistors fit on a chip, how fast they switch, how much power they dissipate—because those numbers bound what is achievable.

1.1.1 Why do we need it?

Computer architecture serves two goals.

1. **Improve performance**, in the broad sense: execution speed, battery life, size, weight, energy efficiency.
2. **Improve abilities**: make the computer able to do things it could not do before, e.g. 3D graphics, hardware support for debugging, security features.

A further reason is timing. A processor that is designed today will be fabricated several years from now, so the architect must *anticipate* where the technology will be at that time rather than design for today’s parameters.

1.2 Technology trends

1.2.1 Moore’s law

Moore’s law is the empirical observation that the number of transistors that can be placed on a chip of a given area doubles roughly every 18 to 24 months. It is a trend, not a law of physics, but it held for several decades and shaped the expectations of the whole industry. The lecture derives the following consequences, all with the same 18–24-month doubling period:

- processor *speed* doubles (more transistors allow more parallelism and smaller, faster gates);
- the *energy per operation* halves;
- *memory capacity* doubles.

Example 1.1. If speed doubles every two years, a factor-of- $2^5 = 32$ improvement is expected over a decade. Conversely, a design team that spends four years producing a processor only $2\times$ faster than the one it replaces has merely kept pace with the trend.

▷ コンピュータアーキテクチャ

Source Lesson 1, videos 2–3; H&P §1.1–1.3.

The lecture summarizes these as “improve performance” and “improve abilities”.

Tip Whenever a design decision in later lessons looks over-engineered, ask which technology trend it was anticipating.

▷ ムーアの法則

Source [2]; Lesson 1, video 6; H&P §1.4.

1.2.2 The memory wall

Not every component follows the same curve. While processor performance doubled every 18–24 months, memory *latency* improved only by about 1.1× over the same interval. The result is a gap between how fast the processor can consume data and how fast memory can supply it that widens every generation. This gap is called the *memory wall* and is the reason why caches (Lessons 12–14) are so important.

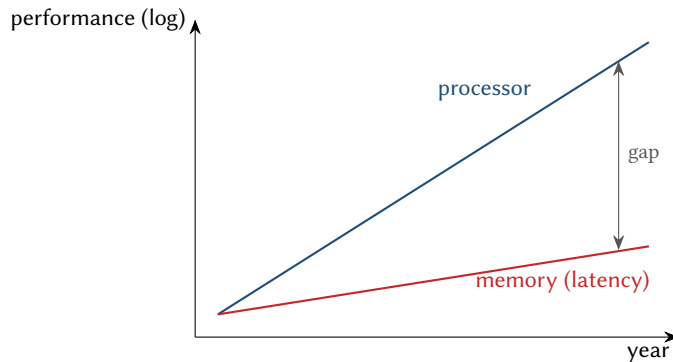


Figure 1.1: Processor performance and memory latency improve at different rates; the difference is the memory wall.

Key point

Memory capacity keeps up with Moore’s law; memory *latency* does not. Most of the memory-system design in this course exists to hide that latency.

1.3 Speed, power and cost

For a long time “better” simply meant “faster”. Today the architect has to balance three quantities:

Performance how fast the machine runs its intended workload;

Power how much energy it consumes, which sets battery life for portable devices and cooling cost for servers;

Cost how much it costs to fabricate.

Which of these dominates depends on the product: weight and battery life for a phone, cost for an embedded controller, performance for a supercomputer.

1.4 Power consumption

The power dissipated by a processor has two components.

1.4.1 Dynamic (active) power

The power consumed by charging and discharging the capacitance of wires and gates whenever a signal switches is called *dynamic power*:

$$P_{\text{dyn}} = \frac{1}{2} CV^2 f \alpha, \quad (1.1)$$

where C is the total capacitance being switched, V the supply voltage, f the clock frequency and α the *activity factor*, the fraction of the capacitance that actually switches each cycle.

Two facts follow directly from Equation (1.1).

- Power scales with the *square* of the voltage, so lowering V is the most effective lever. However, the maximum frequency a circuit can sustain also drops as V drops, so in practice V and f are lowered together and power falls roughly with V^3 .

Source [3]; Lesson 1, video 8; H&P §2.1.

▷ メモリウォール

Source Lesson 1, videos 9–10.

Caution “Faster” without a workload and a power/cost budget is not a meaningful design goal.

Source Lesson 1, videos 11–13; H&P §1.5.

▷ 動的電力

C grows with chip size and transistor count; α depends on the workload and on how much logic is left idle.

- Doubling the frequency at constant voltage doubles the power; the same speedup obtained by doubling the transistor count (more parallel hardware) costs power only through the larger C .

Example 1.2. Reducing the supply voltage by 15 % while keeping C , f and α fixed reduces dynamic power to $0.85^2 \approx 0.72$ of its original value, i.e. by about 28 %.

1.4.2 Static power

The power consumed even when no signal switches is called *static power* (leakage power); it arises because transistors are imperfect switches and leak current when off. Leakage grows as the supply voltage—and with it the transistor threshold voltage—is reduced. There is therefore a trade-off: lowering V reduces dynamic power but increases static power, and the total $P = P_{\text{dyn}} + P_{\text{static}}$ has a minimum at some intermediate voltage (Figure 1.2).

Key point — Power

Dynamic power $\propto CV^2 f \alpha$ favors low voltage; static power penalizes it. Modern designs sit near the minimum of the sum and get further gains from reducing C , α and f rather than V .

1.5 Fabrication cost

Chips are manufactured on circular silicon *wafers*, each of which is cut into many identical rectangular *dies*. The cost of processing a wafer is roughly fixed, so the cost of a single die is

$$\text{cost per die} = \frac{\text{cost per wafer}}{\text{dies per wafer} \times \text{yield}}. \quad (1.2)$$

1.5.1 Yield

Not every die works: defects are scattered over the wafer, and a die that contains a defect is usually discarded. The *yield* is the fraction of dies that are functional. Because defects fall at random positions, a larger die is more likely to contain one, so yield *decreases* as die area increases.

1.5.2 Consequences

Combining Equation (1.2) with the yield behaviour gives the central result of this section: halving the die area *more than* halves the cost per die, because it doubles the number of dies per wafer *and* raises the yield. Read together with Moore's law this offers the architect two options each technology generation:

- keep the die area constant and pack in twice the transistors—a more capable chip at roughly the same cost; or
- keep the design constant and let it shrink to half the area—the same chip at well under half the cost.

Example 1.3. Suppose a wafer costs \$5000 and holds 100 dies of a given design with a yield of 50 %: the cost per working die is $5000/(100 \times 0.5) = \100 . A shrink that fits 200 dies per wafer and raises the yield to 70 % lowers the cost to $5000/(200 \times 0.7) \approx \36 , less than half.

Key point — Lesson summary

Architecture is trade-off engineering under three constraints— performance, power and cost—driven by technology trends (Moore's law) and their exceptions (the memory wall). The next lesson makes “performance” precise.

▷ 静的電力

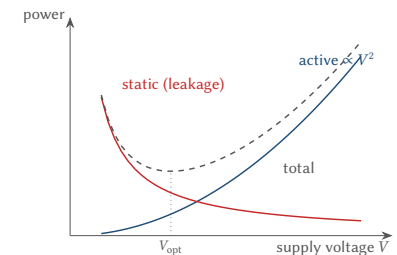


Figure 1.2: Dynamic and static power pull in opposite directions as the supply voltage changes.

▷ ウェハ

▷ ダイ

Source Lesson 1, videos 15–17; H&P §1.6.

▷ 歩留まり

H&P §1.6 gives an empirical model, $\text{yield} \approx 1/(1 + \text{defect density} \times \text{area})^N$.

Metrics and Evaluation

Lecture	Lesson 2, videos 1–25
Textbook	H&P [1] §1.8–1.9
Keywords	latency, throughput, speedup, benchmark, geometric mean, iron law, CPI, Amdahl's law, diminishing returns

Before we can improve performance we must be able to measure it, compare two machines, and predict the effect of a change. This lesson introduces the vocabulary (latency, throughput, speedup), the methodology (benchmarks and how to summarize them) and two analytical tools that recur throughout the course: the iron law of performance and Amdahl's law.

2.1 Performance

“Performance” is ambiguous until we say what we are measuring.

Definition 2.1 (Latency). The *latency* of a task is the time from its start to its completion.

Definition 2.2 (Throughput). The *throughput* of a system is the number of tasks it completes per unit time.

Throughput is *not* simply the inverse of latency. When tasks overlap (pipelining, Lesson 3) or run in parallel (multiprocessors), a system can complete many tasks per second even though each individual task takes a long time.

Example 2.3. A processor that takes 10 s for each of two programs run one after the other has latency 10 s per program and throughput 0.1 programs/s. A second processor that takes 15 s per program but runs two at once has a worse latency yet a throughput of $2/15 \approx 0.13$ programs/s.

2.2 Comparing performance

To say that machine X is “ N times faster” than machine Y we use the *speedup*:

$$N = \frac{\text{speed}(X)}{\text{speed}(Y)} = \frac{\text{latency}(Y)}{\text{latency}(X)} = \frac{\text{throughput}(X)}{\text{throughput}(Y)}. \quad (2.1)$$

Note that latency appears *inverted*: a shorter time means a higher speed. A speedup greater than 1 means X is better; a speedup below 1 means it is worse (a “speedup of 0.8” is a slowdown). Speedups compose multiplicatively: if X is $2\times$ faster than Y and Y is $3\times$ faster than Z , then X is $6\times$ faster than Z .

2.3 Measuring performance

The ideal measurement runs the user's actual workload on the actual machine. This is rarely possible—the machine may not exist yet, and the workload is usually unknown—so we resort to *benchmarks*: programs, together with their input data, that a community agrees to use as a proxy for real workloads. A collection of such programs is a *benchmark suite*.

2.3.1 Types of benchmarks

From most to least representative:

Real applications complete programs with realistic inputs. Most representative, but hardest to set up, and impossible on a simulator or an incomplete prototype.

Source Lesson 2, videos 2–3; H&P §1.8.

▷ レイテンシ

▷ スループット

Caution Throughput = $1/\text{latency}$ holds only when tasks are processed one at a time, strictly one after another.

▷ 高速化率

Source Lesson 2, videos 4–7.

Tip Always keep the ratio in the form old time/new time; most sign errors in this course come from inverting it.

▷ ベンチマーク

Source Lesson 2, videos 8–10; H&P §1.8.

Kernels the most time-consuming part of a real application (e.g. a matrix multiply), extracted so that it can be run on its own. Easier to run; still reflects the real work.

Synthetic benchmarks programs written to *behave like* real applications (similar mix of instructions and memory accesses) while being simpler to run. Useful early in a design.

Peak performance the theoretical maximum rate the hardware can sustain (e.g. instructions per second if every unit were busy). A marketing number that no real program reaches.

Note 2.4. Peak performance is not a benchmark result at all; it is an upper bound that says nothing about how close real programs get to it.

2.3.2 Benchmark standards

Because a vendor could pick benchmarks that flatter its own product, independent organizations define standard suites. SPEC (Standard Performance Evaluation Corporation) publishes CPU, graphics and server suites; TPC does the same for transaction-processing workloads; EEMBC for embedded systems. Members of these organizations agree on the programs, inputs and reporting rules.

Source Lesson 2, video 12; [4].

2.4 Summarizing performance

A suite yields one number per program; we usually want one number per machine.

- **Execution times** may be combined with the ordinary arithmetic mean: the average time is well defined, and the ratio of the average times of two machines equals the speedup of the total runtime.
- **Speedups** (ratios) must *not* be averaged arithmetically. The arithmetic mean of ratios depends on which machine is the reference, and it does not equal the ratio of the mean times. Use the *geometric mean*:

$$\bar{S} = \left(\prod_{i=1}^n S_i \right)^{1/n}. \quad (2.2)$$

Example 2.5. Program A takes 10 s on machine X and 20 s on Y; program B takes 20 s on X and 10 s on Y. The per-program speedups of X over Y are 2 and 0.5. The arithmetic mean, 1.25, suggests X is faster; computed with Y as the reference the arithmetic mean is also 1.25, suggesting Y is faster—a contradiction. The geometric mean is $\sqrt{2 \times 0.5} = 1$ either way, correctly reporting that the machines are equivalent on this suite.

Key point

Average *times* arithmetically; average *speedups* geometrically. The geometric mean is the only mean for which the mean of the ratios equals the ratio of the means.

▷ 幾何平均

Source Lesson 2, videos 13–14; H&P §1.8.

2.5 The iron law of performance

The *iron law* decomposes CPU time into three factors that map onto the different parts of a computer system:

$$\text{CPU time} = \underbrace{\# \text{instructions}}_{\text{IC}} \times \underbrace{\frac{\text{cycles}}{\text{instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{seconds}}{\text{cycle}}}_{T_{\text{clk}}}. \quad (2.3)$$

Instruction count (IC) determined by the algorithm, the compiler and the instruction set (ISA).

▷ 性能の鉄則

Source Lesson 2, videos 15–18; H&P §1.9.

Cycles per instruction (CPI) determined by the ISA and by the processor's microarchitecture (pipelining, out-of-order execution, caches, ...). Most of this course is about lowering CPI.

Clock cycle time (T_{clk}) determined by the microarchitecture (how much logic sits between two latches), the circuit design and the fabrication technology.

The value of the law is that it exposes trade-offs: a change that reduces one factor often increases another. A deeper pipeline lowers T_{clk} but raises CPI; a richer ISA lowers IC but may raise CPI or T_{clk} .

2.5.1 Unequal instruction times

When different instruction classes take different numbers of cycles, the CPI is a weighted average. With n classes,

$$\text{CPU time} = \left(\sum_{i=1}^n \text{IC}_i \times \text{CPI}_i \right) \times T_{\text{clk}}, \quad (2.4)$$

where IC_i and CPI_i are the count and cycles per instruction of class i .

Example 2.6. A program executes 10^9 instructions on a 2 GHz processor: 60 % are ALU operations (1 cycle), 30 % loads/stores (2 cycles) and 10 % branches (3 cycles). The average CPI is $0.6 \cdot 1 + 0.3 \cdot 2 + 0.1 \cdot 3 = 1.5$, so the CPU time is $10^9 \times 1.5 \times 0.5 \text{ ns} = 0.75 \text{ s}$.

2.6 Amdahl's law

Amdahl's law predicts the overall speedup obtained by speeding up only *part* of the execution.

Definition 2.7 (Amdahl's law). If an enhancement speeds up a fraction f_{enh} of the *original* execution time by a factor s_{enh} , the overall speedup is

$$S_{\text{overall}} = \frac{1}{(1 - f_{\text{enh}}) + \frac{f_{\text{enh}}}{s_{\text{enh}}}}. \quad (2.5)$$

Example 2.8. Floating-point instructions account for 40 % of the run time. A new FP unit makes them 4× faster. The overall speedup is $1/(0.6 + 0.4/4) = 1/0.7 \approx 1.43$. Making the FP unit infinitely fast would only reach $1/0.6 \approx 1.67$.

2.6.1 Implications

Letting $s_{\text{enh}} \rightarrow \infty$ in Equation (2.5) gives the upper bound $S_{\text{overall}} \leq 1/(1 - f_{\text{enh}})$. Two design principles follow.

Key point – Make the common case fast

A modest speedup of a large fraction of the time beats a huge speedup of a small fraction. Optimize what the program actually spends its time on.

The lecture also states the converse, jokingly called “Lhadma's law”: *do not sacrifice the uncommon case too much* when speeding up the common one. If the 10 % that is left untouched by an optimization is made 10× slower as a side effect, it now takes as long as the whole original program, and the overall speedup becomes a slowdown.

2.6.2 Diminishing returns

Repeatedly improving the same part of the execution yields *diminishing returns*: once that part has shrunk, it is no longer the common case, and the remaining time is dominated by everything else. After each improvement the architect should re-measure and ask what the common case is *now*.

Caution Never evaluate a change by a single factor. “Higher clock frequency” is not “faster” unless CPI and IC are unchanged.

▷ Amdahl の法則

Source [5]; Lesson 2, videos 19–22; H&P §1.9.

Caution f_{enh} is a fraction of the *original* time, not of the time after the change, and not a fraction of the instruction count.

Source Lesson 2, videos 21–24.

▷ 収穫逡減

Example 2.9. Starting from the previous example (FP = 40 % of time), the first 2× FP speedup gives $S = 1/(0.6 + 0.2) = 1.25$. A second 2× FP speedup on the new machine, where FP is now only $0.2/0.8 = 25\%$ of the time, gives just $1/(0.75 + 0.125) \approx 1.14$.

Key point – Lesson summary

Measure with latency or throughput as appropriate; compare with speedups and summarize them with the geometric mean. The iron law (Equation (2.3)) attributes time to IC, CPI and clock period; Amdahl's law (Equation (2.5)) bounds what any partial improvement can achieve.

Pipelining

Lecture	Lesson 3, videos 1–18
Textbook	H&P [1] Appendix C.1–C.3
Keywords	pipeline, stage, pipeline CPI, stall, flush, control dependence, data dependence, RAW, WAR, WAW, hazard, forwarding, pipeline depth

Pipelining is the first and most fundamental technique for raising throughput: instead of finishing one instruction before starting the next, the processor overlaps the execution of several instructions, each in a different stage of completion. This lesson reviews how a pipelined processor works, quantifies its ideal performance, and then studies the *dependences* between instructions that prevent the ideal from being reached.

3.1 The idea of pipelining

Executing an instruction involves a sequence of steps that use different pieces of hardware. If the steps are arranged as *stages* separated by latches, then as soon as instruction I_1 leaves stage 1 for stage 2, instruction I_2 can enter stage 1. With k stages, up to k instructions are in flight at once.

The consequences for the two metrics of Lesson 2 are different:

- **Throughput** improves by up to a factor of k : in steady state one instruction completes every stage time instead of every k stage times.
- **Latency** of an individual instruction does *not* improve. It is the same total work, and the latches between stages add a small overhead, so per-instruction latency slightly increases.

3.1.1 Pipelining in a processor

The classic five-stage pipeline (Figure 3.1) is used throughout the course as the reference design.

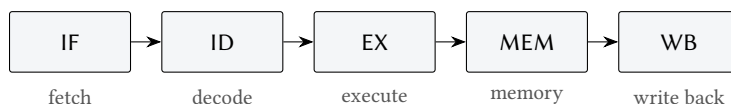


Figure 3.1: The five-stage instruction pipeline.

IF *instruction fetch*: read the instruction at the program counter (PC) from the instruction memory and increment the PC.

ID *instruction decode* and register read: determine the operation and read the source operands from the register file.

EX *execute*: perform the ALU operation, or compute the memory address for a load/store, or evaluate a branch condition.

MEM *memory*: access data memory for loads and stores.

WB *write back*: write the result into the register file.

Figure 3.2 shows five independent instructions flowing through the pipeline. After the first four cycles fill the pipeline, one instruction completes per cycle.

3.2 Pipeline CPI

In the ideal case of Figure 3.2 the pipeline completes one instruction per cycle, so the CPI of the iron law (Equation (2.3)) is 1. Two effects raise it above 1.

▷ ステージ

Source Lesson 3, videos 2–3; H&P §C.1.

Real processors have 10–20 stages; the five-stage model captures every issue that matters here.

Source Lesson 3, video 6; H&P §C.1.

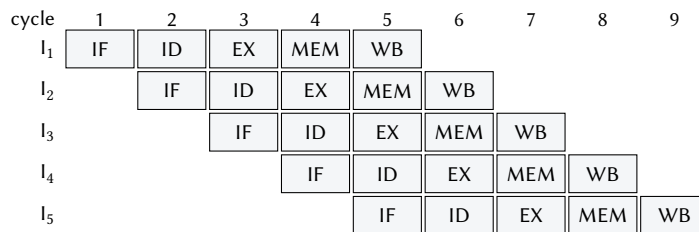


Figure 3.2: Instruction timing in a five-stage pipeline: from cycle 5 on, one instruction completes every cycle.

- *Fill and drain.* The first instruction takes k cycles before anything completes. For a program of n instructions the total is $k+(n-1)$ cycles, i.e. $\text{CPI} = (k+n-1)/n \rightarrow 1$ as n grows. For real programs (n in the billions) this is negligible.
- *Stalls.* Whenever the pipeline cannot advance an instruction to the next stage, a cycle passes without an instruction completing. These cycles are what actually determine the CPI of a pipelined processor:

$$\text{CPI} = 1 + \frac{\text{stall cycles}}{\text{instruction}}. \quad (3.1)$$

3.3 Stalls and flushes

Two mechanisms lose cycles in a pipeline.

Definition 3.1 (Stall). A *stall* holds an instruction (and everything behind it) in its current stage for one or more cycles while the instructions ahead of it continue. A bubble—an empty slot—travels down the pipeline in its place.

Definition 3.2 (Flush). A *flush* discards instructions that have already been fetched into the pipeline because they should not have been executed at all, typically because they follow a branch whose outcome was guessed wrongly.

A stall of c cycles costs c cycles; a flush of m instructions costs m cycles, since each flushed instruction occupied a slot in which a useful instruction could have completed.

3.4 Control dependences

A *control dependence* exists when whether an instruction executes at all depends on the outcome of an earlier branch. In the five-stage pipeline the branch condition and target are known only at the end of EX. By then the two following instructions have already been fetched (Figure 3.3); if the branch is taken they must be flushed and the fetch restarted at the target.

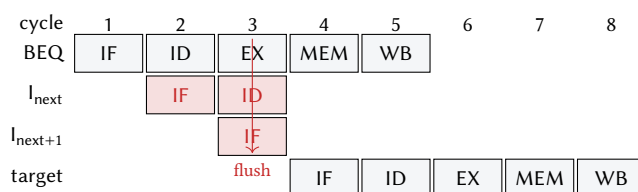


Figure 3.3: A taken branch resolved in EX wastes two fetch slots.

The cost of this on the CPI depends on how often it happens. Typical figures used in the lecture are that 20 % of instructions are branches and 60 % of branches are taken; with the simple policy of always fetching the fall-through instruction (“predict not taken”), only taken branches cause a flush:

$$\text{CPI} = 1 + 0.2 \times 0.6 \times 2 = 1.24. \quad (3.2)$$

Source Lesson 3, videos 7–8; H&P §C.2.

▷ ストール

▷ フラッシュ

▷ 制御依存

Source Lesson 3, videos 9–10; H&P §C.2.

Example 3.3. In a deeper pipeline where the branch is resolved in stage 10, each taken branch flushes nine instructions. With the same branch statistics the CPI becomes $1 + 0.2 \times 0.6 \times 9 = 2.08$ —the processor completes less than one instruction every two cycles. This is why branch prediction (Lesson 4) matters more the deeper the pipeline.

3.5 Data dependences

A *data dependence* exists between two instructions when they access the same register or memory location and at least one of them writes it. There are three kinds, named by the order of the accesses in program order:

RAW (read after write) the later instruction *reads* a value that the earlier one *writes*.

Also called a *true* or *flow* dependence: the data really has to flow from one to the other.

WAR (write after read) the later instruction *writes* a location that the earlier one *reads*. An *anti* dependence.

WAW (write after write) both instructions *write* the same location; the later write must be the one that survives. An *output* dependence.

WAR and WAW are called *false dependences* or *name dependences*: no data moves between the instructions; they merely reuse the same register name. They can be removed by renaming (Lesson 6). A RAW dependence cannot.

Example 3.4. I1: `ADD R1, R2, R3` ; `R1 <- R2 + R3`
 I2: `SUB R4, R1, R5` ; RAW on R1 with I1
 I3: `MUL R1, R6, R7` ; WAW on R1 with I1, WAR on R1 with I2

3.6 Dependences and hazards

A dependence is a property of the *program*; whether it causes a problem depends on the *pipeline*.

Definition 3.5 (Hazard). A *hazard* is a situation in which a particular pipeline would violate a dependence if it continued unmodified—the dependent instruction would read a stale value or execute when it should not.

In the five-stage pipeline:

- A **RAW** dependence is a hazard only if the instructions are close enough. Registers are written in WB and read in ID, so an instruction depends hazardously on the previous one and the one before it, but an instruction three or more slots behind reads the register after it has been written.
- **WAR** and **WAW** dependences never cause hazards here, because instructions read registers in ID and write in WB *in program order*; a later instruction cannot overtake an earlier one. They become hazards in out-of-order pipelines (Lesson 6).
- **Control** dependences are always hazards, because the pipeline fetches instructions after the branch before the branch has resolved.

Key point

Dependences are decided by the program; hazards by the pipeline. The same program has more hazards in a deeper or out-of-order pipeline.

3.7 Handling hazards

Once a hazard is detected, the pipeline has three options.

Tip Cycles lost to control dependences =
 (fraction of instructions that are branches)
 $\times$ (fraction mispredicted) $\times$ (instructions
 flushed per misprediction).

▷ データ依存

Source Lesson 3, videos 11–12; H&P §C.2.

▷ 偽の依存

RAR—two reads of the same register—is not a dependence: order does not matter.

Source Lesson 3, videos 13–14; H&P §C.2.

▷ ハザード

With a register file that writes in the first half of the cycle and reads in the second half, the distance shrinks by one.

Source Lesson 3, videos 15–16; H&P §C.2.

1. **Flush** the dependent instructions—for control hazards, where the wrong instructions were fetched.
2. **Stall** the dependent instruction until the value it needs has been written—always correct, but costs cycles.
3. **Fix** the value by *forwarding* (bypassing): route the result directly from the stage that produced it to the stage that needs it, without waiting for it to reach the register file (Figure 3.4).

▷ フォワーディング

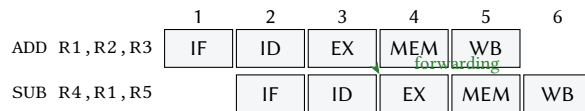


Figure 3.4: Forwarding: the ALU result of ADD is fed straight into the EX stage of SUB, avoiding a stall.

Forwarding removes most RAW stalls but not all of them: a value cannot be forwarded *backwards in time*. The result of a load is only available at the end of MEM, whereas the next instruction needs it at the start of its own EX, which is the same cycle. One stall cycle is therefore unavoidable for a *load-use* pair even with full forwarding.

Example 3.6.

```

    LW  R1, 0(R2)      ; R1 available at end of MEM (cycle 4)
    ADD R3, R1, R4      ; needs R1 at start of EX (cycle 4): stall 1
                           cycle
  
```

With forwarding but no stall the ADD would use a stale R1; without forwarding it would stall two cycles.

3.8 How many stages?

If pipelining a k -stage design gives up to $k\times$ the throughput, why not use hundreds of stages? Three effects pull in different directions as the depth increases:

- **Clock period decreases** (less logic per stage), which lowers T_{clk} in the iron law—the intended benefit. But the latch overhead per stage does not shrink, so the gain flattens out.
- **CPI increases**: more stages means more instructions in flight between a branch and its resolution, hence more instructions flushed per misprediction, and more distance over which RAW hazards need stalls.
- **Power increases**: more latches switch every cycle and the higher frequency raises dynamic power (Equation (1.1)).

Considering performance alone (the product $\text{CPI} \times T_{\text{clk}}$), the optimum is deep—on the order of 30–40 stages. Once power is included, the optimum moves to roughly 10–15 stages, which is where current designs sit.

Key point — Lesson summary

Pipelining raises throughput, not latency; ideal CPI is 1 and every stall or flush adds to it (Equation (3.1)). Control and RAW dependences turn into hazards that cost cycles; forwarding removes most RAW stalls, and branch prediction (next lesson) attacks the control hazards. Pipeline depth trades clock period against CPI and power.

Source Lesson 3, video 17; H&P §C.1, §1.5.

Branches

Lecture	Lesson 4, videos 1–48
Textbook	H&P [1] §3.3, §3.9, §C.2
Keywords	branch prediction, misprediction penalty, predict not taken, branch target buffer, branch history table, two-bit predictor, history-based predictor, PShare, GShare, tournament predictor, hierarchical predictor, return address stack, predecoding

Lesson 3 showed that control dependences are hazards in every pipeline: the processor has to fetch the instructions that follow a branch long before it knows where the branch goes, and every wrong choice flushes work. This lesson studies how the fetch stage makes that choice. Starting from the simplest policy—always continue with the next instruction—it develops the structures used in real processors: branch target buffers, saturating counters, predictors that learn patterns from branch history, combinations of several predictors, and a dedicated predictor for function returns. Along the way it shows why prediction accuracy matters more the deeper the pipeline.

4.1 Branches in a pipeline

Consider a conditional branch `BEQ R1, R2, L` in the five-stage pipeline of Lesson 3. If `R1` equals `R2`, execution continues at label `L`; the branch is then a *taken branch*, and the address of `L` is its *branch target address*.

Otherwise execution continues with the next instruction in memory, at `PC+4`, and the branch is a *not-taken branch*. The pipeline learns which case applies only when the branch has passed through three stages: `IF` fetches it, `ID` reads `R1` and `R2`, and `EX` compares them and computes the target. The branch is *resolved* at the end of `EX` (Figure 4.1).

The fetch stage cannot wait for that. In cycle 2, while the branch is in `ID`, and again in cycle 3, it must load *some* instruction. When the branch resolves, two instructions are already in the pipeline behind it. If they are the instructions that really follow the branch, nothing is lost; if not, they are flushed and two cycles are wasted. Fetching nothing during these cycles would waste the same two cycles unconditionally, so it always pays to fetch the instruction that is most likely to be the right one.

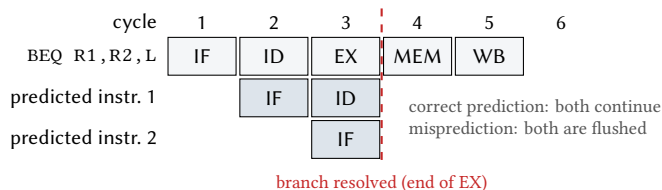


Figure 4.1: A branch in the five-stage pipeline. The two instructions fetched before the branch resolves are predictions; they are flushed if the prediction was wrong.

Definition 4.1 (Branch prediction). The choice, made in the fetch stage, of the address from which the next instruction is fetched, before the current instruction has been decoded or executed, is called *branch prediction*.

Definition 4.2 (Misprediction). A *branch misprediction* occurs when the branch, once resolved, turns out to go elsewhere than predicted. The instructions fetched from the wrong address are flushed and fetching restarts at the correct address.

Source Lesson 4, videos 1–3; H&P §C.2.

- ▷ 成立分岐
- ▷ 分岐先アドレス
- ▷ 不成立分岐

- ▷ 分岐予測
- ▷ 分岐予測ミス

4.1.1 What the predictor knows

At the time of the fetch the only available information is the PC of the instruction being fetched; the instruction itself has not even been decoded yet. From this address alone the predictor has to answer three questions:

1. Is the instruction a branch at all?
2. If it is, is it taken?
3. If it is taken, what is its target address?

The answer is a single number, the next PC: PC+4 for a non-branch or a not-taken branch, the target address for a taken branch.

4.2 Prediction accuracy and performance

With a predictor in place, the only cycles lost to control hazards are those spent on mispredicted paths. Specializing Equation (3.1) to branches gives

$$\text{CPI} = 1 + \frac{\text{mispredictions}}{\text{instruction}} \times \frac{\text{penalty}}{\text{misprediction}}. \quad (4.1)$$

Definition 4.3 (Accuracy and penalty). The *branch prediction accuracy* is the fraction of branches whose outcome is predicted correctly, so that

$$\frac{\text{mispredictions}}{\text{instruction}} = \frac{\text{branches}}{\text{instruction}} \times (1 - \text{accuracy}),$$

assuming that non-branch instructions are always predicted correctly, with PC+4 as the next PC. The *misprediction penalty* is the number of cycles lost per misprediction; in a pipeline that holds one instruction per stage it equals the number of instructions flushed. A branch resolved in stage r has a penalty of $r - 1$ cycles.

The two factors of Equation (4.1) have different origins. The first is set by the predictor (and by how many branches the program executes); the second is set by the pipeline, namely by how late branches are resolved. A better predictor shrinks the first factor, while a deeper pipeline enlarges the second, so the same predictor costs more in a deeper pipeline. A perfect predictor would reach the ideal CPI of 1 regardless of the depth.

Example 4.4. In a program 15 % of the instructions are branches and the predictor is 92 % accurate. Mispredictions per instruction are $0.15 \times 0.08 = 0.012$. If branches are resolved in EX (stage 3, penalty 2), $\text{CPI} = 1 + 0.012 \times 2 = 1.024$. In a pipeline that resolves branches in stage 11 (penalty 10), the same predictor gives $\text{CPI} = 1 + 0.012 \times 10 = 1.12$.

4.3 Predict not taken

The alternative to predicting is to refuse to predict: the fetch stage simply waits until it knows the next PC. Since even whether an instruction is a branch is known only after ID, the wait affects every instruction. A non-branch is followed by a fetch two cycles after its own fetch, and a branch, taken or not, by a fetch three cycles later.

The simplest real predictor is to *predict not taken*: always fetch from PC+4. This needs no memory and no hardware beyond what the fetch stage already has, since

Table 4.1: Cycles from the fetch of an instruction until the correct next instruction is fetched, in the five-stage pipeline.

Instruction	Refuse to predict	Predict not taken
Taken branch	3	3
Not-taken branch	3	1
Not a branch	2	1

Unconditional jumps, calls and returns are branches that are always taken; for them only questions 1 and 3 matter.

Source Lesson 4, videos 4–5; H&P §C.2.

▷ 分岐予測精度

▷ 予測ミスペナルティ

Source Lesson 4, videos 6–8; H&P §C.2.

▷ 不成立予測

it increments the PC anyway. It is right for every non-branch and every not-taken branch, and wrong only for taken branches, which cost three cycles as before. Table 4.1 compares the two policies; predict not taken is never worse.

Its accuracy, however, is poor. Most branches are taken—loop branches jump back most of the time, and unconditional jumps, calls and returns are always taken—so predict not taken mispredicts the majority of branches. With the typical statistics used in Lesson 3 it mispredicts on 12 % of all instructions, giving the CPI of Equation (3.2).

Note 4.5 (Several predictions in flight). In a deep pipeline several branches can be between IF and the resolution stage at the same time, each carrying the prediction made when it was fetched. When a branch resolves as mispredicted, everything fetched after it is flushed, including later branches together with their predictions. A branch fetched on a wrong path therefore never causes a flush of its own: only mispredictions of branches on the correct path cost cycles.

4.4 The need for better prediction

How much a better predictor is worth depends on the pipeline. A processor that handles w instructions per stage flushes w times as many instructions per misprediction as a one-instruction-wide pipeline of the same depth, so the execution time relative to perfect prediction is

$$\frac{T}{T_{\text{perfect}}} = 1 + \frac{\text{mispredictions}}{\text{instruction}} \times (\text{instructions flushed per misprediction}), \quad (4.2)$$

which for $w = 1$ is exactly the CPI of Equation (4.1).

Example 4.6. Let 15 % of the instructions be branches, 70 % of them taken, so that predict not taken is 30 % accurate. Table 4.2 evaluates Equation (4.2) for three predictors and three pipelines: a five-stage pipeline (2 instructions flushed), a pipeline resolving branches in stage 13 (12 flushed), and the same depth with 3 instructions per stage (36 flushed).

Table 4.2: Execution time relative to perfect prediction for the branch statistics of Example 4.6.

Predictor	Mispred./instr.	Stage 3	Stage 13	Stage 13, 3 wide
Not taken (30 %)	0.105	1.21	2.26	4.78
90 % accurate	0.015	1.03	1.18	1.54
99 % accurate	0.0015	1.003	1.018	1.054

Three observations follow. In the shallow pipeline even predict not taken costs only 21 %, and improving from 90 % to 99 % accuracy gains less than 3 %. In the deep pipeline predict not taken more than doubles the execution time, and the same improvement is worth a speedup of $1.18/1.018 \approx 1.16$. In the deep and wide pipeline it is worth $1.54/1.054 \approx 1.46$. Modern processors are both deep and wide, which is why they invest heavily in branch prediction.

Key point

What matters is the *misprediction* rate, not the accuracy. Going from 90 % to 99 % accuracy looks like a small step but removes 90 % of the mispredictions, and the deeper and wider the pipeline, the more each remaining misprediction costs.

4.5 The branch target buffer

Predict not taken uses no information about the instruction being fetched. Better predictors use its *history*: what the instruction at this PC did the previous times it was executed. The next PC is predicted as a function of the PC and of this history. Because the instruction stored at an address does not change, whether it is a branch

Source Lesson 4, videos 9–11; H&P §3.3.

Source Lesson 4, videos 12–15; H&P §3.9; [6].

and where a direct branch jumps to are the same every time; and most branches go in the same direction most of the time.

Definition 4.7 (Branch target buffer). A *branch target buffer* (BTB) is a table, indexed by the PC of the instruction being fetched, that stores the predicted next PC for that instruction.

In IF the BTB is read with the current PC, and its output is used as the next PC. When the branch resolves, the actual next PC is compared with the prediction. On a mismatch the wrongly fetched instructions are flushed and the actual target is written into the BTB entry, so that the next execution of the same instruction is predicted with the new target.

4.5.1 A realistic BTB

A BTB cannot have one entry for every possible instruction address. It must be read within the fetch cycle, in addition to the fetch itself, which limits it to a small table—hundreds to a few thousand entries. The PC is therefore mapped to an entry by a simple hash: a few of its low-order bits. Instructions executed close together in time, such as those of a loop, lie close together in memory; they share their high-order bits but differ in the low-order bits, which spreads them over different entries. Two instructions collide only if they are far apart in the code, and such instructions are less likely to be active at the same time.

With fixed-length 4-byte instructions aligned to word boundaries, the two lowest bits of every PC are zero and carry no information, so a BTB with 2^k entries is indexed by bits $k + 1$ to 2 (Figure 4.2). With variable-length instructions (x86) the index starts at bit 0.

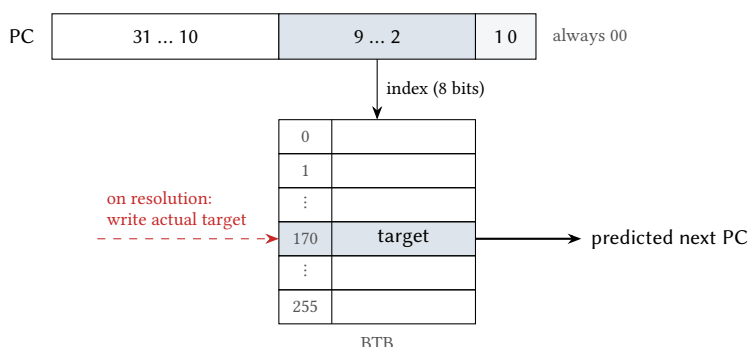


Figure 4.2: A 256-entry BTB indexed by PC bits 9–2. The entry is read in IF and written with the actual target when a branch resolves.

Example 4.8. In a 256-entry BTB, the instruction at 0x004012A8 uses bits 9–2 of its PC: $(0x12A8 \gg 2) \& 0xFF = 0x4AA \& 0xFF = 0xAA$, i.e. entry 170. The next instruction, at 0x004012AC, uses entry 171; the instruction at 0x004016A8, 256 instructions further on, shares entry 170.

4.6 Direction prediction

A BTB alone answers the direction question implicitly: an entry that holds PC+4 means “not taken”. This spends a full target address on every non-branch and every not-taken branch. It is cheaper to split the prediction: a *direction predictor* decides whether the instruction is a taken branch, and the BTB is consulted only when it says so.

Definition 4.9 (Branch history table). A *branch history table* (BHT) is a direction predictor in the form of a table indexed by low-order PC bits. In its simplest form each entry is a single bit holding the outcome of the last execution of the branch:

▷ 分岐先バッファ

Source Lesson 4, videos 16–21; H&P §C.2.

▷ 方向予測器

▷ 分岐履歴テーブル

0 selects PC+4 as the next PC, 1 selects the target from the BTB. This is the *one-bit predictor* (1BP).

When a branch resolves, its BHT entry is set to the actual outcome, and, if the branch was taken, the target is written into the BTB (Figure 4.3). This organization has three advantages. The BHT entry of a non-branch instruction is never written, stays 0, and correctly predicts PC+4. The BTB needs entries only for taken branches, which are far fewer than all instructions. And since a BHT entry is a single bit, the BHT can have many more entries than the BTB at the same cost, so that different branches rarely share an entry.

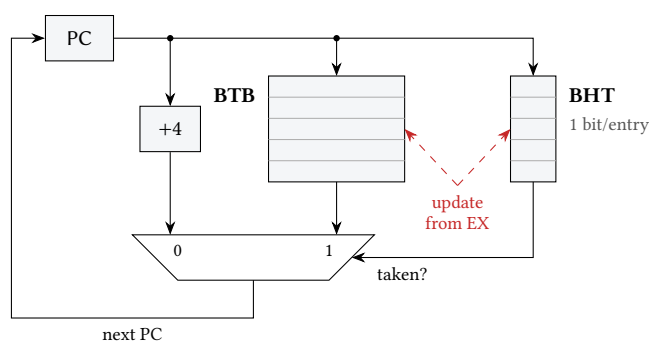


Figure 4.3: Next-PC prediction with a BHT and a BTB. The BHT bit selects between PC+4 and the BTB target; both tables are updated when the branch resolves.

Example 4.10. An inner loop processes three array elements each time it is entered:

```
0x0400 Inner: LW    R2, 0(R3)    ; load element
0x0404      ADD    R5, R5, R2    ; add to sum
0x0408      ADDI   R3, R3, 4     ; next element
0x040C      ADDI   R1, R1, -1    ; decrement count
0x0410      BNE    R1, R0, Inner ; T, T, N per run
```

With a 64-entry BHT (index bits 7–2) the five instructions use BHT entries 0 to 4; the BNE uses entry 4. With an 8-entry BTB (bits 4–2) the BNE uses BTB entry 4, which it shares with the instruction at 0x03F0, eight instructions earlier. If that instruction is not a branch, its BHT entry (entry 60) stays 0, so the BTB entry is used only by the branch. Table 4.3 follows the BNE over two runs of the loop, starting with all tables cleared.

Table 4.3: One-bit prediction of the BNE of Example 4.10 (BHT entry 4) over two runs of the loop.

Execution	1	2	3	4	5	6
BHT[4] before	0	1	1	0	1	1
Predicted next PC	0x414	0x400	0x400	0x414	0x400	0x400
Outcome	T	T	N	T	T	N
Correct?	no	yes	no	no	yes	no

The first execution also writes 0x400 into BTB entry 4; from then on the target is always right, but the direction is wrong in four of the six executions.

4.7 The two-bit predictor

The one-bit predictor works well for branches that always go the same way, but it has two weaknesses.

- A branch that almost always goes one way suffers *two* mispredictions per exception: one on the exception itself, and one on the next normal outcome, because the exception has flipped the bit.

▷ 1 ビット予測器

Source Lesson 4, videos 22–26; H&P §C.2; [7].

- A branch whose direction changes often is predicted badly. An alternating branch (T, N, T, N, ...) is mispredicted every time, and the short loop of Table 4.3 two times out of three.

Definition 4.11 (Two-bit predictor). A *two-bit predictor* (2BP, also called a two-bit counter, 2BC) stores in each BHT entry a two-bit counter that is incremented when the branch is taken and decremented when it is not taken.

The counter is a *saturating counter*: it stays at 11 when incremented and at 00 when decremented. Its more significant bit is the *prediction bit*, taken if 1. The less significant bit is the *hysteresis bit*. Together with the prediction bit it records how strongly the counter believes its prediction: the counter is in a strong state when the two bits are equal and in a weak state when they differ, so the four states are strong not taken (00), weak not taken (01), weak taken (10) and strong taken (11) (Figure 4.4). From a strong state, two consecutive outcomes against the prediction are needed to change it, so a single exception costs a single misprediction.

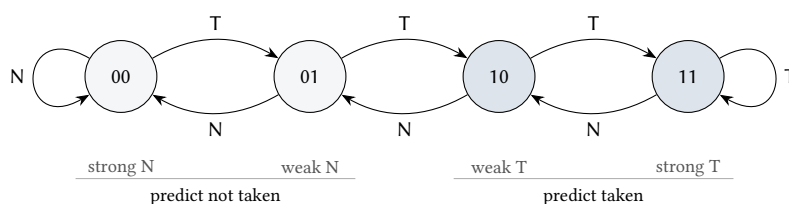


Figure 4.4: State diagram of the two-bit saturating counter. T (taken) moves right, N (not taken) moves left; the upper bit is the prediction.

Table 4.4: The loop branch of Example 4.10 over three runs of the loop with a two-bit counter starting at 00. Mispredictions are marked ×.

Execution	1	2	3	4	5	6	7	8	9
Outcome	T	T	N	T	T	N	T	T	N
Counter before	00	01	10	01	10	11	10	11	11
Prediction	N	N	T	N	T	T	T	T	T
2BP	×	×	×	×		×			×
1BP (for comparison)	×		×	×		×	×		×

Table 4.4 shows the effect on the short loop. Once the counter has reached strong taken (11, before execution 6), a single loop exit only moves it down to weak taken (10) and the next taken outcome restores it, so only the exit is mispredicted: one misprediction per run of the loop instead of two.

4.7.1 Initialization and more bits

A counter initialized to a strong state costs two mispredictions if the branch actually goes the other way; one initialized to a weak state costs only one. The weak states, however, are also where the counter is most vulnerable: an alternating branch T, N, T, N, ... that starts in 01 moves between 01 and 10 and is mispredicted every time, whereas starting in 00 it is mispredicted only on every other execution. Neither choice is best for every branch: each initial state has outcome sequences for which it does worse. Counters are therefore simply reset to the simplest state, 00.

Counters with three or more bits add hysteresis: more consecutive exceptions are needed to change the prediction. This helps only for branches whose exceptions come in runs, it makes the counter slower to follow a branch that really changes its behaviour, and it costs more bits. The step from one to two bits is a large improvement; further bits gain little, which is why the two-bit counter is the standard building block.

▷ 2ビット予測器

▷ 飽和カウンタ

No predictor is immune: for any predictor there is an outcome sequence that always goes against its current prediction.

4.8 History-based predictors

Some branches follow a regular pattern that counters cannot capture, because counters only track which direction is more frequent. A branch that tests whether a loop index is even alternates T, N, T, N; the loop branch of Example 4.10 repeats T, T, N. Both are perfectly predictable once the most recent outcomes are known.

Definition 4.12 (History-based predictor). A *history-based predictor* records the most recent outcomes of a branch (its history) and uses them to select one of several counters, each of which learns what the branch does after one particular history.

4.8.1 One-bit history

The simplest version stores in each BHT entry one history bit, the last outcome of the branch, and two two-bit counters: one used when the last outcome was N, one when it was T. To predict, the history bit selects a counter and the counter's upper bit is the prediction. To update, the selected counter is incremented or decremented with the actual outcome, and the outcome becomes the new history bit. An entry needs $1 + 2 \times 2 = 5$ bits.

For the alternating branch, the counter selected after N sees only T and the counter selected after T sees only N. After each has been trained, the branch is predicted correctly every time. For the pattern T, T, N this is not enough: after a T the branch is sometimes taken and sometimes not, so the counter selected after T keeps mispredicting.

4.8.2 Two-bit and N -bit history

With two history bits and four counters (10 bits per entry, Figure 4.5), each of the four possible histories NN, NT, TN and TT has its own counter. For T, T, N the history TT is always followed by N, while TN and NT are always followed by T. Table 4.5 traces the loop branch: after the counters have been trained, every execution is predicted correctly. The counter for NN is never used again.

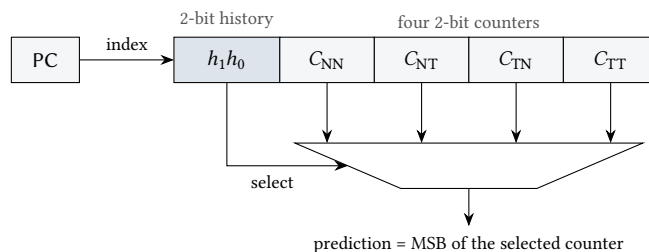


Figure 4.5: An entry of a two-bit history predictor. The history bits select one of four two-bit counters; after the branch resolves, the selected counter is updated and the outcome is shifted into the history.

Table 4.5: Two-bit history predictor on the loop branch of Example 4.10, starting with history NN and all counters at 00. The history is written oldest outcome first; mispredictions are marked ×.

Execution	1	2	3	4	5	6	7	8	9	10	11	12
History	NN	NT	TT	TN	NT	TT	TN	NT	TT	TN	NT	TT
Counter used	00	00	00	00	01	00	01	10	00	10	11	00
Prediction	N	N	N	N	N	N	N	T	N	T	T	N
Outcome	T	T	N	T	T	N	T	T	N	T	T	N
Mispredicted	×	×		×	×		×					

Definition 4.13 (N -bit history predictor). An *N -bit history predictor* keeps the last N

Source Lesson 4, videos 27–33; H&P §3.3; [8].

▷ 履歴ベース予測器

▷ N ビット履歴予測器

outcomes of each branch and 2^N two-bit counters per entry, one for each possible history. An entry costs

$$N + 2 \cdot 2^N \text{ bits.} \quad (4.3)$$

After training, an N -bit history predictor predicts every repeating pattern whose period is at most $N + 1$, because within such a pattern the last N outcomes determine the next one. The price grows exponentially with N , and most of it is wasted: a branch with period p uses at most p of its 2^N counters. Longer histories also take longer to train, since every counter that is used has to be trained separately.

Example 4.14. A loop that always runs 10 iterations has a branch with the pattern TTTT TTTT N, of period 10. Predicting it perfectly requires $N = 9$, i.e. $9 + 2 \cdot 512 = 1033$ bits per BHT entry, of which the branch uses 10 of the 512 counters. A BHT of 1024 such entries takes more than a megabit.

Key point

History lets a predictor learn patterns, not only biases. Giving every branch its own set of 2^N counters, however, makes long histories prohibitively expensive, and nearly all of those counters are never used.

4.9 Shared counters: PShare and GShare

The waste disappears if the counters are not private to a branch. The BHT keeps only an N -bit history per entry, and all branches share one large table of two-bit counters.

Definition 4.15 (Pattern history table). A *pattern history table* (PHT) is a table of two-bit counters shared by all branches and indexed by a combination of the branch's history and its PC, typically their bitwise XOR.

A branch now occupies only as many counters as it has distinct histories. An always-taken branch has the history T T ... T and uses a single counter, an alternating branch uses two, and the 10-iteration loop branch of Example 4.14 uses ten. Mixing in the PC bits sends different branches with the same history to different counters. Collisions remain possible, but with a large PHT they are rare.

Example 4.16. A BHT of 1024 entries holding 12-bit histories takes 12 288 bits, and a PHT of $2^{12} = 4096$ counters takes 8192 bits: 20 480 bits in total. A 1024-entry predictor with 12-bit history and private counters would need $1024 \times (12 + 2 \cdot 4096) = 8\,400\,896$ bits, about 410 times as much.

4.9.1 Private and global history

The history that indexes the PHT can be chosen in two ways (Figure 4.6).

Definition 4.17 (PShare). The *private branch history* of a branch is the sequence of its own most recent outcomes, kept in its BHT entry. A *PShare predictor* indexes the shared PHT with the private history XORed with the PC.

Definition 4.18 (GShare). The *global branch history* is the sequence of the most recent outcomes of all branches, kept in a single shift register into which every resolved branch shifts its outcome. A *GShare predictor* indexes the shared PHT with the global history XORed with the PC.

PShare is suited to branches with patterns of their own: alternating branches and short loops. GShare is suited to *correlated* branches, whose outcome depends on the outcomes of other branches executed just before.

Source Lesson 4, videos 34–39; H&P §3.3; [9].

▷ パターン履歴テーブル

▷ ローカル分岐履歴

▷ PShare 予測器

▷ グローバル分岐履歴

▷ GShare 予測器

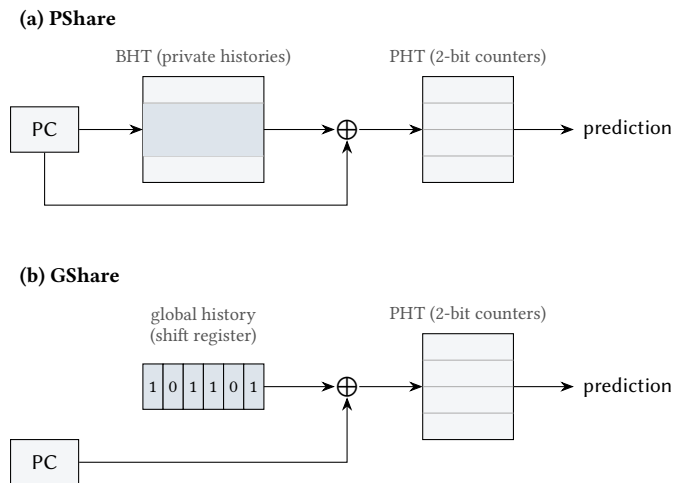


Figure 4.6: Predictors with shared counters. (a) PShare combines the branch’s own history with the PC; (b) GShare combines the global history of all branches with the PC.

Example 4.19. In the fragment below the second branch is taken exactly when the first is not.

```

        BNE  R1, R0, Skip1    ; skip unless R1 == 0
        ADDI R2, R2, 1        ; count zeros
Skip1:  BEQ  R1, R0, Skip2    ; skip if R1 == 0
        ADD  R3, R3, R1      ; sum nonzero values
Skip2:  ...

```

If the value in R1 depends on the data, the private history of either branch looks random, and PShare predicts it poorly. The global history seen by the second branch contains the outcome of the first, so once the PHT counters for these histories have been trained, GShare predicts the second branch perfectly, barring aliasing in the PHT.

Neither is better for all branches. GShare can in principle learn a branch’s own pattern as well, but the outcomes of the other branches executed in between are interleaved in the global history, so the same pattern needs a considerably longer history—and a larger PHT—than in PShare. Since programs contain both kinds of branches, the natural next step is to use both predictors.

4.10 Tournament and hierarchical predictors

Since different branches are best served by different predictors, a processor can combine several predictors and use, for each branch, the one that suits it. The combination can be symmetric or layered.

4.10.1 Tournament predictors

A *tournament predictor* runs two predictors, for example GShare and PShare, side by side and decides separately for each branch which of them to trust.

Definition 4.20 (Meta-predictor). The *meta-predictor* of a tournament predictor is a table of two-bit counters, indexed by the PC, whose counter for a branch predicts which of the two component predictors will be right (Figure 4.7).

Both component predictors are trained on every branch as usual. The meta-predictor counter changes only when the components disagree in correctness: it moves towards the component that was right. When both are right or both are wrong, it stays unchanged.

Source Lesson 4, videos 40–43; H&P §3.3; [9].

▷ トーナメント予測器

▷ メタ予測器

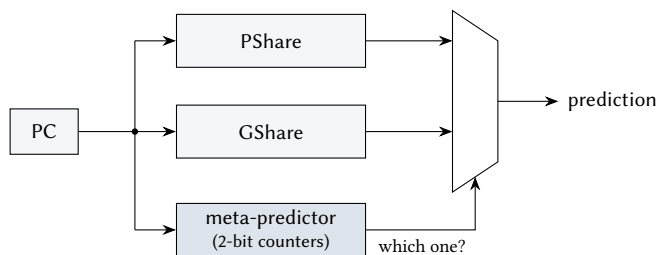


Figure 4.7: A tournament predictor. The meta-predictor selects, per branch, the component predictor whose prediction is used.

4.10.2 Hierarchical predictors

A tournament predictor combines two good—and therefore expensive—predictors and updates both on every branch. Most branches, however, are easy to predict.

Definition 4.21 (Hierarchical predictor). A *hierarchical predictor* combines a cheap predictor, which handles all branches, with one or more better and more expensive predictors that are used and trained only for the branches the cheap predictor handles poorly.

Because the expensive predictor has far fewer branches to learn, it can be much smaller than in a tournament predictor for the same accuracy, or more accurate for the same size.

A typical arrangement has three levels (Figure 4.8). A large table of two-bit counters indexed by the PC predicts every branch. Above it, a small local-history predictor and a small global-history predictor hold entries only for selected branches. Their entries are *tagged* with bits of the PC, so that a lookup can tell whether the branch has an entry. The prediction comes from the highest level with a matching entry. The counter table is always updated, and a local or global entry that matches the branch is updated as well; a branch that the counter table mispredicts is given an entry in the local predictor, and a branch that the local predictor mispredicts is given one in the global predictor. Branches that the counters predict well take entries in the expensive levels only rarely, so those small tables are left for the branches that need them.

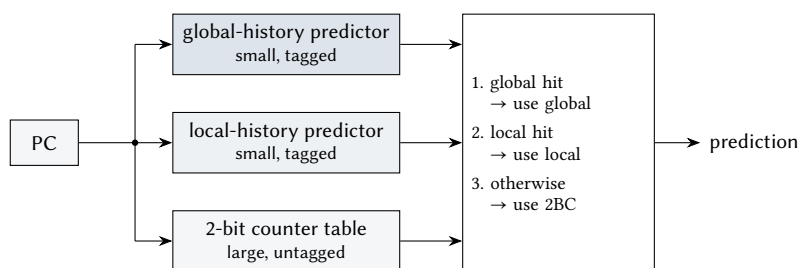


Figure 4.8: A hierarchical predictor. Tagged upper levels override the two-bit counter table for the branches they hold.

Table 4.6 summarizes which kind of predictor handles which kind of branch.

Table 4.6: Branch behaviour and the simplest predictor that handles it well.

Branch behaviour	Predictor
Almost always the same direction	Two-bit counter
Own repeating pattern (alternation, short loop)	Private history (PShare)
Correlated with preceding branches	Global history (GShare)
Mixture of the above in one program	Tournament or hierarchical
Function return	Return address stack (next section)

▷ 階層型予測器

Commercial processors have used predictors of this layered kind, for example Intel's Pentium M.

4.11 The return address stack

A function return (RET) is an unconditional branch, so its direction is trivially predicted as taken. Its target, however, depends on where the function was called from. The BTB remembers only the most recent target, so when a function is called from different places in turn, every return is mispredicted.

Example 4.22. A function is called from two places:

```

0x1000      CALL Func      ; return address 0x1004
...
0x1040      CALL Func      ; return address 0x1044
...
0x2000  Func:  ...
0x2030      RET              ; to 0x1004, then to 0x1044

```

When the first return resolves, the BTB entry of the RET receives 0x1004; that return is itself mispredicted, since the entry was still empty. The second return, which goes to 0x1044, is mispredicted and overwrites the entry. If calls from the two places alternate, every return is mispredicted.

Definition 4.23 (Return address stack). A *return address stack* (RAS) is a small hardware stack in the fetch stage. When a call is fetched, its return address (PC+4) is pushed; when a return is fetched, the top entry is popped and used as the predicted target.

Since calls and returns nest, the RAS predicts returns almost perfectly (Figure 4.9). Being small, it can fill up when calls nest deeply. When a call finds the RAS full, the push wraps around and overwrites the oldest entry. Returns occur in reverse order of the calls, so the entries needed soonest are the most recent ones, and most calls and returns happen deep in the call tree. Overwriting the oldest entry loses only the predictions for returns from the outermost functions, which are rare, whereas discarding the new return address would lose the predictions for the frequent calls at the deepest level.

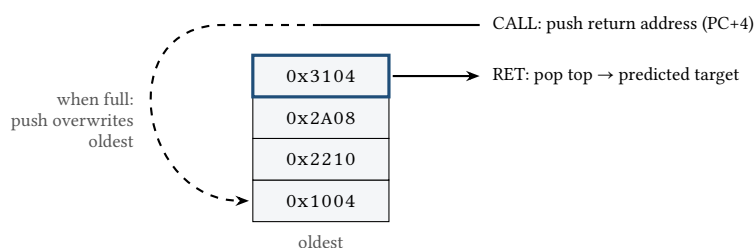


Figure 4.9: A four-entry return address stack. Calls push, returns pop; when the stack is full, a push wraps around and overwrites the oldest entry.

4.11.1 Recognizing a return

The RAS has to be used while the RET is being fetched, before it has been decoded. There are two ways to know that the instruction is a return.

- **Predict it.** The predictor tables store, per PC, whether the instruction is a return; the entry is set the first time the instruction is decoded. Since the instruction at a given address does not change, this prediction is almost always right.
- **Predecode it.** With *predecoding*, the processor partially decodes each instruction when it is brought from main memory into the processor's cache, and stores a few extra bits with the cached instruction, for example whether it is a branch or a return. The fetch stage reads these bits together with the instruction and knows its kind immediately.

Source Lesson 4, videos 44–47; H&P §3.9; [10].

▷ リターンアドレススタック

▷ プリデコード

Key point — Lesson summary

Every fetch is a prediction. A misprediction wastes the work of every instruction it flushes (Equation (4.2)), and the number of flushed instructions grows with pipeline depth and width. Predict not taken is free but inaccurate. A BTB predicts targets and a BHT predicts directions, best with two-bit saturating counters. History-based predictors learn patterns; sharing their counters in a PHT makes long histories affordable, with private history (PShare) for self-repeating branches and global history (GShare) for correlated ones. Tournament and hierarchical predictors combine several predictors, and a return address stack predicts function returns.

Predication

Lecture	Lesson 5, videos 1–12
Textbook	H&P [1] §A.6, §C.2, §H.4, §H.6
Keywords	predication, hard-to-predict branch, if-conversion, conditional move, MOVZ, MOVN, CMOVcc, condition code, break-even misprediction rate, full predication, Itanium, predicate register

The predictors of Lesson 4 guess most branches correctly, but some branches remain hard to predict for any predictor, typically because their direction depends on the data being processed, and in a deep pipeline every misprediction flushes tens of instructions. This lesson studies the alternative for such branches: *predication*, which executes the work of both directions of a branch and keeps only the results of the correct one, so that nothing is left to mispredict. We examine which branches benefit, how a compiler removes a branch by if-conversion, the conditional move instructions this relies on, when the transformation pays off, and instruction sets in which every instruction can be made conditional.

5.1 Prediction and predication

A branch predictor guesses in the fetch stage where a branch goes, and the processor continues to fetch along the guessed path. A correct guess costs nothing. A wrong guess flushes every instruction fetched after the branch, which in a modern processor, several instructions wide and many stages deep, amounts to tens of instructions (Example 4.6). The cost of a branch is thus zero most of the time and large once in a while.

Definition 5.1 (Predication). Executing the work of *both* directions of a branch and throwing away the results of the direction that turns out to be wrong is called *predication*.

Predication has no misprediction penalty, since nothing is predicted, but it always wastes the work of the direction that is not needed: if both directions contain equally many instructions, half of the work is thrown away. Per execution of the branch, predication wastes the instructions of the unneeded direction, while prediction wastes the misprediction penalty multiplied by the probability of a misprediction. Which is smaller depends on the kind of branch (Table 5.1); predication can be the better choice only for a small if-then-else, whose directions contain far fewer instructions than a misprediction flushes.

Table 5.1: Prediction and predication for different kinds of branches.

Branch	Prediction	Predication	Better
Loop	accurate	also runs the code after the loop in every iteration	prediction
Call, return	always taken; BTB and RAS give the target	work on the other path is always wasted	prediction
Large if-then-else	occasionally flushes tens of instructions	wastes a large direction every time	prediction
Small if-then-else	flushes tens of instructions per misprediction	wastes a few instructions every time	predication, if hard to predict

For a small if-then-else the choice depends on how well the branch is predicted. If a misprediction flushes 40 instructions and the unneeded direction contains 4,

Source Lesson 5, videos 1–2; H&P §C.2.

▷ プレディケーション

predication wastes less than prediction as soon as more than one execution in ten is mispredicted. Section 5.4 makes this comparison precise for the code that a compiler actually produces.

Key point

Prediction costs nothing when it is right and tens of instructions when it is wrong; predication always wastes the unneeded direction. Branches are therefore predicted by default, and predication is worth considering only for small if-then-else branches that are hard to predict.

5.2 If-conversion

Predication needs the work of both directions in the form of instructions that are executed one after the other, with no branch between them. Producing such code is the task of the compiler. Consider the following if-then-else, which sums the values below and above a threshold separately and counts the values below it. If the values of v fall on either side of t in no regular order, the branch is hard to predict.

```
if (v < t) {
    lo = lo + v;
    n  = n + 1;
} else {
    hi = hi + v;
}
```

Definition 5.2 (If-conversion). The compiler transformation that removes such a branch by turning the control dependence (Lesson 3) of the statements of both directions into a data dependence on the branch condition is called *if-conversion*.

With ordinary instructions, the converted code (Figure 5.1) executes the statements of both directions unconditionally, writing their results into temporaries, and then uses the condition c as data to select the new value of every variable assigned in either direction. When both directions assign a variable x , the selection chooses between their two temporaries, $x = c ? x1 : x2$; when only one direction assigns it, as for each variable in this example, the other alternative is the old value. The converted code contains no branch.

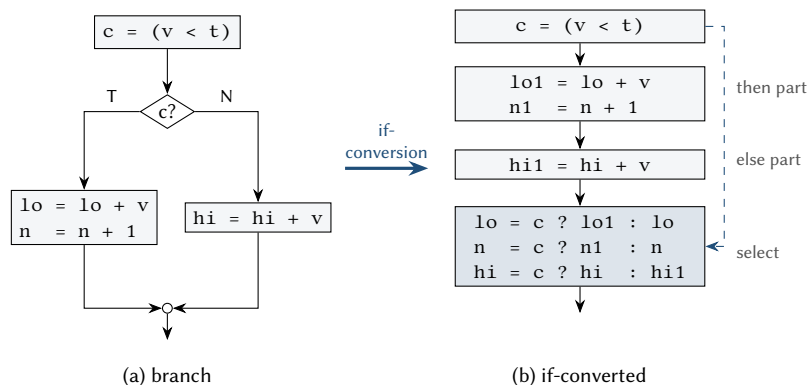


Figure 5.1: If-conversion of a small if-then-else. (a) The original code branches on c . (b) The converted code computes both directions into temporaries and uses c as data to select the results.

The transformation gains something only if the selections can be carried out without a branch. Translated into ordinary branch instructions, each selection is itself an if-then-else on c , as hard to predict as the branch that was removed, so the converted code would still mispredict and in addition execute the work of both directions. Ordinary arithmetic can select without a branch by masking both alternatives with

Source Lesson 5, video 3; H&P §H.4; [11].

▷ if 变换

Caution The unneeded direction also runs: it must not store to memory or fault, e.g. by loading via an invalid pointer.

AND and OR, but that costs several instructions per selection; if-conversion is cheap only if the instruction set can make the effect of an instruction conditional.

5.3 Conditional move instructions

The simplest such instruction makes only a copy conditional.

Definition 5.3 (Conditional move). A *conditional move* is an instruction that copies a value into its destination if a condition holds and leaves the destination unchanged otherwise.

The instruction is executed in either case; only whether it writes depends on the condition. It does not change the flow of control, so there is nothing to predict.

The MIPS instruction set has two conditional moves whose condition is the contents of a register. The instruction *MOVZ* (move if zero) copies register *Rs* into register *Rd* if register *Rt* holds zero, and the instruction *MOVN* (move if not zero) copies it if *Rt* does not hold zero:

```
MOVZ Rd, Rs, Rt      ; if (Rt == 0) Rd <- Rs
MOVN Rd, Rs, Rt      ; if (Rt != 0) Rd <- Rs
```

A selection $x = c ? x_1 : x_2$ becomes two moves into the register of *x*, of which exactly one writes. With *c* in *R3*, *x1* and *x2* in *R1* and *R2*, and *x* in *R4*:

```
MOVN R4, R1, R3      ; if c != 0: x <- x1
MOVZ R4, R2, R3      ; if c == 0: x <- x2
```

The condition register is typically set by a comparison such as *SLT*, which writes 1 if the condition holds and 0 otherwise.

Example 5.4. With *v* in *R1*, *t* in *R2*, and *lo*, *n*, *hi* in *R4*, *R5*, *R6*, the code of Section 5.2 compiles with a branch into

```
SLT  R3, R1, R2      ; R3 <- (v < t)
BEQ  R3, R0, Skip    ; v >= t: go to else part
ADD  R4, R4, R1      ; lo <- lo + v
ADDI R5, R5, 1       ; n <- n + 1
J    Cont            ; skip else part
Skip: ADD R6, R6, R1  ; hi <- hi + v
Cont: ...
```

and, if-converted, into

```
SLT  R3, R1, R2      ; R3 <- (v < t)
ADD  R7, R4, R1      ; then part into R7, R8
ADDI R8, R5, 1
ADD  R9, R6, R1      ; else part into R9
MOVN R4, R7, R3      ; lo <- R7 if v < t
MOVN R5, R8, R3      ; n <- R8 if v < t
MOVZ R6, R9, R3      ; hi <- R9 if v >= t
```

The temporaries *R7*–*R9* are needed because *R4*–*R6* must keep their old values until the moves decide which of them change. Each variable here is assigned in only one direction, so a single move per variable suffices.

Other instruction sets base conditional moves on flags rather than on the contents of a register.

Definition 5.5 (Condition code). A *condition code*, or flag, is a one-bit status value, for example zero, negative, carry or overflow, that arithmetic and comparison instructions set as a side effect and that later instructions test.

The x86 instruction set has a family of conditional moves, *CMOVcc*, in which *cc* names a condition on the condition codes: *CMOVZ* moves if the zero flag is set, *CMOVNZ* if it is clear, *CMOVG* if the last comparison found “greater”, *CMOVL* if it found “less”, and

Source Lesson 5, videos 4–5; H&P §H.4, §A.6.

▷ 条件付きムーブ

▷ MOVZ

▷ MOVN

▷ 条件コード

▷ CMOVcc

so on. A common pattern assigns one alternative unconditionally and lets a single conditional move overwrite it with the other, as in this computation of $m = \max(a, b)$:

```
MOV    EAX, EBX    ; m <- a
CMP    EBX, ECX    ; set flags from a - b
CMOVL  EAX, ECX    ; if a < b: m <- b
```

5.4 Performance of if-conversion

If-conversion is not free: the converted code executes the work of both directions and the conditional moves on every execution. To compare the two versions of a code fragment we count pipeline slots, as in Lesson 3: every executed instruction occupies one slot, and every misprediction wastes as many slots as it flushes instructions. Let the branch version execute n_b instructions per execution on average and mispredict m times per execution with a penalty of P instructions, and let the converted version execute n_c instructions. If-conversion pays off when

$$n_c < n_b + mP \quad \Longleftrightarrow \quad m > \frac{n_c - n_b}{P}. \quad (5.1)$$

The right-hand side is the *break-even misprediction rate*: the extra instructions of the converted code, measured in units of the misprediction penalty. Compared with the rough comparison of Section 5.1, the extra instructions include the conditional moves, less the branch and jump instructions that the conversion removes.

Example 5.6. The branch version of Example 5.4 executes five instructions when the then part applies (SLT, BEQ, ADD, ADDI, J) and three otherwise (SLT, BEQ, ADD). If both parts apply equally often, $n_b = 4$; the converted version always executes $n_c = 7$. Suppose the predictor is 75 % accurate on this branch and $P = 20$. The branch version costs $4 + 0.25 \times 20 = 9$ slots per execution and the converted version 7, a speedup of $9/7 \approx 1.29$ for the fragment. By Equation (5.1) the break-even rate is $m = 3/20 = 0.15$: the branch version is faster only if the predictor is more than 85 % accurate. With $P = 40$ the break-even rate halves to 0.075, an accuracy of 92.5 %.

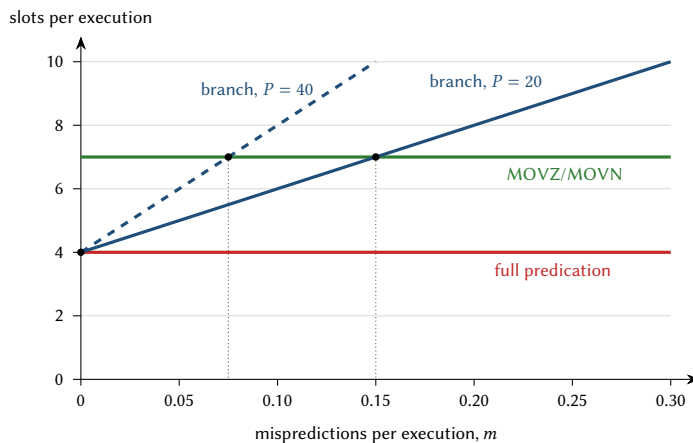


Figure 5.2: Slots per execution of the fragment of Example 5.6 as a function of the misprediction rate. Dots mark the break-even rates; the fully predicated version is discussed in Section 5.6.

Figure 5.2 shows the comparison as a function of m . The cost of the branch version grows linearly with the misprediction rate, with a slope equal to the penalty, while the converted version costs the same for every m . A deeper or wider pipeline raises the penalty and so favours if-conversion; larger then and else parts raise $n_c - n_b$ and favour the branch.

Source Lesson 5, videos 6–7; H&P §C.2.

Key point

If-conversion replaces an occasional large penalty by a constant overhead. It pays off when the mispredictions per execution times the penalty exceed the extra instructions of the converted code (Equation (5.1)).

5.5 Benefits and costs of conditional moves

Removing branches with conditional moves has one benefit and several costs.

- **Benefit:** hard-to-predict branches, and with them their mispredictions, disappear.
- **Compiler support** is needed. The processor executes the code it is given; a branch is removed only if the compiler recognizes a suitable if-then-else and converts it.
- **More registers** are needed: every result is first computed into a temporary.
- **More instructions** are executed: the work of both directions, even on executions where the branch would have been predicted correctly, and the conditional moves that select the results.

Of these, the compiler support, the removal of the branch and the execution of both directions are inherent to predication; the extra registers and the moves are not. They arise because only the move is conditional: every result has to be computed into a temporary so that a conditional move can decide whether to keep it. If the instruction that computes a result could itself be made conditional, neither the temporaries nor the moves would be needed.

Source Lesson 5, video 8; H&P §H.4.

5.6 Full predication

A conditional move is a separate opcode, and the only operation it makes conditional is a copy. The alternative is support in the instruction set for making every instruction conditional.

Definition 5.7 (Full predication). An instruction set provides *full predication* if condition bits are added to every instruction: the instruction takes effect only if its condition holds, and has no effect otherwise.

The IA-64 instruction set of Intel's Itanium processors is an example (Figure 5.3). In most IA-64 instructions, the six least significant bits of the 41-bit instruction hold a *qualifying predicate*: the number of one of 64 one-bit registers, each called a *predicate register*. The instruction is fetched and occupies its pipeline slot in either case, but it takes effect only if the selected predicate register holds 1; otherwise it is annulled: it writes no result and raises no exception. Predicate register 0 always holds 1, so an instruction whose qualifying predicate is 0 is unconditional. The predicate registers are set by compare instructions, which write a pair of predicates with complementary values, one for the condition and one for its negation.

Example 5.8. With full predication, if-conversion guards each statement of both directions directly with the condition or its negation, and the code of Section 5.2 becomes

```

CMPLT P1, P2, R1, R2 ; P1 <- (v < t), P2 <- !P1
(P1) ADD    R4, R4, R1    ; lo <- lo + v if v < t
(P1) ADDI   R5, R5, 1     ; n  <- n + 1  if v < t
(P2) ADD    R6, R6, R1    ; hi <- hi + v if v >= t

```

where the qualifying predicate is written in parentheses before the instruction and CMPLT denotes a compare instruction that sets P1 to 1 and P2 to 0 if R1 is less than R2, and the reverse otherwise. The additions themselves are conditional, so they write directly into the variables, and the compare replaces both the SLT and the branch. Table 5.2 compares the three versions with the numbers of Example 5.6.

Source Lesson 5, videos 9–12; H&P §H.4, §H.6; [12].

▷ 完全ブレディケーション

▷ 述語レジスタ

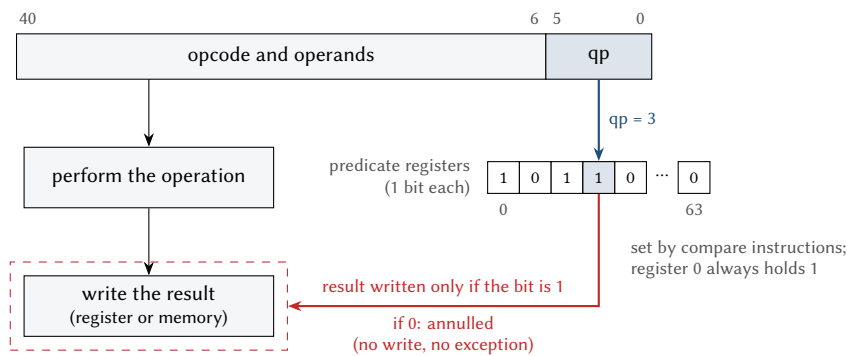


Figure 5.3: Full predication in IA-64. The qualifying predicate (qp) of an instruction selects one of 64 one-bit predicate registers. The instruction passes through the pipeline in either case, but its result is written only if that register holds 1; otherwise the instruction is annulled, with no write and no exception.

Table 5.2: Three versions of the example fragment with a 75 % accurate predictor and $P = 20$.

Version	Instructions	Temporaries	Moves	Mispred.	Slots
Branch	5 or 3	0	0	0.25	9
MOVZ/MOVN	7	3	3	0	7
Full predication	4	0	0	0	4

The fully predicated version executes the compare and the instructions of both directions, and nothing else. Compared with the branch version it executes the unneeded direction but no branch and no jump; here the two balance, and its $n_c = 4$ equals the average n_b of the branch version. Its break-even rate by Equation (5.1) is $m = (4 - 4)/20 = 0$: the branch version is not faster even with perfect prediction, and every misprediction makes it slower (Figure 5.2). In general the fully predicated version executes one instruction fewer than the MOVZ/MOVN version per conditional move, so with M moves its break-even rate is lower by M/P ; if the conditional moves test a register that already holds the condition, they need no compare while full predication still needs one, and the difference is $(M - 1)/P$. By lowering the break-even rate, full predication makes if-conversion pay off also for branches that are predicted too well to justify conditional moves.

Key point – Lesson summary

Branch prediction costs nothing when right and tens of instructions when wrong; predication executes both directions of a branch and never mispredicts, at the price of always doing the unneeded work. Branches are therefore predicted by default, and compilers use predication only for small, hard-to-predict if-then-else branches, not for loops, calls, returns or large if-then-else branches. If-conversion removes such a branch by turning the control dependence into a data dependence on the condition. With ordinary instructions it computes both directions into temporaries and selects the results with conditional moves, such as MOVZ and MOVN in MIPS or CMOVcc on the condition codes in x86. If-conversion pays off when the mispredictions per execution times the penalty exceed its extra instructions (Equation (5.1)); deeper pipelines favour it. Full predication, with a condition on every instruction as in IA-64, avoids the temporaries and moves and so lowers the break-even misprediction rate.

ILP

Lecture	Lesson 6, videos 1–20
Textbook	H&P [1] §3.1, §3.4; [13, 14]
Keywords	RAW dependence, WAW dependence, false dependence, architectural register, physical register, register renaming, register alias table, instruction-level parallelism, ideal processor, instructions per cycle, processor width, in-order execution, out-of-order execution

Branch prediction (Lesson 4) and predication (Lesson 5) remove most of the cycles that control dependences cost a pipeline, which brings its CPI close to the ideal value of 1. Going below 1 requires executing several instructions in the same cycle, and what prevents this are the data dependences between instructions. This lesson shows that only read-after-write dependences are fundamental, that the others are removed by register renaming, and that the remaining dependences define the instruction-level parallelism of a program: the bound against which the instructions per cycle of real processors are measured.

6.1 All instructions in the same cycle

A pipeline that never stalls or flushes completes one instruction per cycle, so its CPI is 1 (Equation (3.1)). A lower CPI means that more than one instruction completes per cycle. In the extreme, all N instructions of a program would be fetched in the same cycle, decoded together in the next cycle, executed together in the cycle after that, and so on (Figure 6.1). A five-stage processor would complete the whole program in 5 cycles, a CPI of $5/N$, which tends to 0 for long programs.

A real program cannot be executed this way, because its instructions use the results of other instructions. Consider the following five instructions, the running example of the first half of this lesson:

```
I1:  ADD R3, R1, R2      ; R3 <- R1 + R2
I2:  SUB R5, R3, R4      ; R5 <- R3 - R4
I3:  AND R6, R1, R4      ; R6 <- R1 AND R4
I4:  OR  R4, R2, R7      ; R4 <- R2 OR R7
I5:  XOR R5, R1, R7      ; R5 <- R1 XOR R7
```

When all five instructions are in the execute stage in the same cycle, I2 computes $R3 - R4$ while I1 is still computing $R3$, so I2 uses the old value of $R3$. Forwarding (Figure 3.4) cannot correct this: it delivers a result computed in one cycle to an instruction that executes in a later cycle, whereas here the result and its use fall into the same cycle. How close a processor can come to the ideal is therefore determined by the data dependences of the program.

6.2 Dependences in same-cycle execution

The read-after-write dependence of I2 on I1 must be respected: I2 can execute no earlier than one cycle after I1, when the result of I1 can be forwarded to it. I3, I4 and I5 read no register that an earlier instruction writes, so they can still execute in the same cycle as I1. Figure 6.2 shows the resulting timing: I2 waits one cycle, and I3 to I5 execute before I2 although they follow it in the program. RAW dependences therefore bound how far the CPI can fall even with unlimited width: if every instruction read the result of its predecessor, one instruction would complete per cycle and the CPI would remain 1.

Executing instructions ahead of an earlier one breaks the write-after-write dependence between I2 and I5. Both write $R5$, and in program order I5 writes last, so after these instructions $R5$ must hold the result of I5; every later instruction that reads $R5$

Source Lesson 6, videos 1–3; H&P §3.1.

cycle	1	2	3	4	5
I1	IF	ID	EX	MEM	WB
I2	IF	ID	EX	MEM	WB
I3	IF	ID	EX	MEM	WB
⋮	⋮	⋮	⋮	⋮	⋮
IN	IF	ID	EX	MEM	WB

Figure 6.1: The ideal: all N instructions pass through the stages in the same five cycles.

Source Lesson 6, videos 4–6; H&P §3.1.

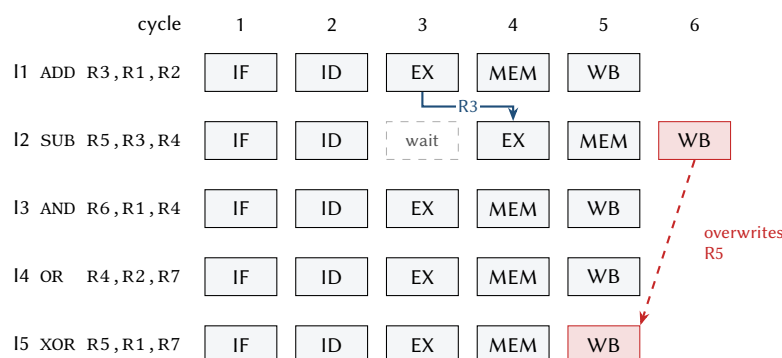


Figure 6.2: Same-cycle execution of the running example. The RAW dependence on R3 delays I2 by one cycle; as a consequence I2 writes R5 after I5, which violates their WAW dependence.

expects that value. In Figure 6.2, however, I5 writes R5 in cycle 5 and the delayed I2 overwrites it in cycle 6. To keep the correct value, I5 would also have to be delayed until after the write of I2, although its computation needs nothing from I2.

A write-after-read dependence is at risk in the same way. I4 writes R4, which I2 reads, and program order requires I2 to use the old value of R4. Once I2 is delayed, the new value produced by I4 already exists when I2 executes, and I2 must be kept from using it. A single RAW delay thus spreads: every later instruction that writes a register the delayed instruction reads or writes must be delayed as well, so that every instruction still sees the register values of its position in the program.

6.3 Removing false dependences

The three kinds of data dependence of Lesson 3 differ in nature. In a RAW dependence the later instruction uses the value computed by the earlier one: the data really flows from one instruction to the other, and no hardware can let the reader execute before the writer. WAW and WAR dependences are false dependences: no value passes between the two instructions. They exist only because the program stores different values in the same register, a consequence of the limited number of registers and of the fact that an instruction executed repeatedly, for example in every iteration of a loop, always writes the same register. In the example, the results of I2 and I5 are unrelated values that happen to be named R5.

False dependences therefore disappear if different values are kept in different places. One way to achieve this is to keep several versions of a register. When I2 and I5 both write R5, the processor stores both results instead of letting one overwrite the other, and each instruction that reads R5 uses the version selected by program order: an instruction located between I2 and I5 uses the result of I2, an instruction after I5 the result of I5. The order in which the two writes actually happen no longer matters, so I5 can execute as early as if I2 did not exist. Likewise, I2 reads the old version of R4 no matter when I4 produces the new one.

The difficulty is the bookkeeping. For every read, the processor must find the right version, the one produced by the closest preceding writer in program order, and it must recognize when a version is no longer needed. Register renaming performs this bookkeeping systematically.

6.4 Register renaming

Renaming separates the registers that a program names from the storage in which the processor keeps their values.

Definition 6.1 (Architectural and physical registers). A register defined by the instruction set, and therefore named in programs (for example R0 to R31), is an *architectural register*. A storage location in which the processor actually keeps a

Source Lesson 6, videos 7–8; H&P §3.1.

Source Lesson 6, videos 9–11; H&P §3.1, §3.4; [13].

▷ アーキテクチャレジスタ

register value is a *physical register*. A processor that renames has many more physical registers than architectural ones.

Definition 6.2 (Register renaming). Rewriting each instruction, as it is decoded, so that it reads and writes physical instead of architectural registers, with a newly allocated physical register for every result, is called *register renaming*.

Definition 6.3 (Register alias table). The *register alias table* (RAT) has one entry per architectural register. The entry names the physical register that holds the most recent value of that architectural register, in program order, among the instructions renamed so far.

Each instruction is renamed in three steps (Figure 6.3):

1. For each source register, read its RAT entry; the renamed instruction reads that physical register.
2. Allocate a free physical register, one that holds no value still needed, for the result; the renamed instruction writes it.
3. Overwrite the RAT entry of the destination register with the new physical register, so that later instructions reading this architectural register use the new result.

The sources must be looked up before the destination entry is overwritten; otherwise an instruction such as `ADD R1, R1, R2` would read its own result instead of the previous value of R1.

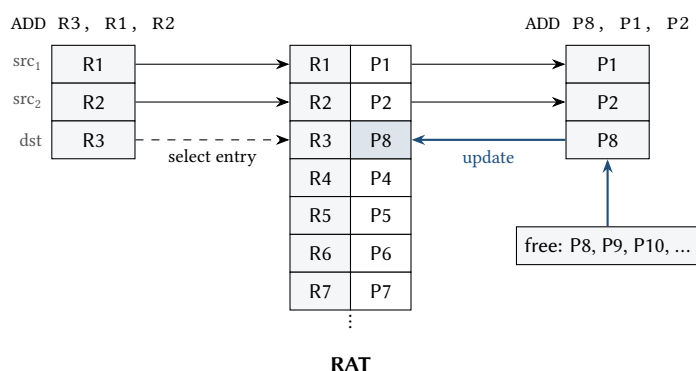


Figure 6.3: Renaming `ADD R3, R1, R2`. R1 and R2 are looked up in the RAT, the result is given the free physical register P8, and the RAT entry of R3, which held P3, is overwritten with P8.

The physical registers are exactly the versions of Section 6.3. Every write creates a new version, and the RAT records which version is current in program order, so a reader automatically uses the version produced by its closest preceding writer.

Example 6.4. We rename the running example. Before I1, the RAT maps R1 to R7, the registers the example uses, to P1 to P7, and the other architectural registers to physical registers that the example does not use; P8, P9, ... are free and are allocated in increasing order. Table 6.1 lists each renamed instruction and the RAT entries after it. I2 reads P8, the result of I1, because I1 has already updated the entry of R3. I3 reads P4, the old R4, before I4 maps R4 to P11. The two writes to R5 go to P9 and P12, and at the end the RAT maps R5 to P12, so an instruction that follows reads the result of I5, as program order requires.

6.5 Dependences after renaming

In a renamed program every result is written into its own physical register. No two instructions write the same location, and no instruction writes a location that an earlier instruction reads, so WAW and WAR dependences between register accesses no longer exist. Every read, on the other hand, still refers to the result of the instruction that

▷ 物理レジスタ

▷ レジスタリネーミング

▷ レジスタエイリアステーブル

The lecture expands RAT as *register allocation table*; both names denote the same table.

Lessons 7 and 8 show how renaming is combined with out-of-order execution in real designs.

Source Lesson 6, video 12; H&P §3.4.

Dependences through memory locations are not affected by register renaming; Lesson 9 treats them.

Table 6.1: Renaming the running example. Entries changed by an instruction are in bold; R1, R2 and R7 keep P1, P2 and P7 throughout.

	Instruction	Renamed	R3	R4	R5	R6
	(initial)		P3	P4	P5	P6
I1	ADD R3, R1, R2	ADD P8, P1, P2	P8	P4	P5	P6
I2	SUB R5, R3, R4	SUB P9, P8, P4	P8	P4	P9	P6
I3	AND R6, R1, R4	AND P10, P1, P4	P8	P4	P9	P10
I4	OR R4, R2, R7	OR P11, P2, P7	P8	P11	P9	P10
I5	XOR R5, R1, R7	XOR P12, P1, P7	P8	P11	P12	P10

produced the value in the original program, so every RAW dependence is preserved. In Table 6.1 the only dependence left is that of I2 on I1 through P8.

Same-cycle execution now only has to respect this one dependence. I1, I3, I4 and I5 execute in the first cycle and I2 in the second, with the result of I1 forwarded to it. I2 and I5 write P9 and P12, so the order of their writes is irrelevant, and I2 reads P4, which I4 does not touch. The five instructions execute in two cycles, and every later reader still finds the correct values through the RAT.

Key point

RAW dependences are the only true dependences between instructions. Register renaming removes all WAW and WAR dependences on registers, so the only constraint left on executing instructions in parallel is the flow of data itself.

6.6 Instruction-level parallelism

Once false dependences are removed, the number of cycles needed to execute a program depends only on its RAW dependences. This number is a property of the program itself, not of a particular processor. It is measured on a hypothetical *ideal processor*, which

- renames registers, so that no false dependences exist;
- executes every instruction in the first cycle in which all the values it reads are available: the first cycle if it depends on no earlier instruction, otherwise the cycle after the latest of the instructions it depends on;
- has no limit on the number of instructions it executes in a cycle;
- has unlimited hardware, so that no instruction waits for a busy execution unit: there are no structural dependences;
- executes every instruction in exactly one cycle, whatever its operation.

Definition 6.5 (Instruction-level parallelism). The *instruction-level parallelism* (ILP) of a program that executes N instructions is

$$\text{ILP} = \frac{N}{C}, \quad (6.1)$$

where C is the number of cycles in which the ideal processor executes them.

The cycle $c(I)$ in which the ideal processor executes instruction I follows directly from the RAW dependences:

$$c(I) = 1 + \max_{J \in D(I)} c(J), \quad C = \max_I c(I), \quad (6.2)$$

where $D(I)$ is the set of instructions whose results I reads, and the maximum over an empty set is 0. C is thus the length of the longest chain of RAW dependences in the program. Since no processor can execute an instruction before the values it needs exist, the ILP is an upper bound on the rate, in instructions per cycle, at which any processor can execute this program (Section 6.8).

Example 6.6. We compute the ILP of the following eight instructions:

Source Lesson 6, videos 13–15; H&P ch. 3; [14].

▷ 理想プロセッサ

▷ 命令レベル並列性

Tip Renaming need not be done on paper: for each register read, find the closest preceding writer; later writes do not matter.

```

I1:  ADD R1, R2, R3
I2:  MUL R4, R1, R5    ; R1 from I1; R5 written later
I3:  SUB R6, R2, R7    ; R2, R7 written only later
I4:  ADD R1, R6, R8    ; R6 from I3
I5:  AND R9, R1, R8    ; R1 from I4, not from I1
I6:  OR  R2, R8, R3
I7:  XOR R7, R4, R2    ; R4 from I2, R2 from I6
I8:  ADD R5, R3, R8

```

I1, I3, I6 and I8 read only registers not written before them and execute in cycle 1. I2 and I4 depend on instructions of cycle 1 and execute in cycle 2. I5 and I7 each depend on an instruction of cycle 2 and execute in cycle 3 (Figure 6.4). Hence $C = 3$ and $ILP = 8/3 \approx 2.67$. Two pitfalls are visible. R1 is written by both I1 and I4, and I5 reads the value of I4; taking I1 as its producer would wrongly place I5 in cycle 2. And I3 reads R2 and R7, which I6 and I7 write only later; these are WAR dependences, removed by renaming, and they do not delay I3.

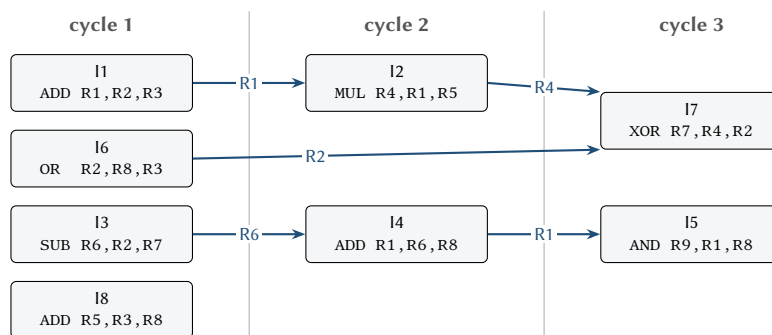


Figure 6.4: RAW dependences of Example 6.6, labelled with the register that carries the value. Each instruction is placed in the cycle in which the ideal processor executes it; the longest chain has length 3.

6.7 Structural and control dependences

Structural dependences, in which an instruction waits for hardware that another instruction is using, do not arise on the ideal processor, which has enough hardware for every instruction that is ready. Branches, however, add control dependences: whether the instructions after a branch are executed depends on its outcome. The ideal processor that defines the ILP is also assumed to predict every branch perfectly. It therefore knows from the start which instructions are executed, and the instructions after a branch do not wait for the branch to execute. The branch itself is an instruction like any other: it counts towards N and executes one cycle after the instructions that produce its operands. Instructions skipped because of the branch outcome are not executed and do not count.

Example 6.7. In the following fragment the branch is taken, so I4 is not executed:

```

I1:  ADD R1, R2, R3    ; cycle 1
I2:  SUB R4, R1, R5    ; cycle 2: R1 from I1
I3:  BEQ R4, R6, Skip  ; cycle 3: R4 from I2
I4:  ADD R7, R8, R9    ; not executed
I5:  Skip: XOR R8, R2, R6 ; cycle 1
I6:  AND R9, R8, R1    ; cycle 2: R8 from I5

```

With perfect branch prediction I5 executes in cycle 1 although the branch executes only in cycle 3. Five instructions execute in 3 cycles, so $ILP = 5/3 \approx 1.67$. If I5 and I6 had to wait for the branch, they would execute in cycles 4 and 5, and the ILP would drop to 1.

Source Lesson 6, video 16; H&P §3.1.

6.8 ILP and IPC

The ILP describes a program on the ideal processor; a real processor is described by the rate it actually achieves.

Definition 6.8 (Instructions per cycle). The average number of instructions a processor completes per cycle is its *instructions per cycle* (IPC). It is the reciprocal of the CPI of the iron law (Equation (2.3)): $IPC = 1/CPI$.

For every processor and program, $IPC \leq ILP$, because the ideal processor executes each instruction as early as the dependences allow. A real processor falls short of the ideal for several reasons.

Width It can execute only a limited number w of instructions per cycle. A *narrow* processor executes one or two, a *wide* one four or more. Whatever the program, $IPC \leq w$.

Execution units It has a limited number of adders, multipliers and other execution units, and operations such as multiplication may take several cycles. An instruction whose kind of unit is busy waits even when its inputs are ready: a structural dependence, which the ideal processor does not have.

Execution order A processor with *in-order execution* never executes an instruction before an instruction that precedes it in the program. Consecutive independent instructions may execute in the same cycle, but an instruction that has to wait for its inputs also holds back every instruction behind it. A processor with *out-of-order* execution may execute any instruction whose inputs are available (Lesson 7).

An out-of-order processor needs renaming to do so; without it, it would also have to respect the WAW and WAR dependences of Section 6.2. An in-order processor reads and writes registers in program order and needs no renaming.

Example 6.9. We run the program of Example 6.6 on four processors, each executing every instruction in one cycle and having enough execution units of every kind; the out-of-order ones rename registers and, when more instructions are ready than the width allows, execute the oldest ones first. Table 6.2 lists the instructions executed in each cycle. The in-order processors execute only I1 in cycle 1, because I2 must wait for R1 and blocks I3 to I8. The same blocking recurs at I4 (waiting for R6), at I5 (waiting for R1 from I4) and at I7 (waiting for R2), so these processors need 5 cycles whatever their width. The two-wide out-of-order processor executes two ready instructions in every cycle and needs 4 cycles; only the four-wide out-of-order processor reaches the ILP.

Table 6.2: Instructions of Example 6.6 executed in each cycle. The schedule of the ideal processor equals that of the four-wide out-of-order processor.

Processor	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	IPC
In-order, 2 wide	1	2, 3	4	5, 6	7, 8	1.60
In-order, 4 wide	1	2, 3	4	5, 6	7, 8	1.60
Out-of-order, 2 wide	1, 3	2, 4	5, 6	7, 8		2.00
Out-of-order, 4 wide	1, 3, 6, 8	2, 4	5, 7			2.67

In the example both in-order processors are held back by the order: widening from two to four gains nothing. Out-of-order execution lifts the two-wide processor only to its width (from 1.60 to $w = 2$), but the four-wide one to the ILP (from 1.60 to 2.67). In general (Table 6.3), the width is the main limit of a narrow processor (with $w = 1$, $IPC \leq 1$ in any order), so out-of-order execution raises its IPC only moderately. In a wide in-order processor, waiting instructions block the ones behind them and most of the width stays unused. Wide processors therefore need out-of-order execution to use their width, and out-of-order execution pays off fully only in a wide processor.

Source Lesson 6, videos 17–20; H&P ch. 3.

▷ サイクル当たり命令数

▷ インオーダー実行

Table 6.3: What limits the IPC of a processor relative to the ILP of the program.

	In-order	Out-of-order
Narrow (1–2 wide)	mainly width	mainly width
Wide (4 or more)	mainly order; width unused	little: IPC approaches ILP

Key point – Lesson summary

Executing many instructions in the same cycle would bring the CPI far below 1, but data dependences prevent it. Only RAW dependences are true dependences; WAW and WAR dependences come from reusing register names. Register renaming, driven by the register alias table, maps architectural registers to newly allocated physical registers and removes all false dependences between register accesses. The ILP (Equation (6.1)) of a program is the number of instructions per cycle of an ideal processor with renaming, unlimited width, no structural dependences and perfect branch prediction: the instruction count divided by the length of the longest RAW chain; it is a property of the program and bounds the IPC of every processor. Real processors fall below it, limited by their width, their execution units and in-order execution; approaching the ILP takes a wide out-of-order processor. The next lesson presents Tomasulo's algorithm, which executes instructions out of order with renaming.

Instruction Scheduling

Lecture	Lesson 7, videos 1–30
Textbook	H&P [1] §3.4–3.5; [15]
Keywords	out-of-order execution, Tomasulo’s algorithm, instruction queue, reservation station, register alias table, tag, common data bus, issue, dispatch, write result, stale result, oldest first

Lesson 6 showed that programs contain far more instruction-level parallelism than an in-order processor can use, and that approaching it requires a processor that renames registers and executes instructions out of order. This lesson describes how such a processor decides, cycle by cycle, which instructions to execute. The method is Tomasulo’s algorithm, which combines register renaming with the detection of instructions whose operands are available and is still the basis of today’s out-of-order processors. We follow an instruction through its three steps, examine the choices that arise when several instructions or results compete for the same hardware, trace a complete example, and derive a shortcut for computing when each instruction executes.

7.1 Improving IPC

Since the ILP of real programs is well above 1, a processor whose IPC is to come close to it must deal with four kinds of dependences between instructions. Each has its own remedy:

Control dependences The instructions that follow a branch can enter the pipeline before the branch executes only if its outcome is guessed. Branch prediction (Lesson 4), and predication for hard-to-predict branches (Lesson 5), take care of them.

False data dependences WAW and WAR dependences exist only because instructions reuse register names. Register renaming (Lesson 6) removes them.

True data dependences RAW dependences cannot be removed: an instruction has to wait until the values it reads have been computed. What can be avoided is that instructions which do not depend on it wait as well.

Structural dependences Instructions also compete for hardware: reservation stations, execution units and the bus introduced below. The remedy is a wider processor (Lesson 6) with more of these resources, so that more instructions can be issued and executed in the same cycle.

Definition 7.1 (Out-of-order execution). Executing each instruction as soon as the values it reads are available, even while instructions that precede it in program order are still waiting, is called *out-of-order execution*.

Out-of-order execution is combined with renaming: without it, an instruction could overtake only earlier instructions with which it has no WAW or WAR dependence, and much of the ILP would be lost. What remains is the scheduling problem: in every cycle the processor must determine which instructions have all their operands, and decide which of them to execute.

7.2 Tomasulo’s algorithm

The scheduling method used in out-of-order processors was introduced in the IBM System/360 Model 91.

Definition 7.2 (Tomasulo’s algorithm). The hardware algorithm published by Tomasulo in 1967, which renames registers and determines which instructions have their

Source Lesson 7, videos 1–2; H&P §3.1.

▷ アウトオブオーダー実行

Source Lesson 7, video 3; H&P §3.4; [15].

operands available so that each instruction executes as soon as its operands have been computed, is called *Tomasulo's algorithm*.

Current processors schedule instructions in essentially the same way. The differences are listed in Table 7.1. The first two concern only the scope of the algorithm; the third changes its mechanism. Tomasulo's algorithm has no provision for exceptions: once instructions have executed out of order, the registers need not correspond to any point in the program at which an exception could be taken. Supporting exceptions requires an additional structure, the subject of Lesson 8.

Table 7.1: Tomasulo's algorithm and modern out-of-order processors.

	Tomasulo's algorithm	Modern processors
Instructions scheduled	floating-point operations	all instructions
Instructions in flight at once	a few	hundreds
Exceptions	not supported	supported (Lesson 8)

▷ Tomasulo のアルゴリズム

7.3 Organization

Figure 7.1 shows the hardware that carries out Tomasulo's algorithm. The fetch stage appends each fetched instruction to the *instruction queue*, a first-in first-out buffer that keeps the fetched instructions in program order until they can be moved on to the reservation stations.

Source Lesson 7, video 4; H&P §3.4.

▷ 命令キュー

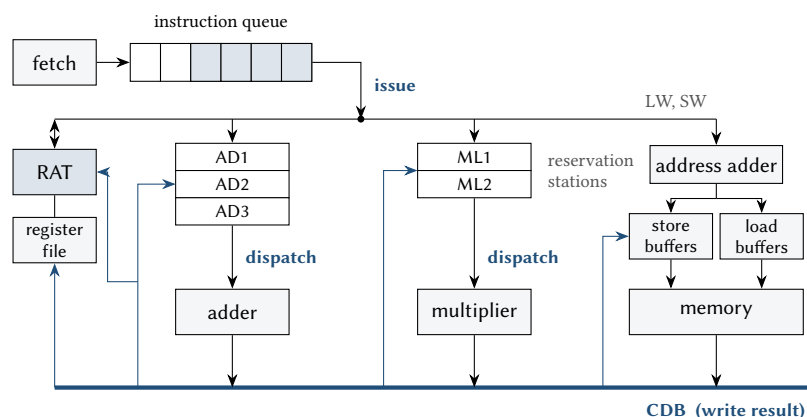


Figure 7.1: Organization of a processor with Tomasulo's algorithm. Issue moves the instruction at the head of the queue into a reservation station, using the RAT and the register file; dispatch sends a ready instruction to its execution unit; results return over the common data bus to the reservation stations, the RAT, the register file and the store buffers. Loads and stores take the path through the address adder and the load and store buffers.

Definition 7.3 (Reservation station). A reservation station (RS) holds one instruction from the moment it leaves the instruction queue until its result has been broadcast. It records whether it is busy; the operation; for each operand either its value or, if the value has not been computed yet, the tag of the value; and whether the instruction has already been dispatched.

Reservation stations are grouped by the execution unit that serves them. The examples of this lesson use three stations AD1, AD2 and AD3 for additions and subtractions, served by one adder, and two stations ML1 and ML2 for multiplications and divisions, served by one multiplier. The name of a station is the *tag* under which the result of its instruction will be broadcast.

The register alias table of Lesson 6 has one entry per register. An empty entry means that the register file holds the current value of the register; otherwise the entry

▷ リザーベーションステーション

holds the tag of the station whose instruction will produce that value. Tags thus play the role of the physical registers of Lesson 6: every result receives a new name, and an instruction that reads a register refers to the result of its closest preceding writer.

Definition 7.4 (Common data bus). The *common data bus* (CDB) carries each result computed by an execution unit, together with its tag, to all reservation stations, to the RAT and to the register file.

Loads and stores use a separate path. An adder computes the memory address; a load then waits in a load buffer, which holds the address, and a store in a store buffer, which holds the address and the value to be stored, possibly received from the CDB. The value read by a load is placed on the CDB like any other result (Section 7.8).

Every instruction passes through three steps, marked in Figure 7.1:

Issue The instruction at the head of the instruction queue is placed in a free reservation station, with its operands and its destination resolved through the RAT. This step is called *issue*.

Dispatch An instruction whose operands are all available is sent from its reservation station to an execution unit; this is its *dispatch*.

Write result The result is placed on the CDB, from which it reaches the waiting reservation stations, the RAT and the register file. This step is called *write result*, or broadcast.

▷ 共通データバス

▷ 発行

▷ デイスパッチ

▷ 結果書き込み

7.4 Issue

Issue takes the instruction at the head of the instruction queue and performs the following steps. In the examples of this lesson one instruction is issued per cycle; a wider processor issues several per cycle, still in program order.

1. Take the next instruction, in program order, from the head of the queue.
2. Determine where each operand comes from. If the RAT entry of the operand register is empty, the value is in the register file; otherwise the entry names the tag of the value.
3. Find a free reservation station for the operation. If every station of that kind is busy, the instruction is not issued in this cycle; it stays at the head of the queue, and the instructions behind it wait as well.
4. Place the instruction in the station: the operation, and for each operand the value from the register file or the tag from the RAT.
5. Set the RAT entry of the destination register to the name of the station.

Instructions are issued strictly in program order. The RAT then describes the register values seen at the position of the instruction being issued, so that each operand refers to its closest preceding writer. As in renaming, the operands are looked up before the destination entry is overwritten.

Example 7.5. The following program is the running example of this lesson:

```
I1:  MUL R1, R2, R3      ; R1 <- R2 * R3
I2:  ADD R4, R1, R5      ; R4 <- R1 + R5
I3:  SUB R6, R2, R5      ; R6 <- R2 - R5
I4:  ADD R1, R5, R3      ; R1 <- R5 + R3
I5:  DIV R2, R6, R3      ; R2 <- R6 / R3
I6:  MUL R3, R4, R1      ; R3 <- R4 * R1
I7:  SUB R5, R3, R2      ; R5 <- R3 - R2
```

Initially the RAT is empty and the register file holds $R1 = 1$, $R2 = 14$, $R3 = 4$, $R4 = 3$, $R5 = 6$ and $R6 = 7$. In cycle 1, I1 is issued into ML1 with the operand values 14 and 4, and the RAT entry of R1 becomes ML1. In cycle 2, I2 is issued into AD1 (Figure 7.2). The RAT entry of R1 holds ML1, so the first operand is the tag ML1; the entry of R5 is empty, so the second operand is the value 6. The RAT entry of R4 becomes AD1. I3 and I4 follow in cycles 3 and 4. I4 writes R1 again and replaces ML1 with AD3 in the entry of R1: every instruction issued from now on that reads R1 uses the result of I4, not that of I1.

Source Lesson 7, videos 5–7; H&P §3.5.

Caution Issue never waits for operands, only for a free reservation station.

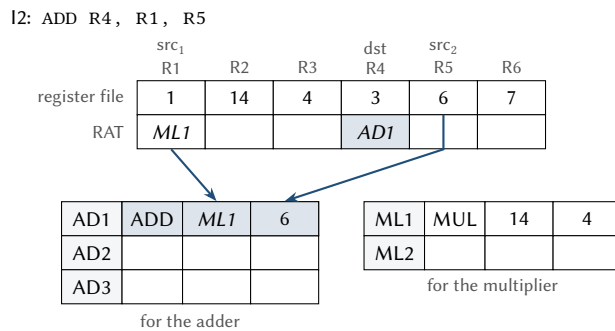


Figure 7.2: Issue of I2 in cycle 2 of Example 7.5. The RAT entry of R1 yields the tag ML1 (tags in italics), the empty entry of R5 lets the value 6 be taken from the register file, and the entry of R4 is set to AD1. Shaded cells are written in this step.

7.5 Dispatch

An instruction waits in its reservation station until all its operands are values. Whenever a result appears on the CDB, every reservation station compares the tags of its operands with the tag on the bus and, where they match, replaces the tag with the value from the bus; this is called *capturing* the value. A station whose operands are all values holds a *ready* instruction. When the execution unit that serves the station is free, a ready instruction is dispatched: its operation and operand values are sent to the unit, which starts executing it. The station remains busy while the instruction executes, because its name is the tag under which the result will be broadcast.

Source Lesson 7, video 8; H&P §3.5.

7.5.1 More than one ready instruction

Several instructions may be ready when an execution unit becomes free, and one of them must be chosen. Every choice is correct, since a ready instruction has all the values it needs; the choice affects only performance and hardware cost.

Oldest first Dispatch the instruction that was issued first. Other things being equal, more of the instructions in flight were issued after an older instruction and may be waiting for its result, so this approximates the next policy while requiring only that the order of issue be recorded.

Most dependents first Dispatch the instruction whose result is needed by the largest number of waiting instructions, so that as many instructions as possible become ready soon. This promises the best performance, but finding that instruction means comparing the tag of every candidate with the operands of all reservation stations, which is too slow and costly.

Random The simplest selection logic. It may keep an instruction on which many others depend waiting behind unimportant ones, so performance suffers.

Oldest first is a good compromise between the two others and is the rule used in the examples of this lesson. In the running example, both I2 and I4 are ready when the adder becomes free in cycle 7 (Section 7.9); oldest first dispatches I2, although I4 has been ready two cycles longer.

Source Lesson 7, videos 9–10.

7.6 Write result

When an execution unit has finished an instruction, the write-result step broadcasts its result.

1. The result and the tag of the instruction's reservation station are placed on the CDB.
2. Every operand that holds this tag captures the value (Section 7.5).
3. If the RAT entry of a register holds the tag, the value is written into that register in the register file and the RAT entry is cleared, so that instructions issued later read the value from the register file.
4. The reservation station is freed.

Source Lesson 7, video 11; H&P §3.5.

The station need not record the destination register, because the RAT entry that holds the tag identifies it. Figure 7.3 shows the broadcast of the result of I2 in the running example.

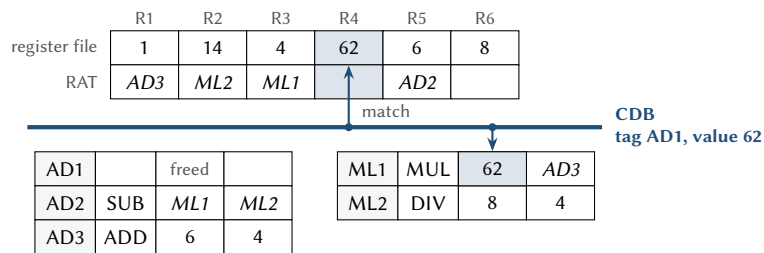


Figure 7.3: Write result of I2 in cycle 9 of the running example. ML1 (I6) captures 62 in place of the tag AD1; the RAT entry of R4 holds AD1, so R4 becomes 62 and the entry is cleared; AD1 is freed. No other operand or RAT entry holds AD1.

7.6.1 More than one result in a cycle

Two execution units may finish in the same cycle, but one bus carries one result at a time. One solution is a separate bus for each unit. Every operand of every reservation station must then compare its tag with each bus and select the value from the matching one, and the register file needs a write port per bus; the comparison logic, which is already large, grows in proportion to the number of buses. The alternative keeps a single bus and lets one result wait for the next cycle. The rule used here gives priority to the result of the *slower* unit: its instruction was dispatched longer ago, so it tends to be the older instruction, and more instructions are likely to be waiting for its result. In the examples we assume that a unit whose result is waiting for the bus cannot start another instruction.

Source Lesson 7, video 12.

7.6.2 Stale results

By the time a result is broadcast, a later instruction that writes the same register may already have been issued. The RAT entry of the register then holds the tag of the later instruction, not the tag on the bus, and step 3 has no effect: neither the register file nor the RAT changes. The result is *stale* only for the register, and ignoring it there is correct. After both instructions the register must hold the later result, which arrives when the later instruction broadcasts. The earlier result is still needed, but only by the instructions issued between the two writers; they hold its tag and capture the value in step 2. Instructions issued after the later writer refer to the later tag.

Source Lesson 7, video 13; H&P §3.5.

Example 7.6. In the running example, I1 and I4 both write R1. When I1 broadcasts 56 under the tag ML1 in cycle 6, the RAT entry of R1 already holds AD3, because I4 was issued in cycle 4. R1 in the register file keeps its old value 1 and the entry keeps AD3, while AD1, which holds I2, captures 56. R1 becomes 10 only in cycle 11, when I4 broadcasts under AD3.

Key point

A broadcast delivers its value to every operand that waits for its tag, but it updates the register file and the RAT only if the RAT entry of the destination still holds that tag.

7.7 The steps in one cycle

For a single instruction, Tomasulo's algorithm is a sequence: issue, waiting and capturing in a reservation station, dispatch, execution, write result. For the processor as a whole, all steps take place in every cycle, each for different instructions: in the

Source Lesson 7, videos 14–19; H&P §3.5.

same cycle one instruction may be issued, several stations may capture a broadcast value, ready instructions may be dispatched to free units, and one result is broadcast.

Whether two steps of the *same* instruction can take place in the same cycle is a matter of circuit design. In the simplest design every step examines the structures as they are at the start of the cycle, and all updates take effect at its end. Then:

- An instruction issued in cycle C is in its reservation station only from cycle $C + 1$, so it can be dispatched in $C + 1$ at the earliest.
- An instruction that captures its last missing operand in cycle C is ready from cycle $C + 1$ and can be dispatched in $C + 1$ at the earliest.
- A station freed by a broadcast in cycle C can receive a new instruction in cycle $C + 1$.

The delay between capture and dispatch can be removed with additional logic, a dispatch that also considers operands arriving on the bus in the current cycle, and a station freed by a broadcast could similarly be reused in the same cycle. The additional logic lies in the same cycle as the step it extends and therefore lengthens the clock cycle. An instruction, however, is not dispatched in the cycle in which it is issued: while issue writes it into its station, dispatch does not yet see it there.

Two interactions between issue and write result in the same cycle need care.

- Both can change the RAT entry of the same register: the broadcast of an older writer clears it, and the issue of a newer writer sets it to a new tag. The issue must prevail, because it belongs to the later instruction in program order; equivalently, the write result is applied first and the issue second. Otherwise later instructions would read the register file and miss the result of the newly issued instruction.
- An instruction may be issued in the cycle in which one of its operands is broadcast. In the simplest design it finds the tag in the RAT, since the entry is cleared only at the end of the cycle, but enters its station after the value has passed on the bus; issue must then compare operand tags with the bus and take the value directly, or the instruction would wait for a broadcast that never comes. If the updates of the broadcast are visible to the issue in the same cycle, the issue simply reads the new value from the register file.

Key point

In the simplest design an instruction advances by at most one step per cycle: issued in cycle C , it is dispatched in $C + 1$ at the earliest, and a result broadcast in cycle W lets the instructions waiting for it be dispatched in $W + 1$ at the earliest.

7.8 Load and store instructions

Loads and stores create dependences through memory that are analogous to the dependences through registers.

```
SW  R1, 0(R2)      ; MEM[R2] <- R1
LW  R3, 8(R4)      ; R3 <- MEM[R4 + 8]
```

If $R4 + 8$ equals $R2$, the load reads the value just stored, a RAW dependence through memory. Two stores to the same address have a WAW dependence, and a store to an address that an earlier load reads has a WAR dependence. The RAT cannot remove the false ones, because it renames registers, not memory locations, and whether two memory accesses conflict is known only once both addresses have been computed.

Tomasulo's algorithm therefore performs loads and stores in program order. They are issued like other instructions, with their operands resolved through the RAT, and the address adder computes the address. A load then waits in a load buffer and a store in a store buffer (Figure 7.1); a store whose value has not been computed yet captures it from the CDB. The memory accesses are made from these buffers in the order in which the loads and stores were issued. A load broadcasts the value it reads on the CDB under its own tag, exactly like an arithmetic result. Modern processors also execute loads and stores out of order, which requires the processor to detect and

Source Lesson 7, video 20; H&P §3.5.

handle the memory dependences; Lesson 9 covers this.

7.9 A complete example

We now execute the program of Example 7.5 to the end. The assumptions are those of the simplest design of Section 7.7, together with the following:

- At most one instruction is issued per cycle, into the free station of the required kind with the lowest number.
- The adder executes ADD and SUB in 2 cycles; the multiplier executes MUL and DIV in 4 cycles. An instruction dispatched in cycle D with an execution time of k cycles executes in cycles D to $D + k - 1$ and broadcasts in cycle $D + k$, in which its unit can already accept the next instruction.
- There is one CDB, and the multiplier has priority over the adder (Section 7.6.1); a unit whose result waits for the bus accepts no new instruction. Ready instructions are dispatched oldest first.

Table 7.2 lists what happens in each cycle, and Table 7.3 shows the reservation stations and the RAT after cycles 5, 6 and 7.

Table 7.2: Events in each cycle of the complete example. Captures are listed with the broadcast that causes them.

Cycle	Issue	Dispatch	Write result and remarks
1	I1 → ML1		
2	I2 → AD1	I1	
3	I3 → AD2		I2 waits for ML1
4	I4 → AD3	I3	RAT entry of R1 becomes AD3
5	I5 → ML2		all stations busy; I4 ready, adder busy
6			I1: ML1 = 56, stale for R1; AD1 captures; I3 finished, waits for the bus; no free station for I6
7	I6 → ML1	I2	I3: AD2 = 8, R6 ← 8; ML2 captures; I2 and I4 ready, I2 older
8	I7 → AD2	I5	
9		I4	I2: AD1 = 62, R4 ← 62; ML1 captures
10			
11			I4: AD3 = 10, R1 ← 10; ML1 captures
12		I6	I5: ML2 = 2, R2 ← 2; AD2 captures
13–15			I6 executes
16			I6: ML1 = 620, R3 ← 620; AD2 captures
17		I7	
18			
19			I7: AD2 = 618, R5 ← 618

Table 7.3: Reservation stations and RAT at the end of cycles 5, 6 and 7. Tags are in italics; “exec” marks an instruction that has been dispatched. An empty RAT entry is shown with the register-file value.

	After cycle 5	After cycle 6	After cycle 7
AD1	I2: <i>ML1</i> , 6	I2: 56, 6	I2: 56, 6 (exec)
AD2	I3: 14, 6 (exec)	I3: 14, 6 (exec)	free
AD3	I4: 6, 4	I4: 6, 4	I4: 6, 4
ML1	I1: 14, 4 (exec)	free	I6: <i>AD1</i> , <i>AD3</i>
ML2	I5: <i>AD2</i> , 4	I5: <i>AD2</i> , 4	I5: 8, 4
R1	<i>AD3</i>	<i>AD3</i>	<i>AD3</i>
R2	<i>ML2</i>	<i>ML2</i>	<i>ML2</i>
R3	4	4	<i>ML1</i>
R4	<i>AD1</i>	<i>AD1</i>	<i>AD1</i>
R5	6	6	6
R6	<i>AD2</i>	<i>AD2</i>	8

Source Lesson 7, videos 21–26; H&P §3.5.

Cycles 1–5. One instruction is issued per cycle. I1 and I3 have all their operands at issue and are dispatched in the next cycle, I1 to the multiplier and I3 to the adder. I4 is ready from cycle 5, but the adder is busy with I3. I2 waits for the tag ML1 and I5 for AD2. After cycle 5 every station is busy.

Cycle 6. Both units finished in cycle 5. The bus takes the result of the slower multiplier, 56 under ML1, which is stale for R1 (Example 7.6); AD1 captures it and ML1 is freed. The result of I3 waits in the adder. I6 is not issued, because at the start of the cycle both multiplier stations are busy.

Cycle 7. I3 broadcasts 8 under AD2. The RAT entry of R6 holds AD2, so R6 becomes 8 and the entry is cleared; ML2 captures the value for I5, and AD2 is freed. The adder can take a new instruction, and both I2 and I4 are ready; oldest first dispatches I2. I6 is issued into ML1, freed in the previous cycle, and receives two tags, AD1 and AD3.

Cycles 8–19. I7 is issued into AD2 and waits for ML1 and ML2. I5 starts on the multiplier in cycle 8, and I4 on the adder in cycle 9, when I2 releases it. I6 needs the results of I2 (cycle 9) and I4 (cycle 11) and the multiplier, which I5 releases in cycle 12; I7 needs the result of I6, broadcast in cycle 16. After the last broadcast, in cycle 19, the register file holds $R1 = 10$, $R2 = 2$, $R3 = 620$, $R4 = 62$, $R5 = 618$ and $R6 = 8$, exactly the values produced by executing the program in order.

Key point

The instructions of the example broadcast in the order I1, I3, I2, I4, I5, I6, I7, and each executes as soon as its operands, a free station and a free unit allow. Because every operand is resolved through a tag at issue and stale results never reach the register file, the final register values are those of sequential execution.

7.10 Timing without tracing the structures

Tracing every structure cycle by cycle is laborious. When only the timing matters, it suffices to determine the cycles of issue, dispatch and write result of each instruction. Let I_n , D_n and W_n denote these cycles for instruction n , and k_n its execution time. Under the assumptions of Section 7.9:

Issue I_n is the first cycle after I_{n-1} in which a station of the required kind is free. A station freed by the write result of instruction m is free from cycle $W_m + 1$.

Dispatch D_n is the first cycle that satisfies

$$D_n \geq 1 + \max\left(I_n, \max_{p \in P(n)} W_p\right), \quad (7.1)$$

where $P(n)$ is the set of instructions that produce the operands of n (its closest preceding writers), and in which the execution unit is free: not held by another instruction m , which holds it in cycles D_m to $W_m - 1$, and not taken by an older instruction that is also ready.

Write result $W_n = D_n + k_n$, or later if the bus is taken in that cycle by a result with priority.

A unit or the bus may be taken by a *later* instruction that became ready earlier, so the rows cannot simply be completed in program order. Units and the bus are assigned in cycle order across all instructions: in each cycle a free unit takes the oldest of the instructions ready in that cycle, and the bus the result with priority. A unit never stays idle for an older instruction that is not yet ready.

Example 7.7. Table 7.4 applies these rules to the running example, and Figure 7.4 draws the result. I3, I4 and I6 show every rule. I3 is dispatched in cycle 4, one cycle after its issue, and finishes executing in cycle 5; its write result would be cycle 6, but the bus then carries the result of I1, so $W_3 = 7$. Whether the adder is free for I2 in

Source Lesson 7, videos 27–29; H&P §3.5.

Tip $P(n)$: for each register n reads, its closest preceding writer (Lesson 6).

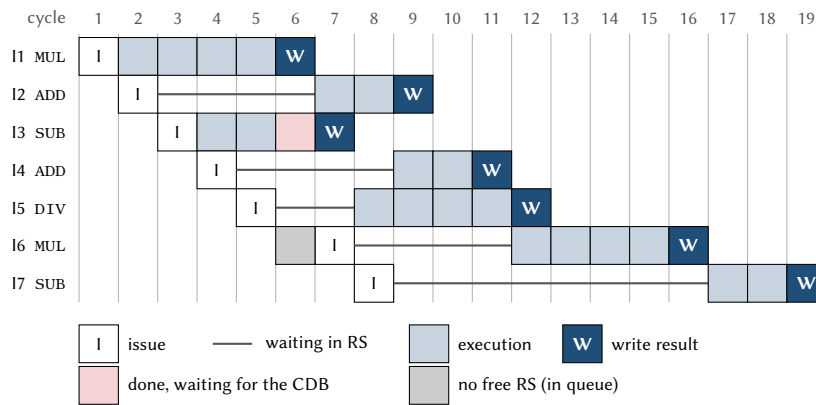


Figure 7.4: Timing of the running example. Out-of-order execution lets I3 run while I2 waits and I5 start while I4 waits; the delays of I3 (bus), I4 (adder) and I6 (reservation station) are those derived in Example 7.7.

cycle 7 thus depends on the row of the later I3. I4 is ready from cycle 5, but the adder is occupied by I3, whose result waits for the bus, until its write result in cycle 7, and then by the older I2 until cycle 9, so $D_4 = 9$ and $W_4 = 11$. I6 finds both multiplier stations busy until ML1 is freed by $W_1 = 6$, so $I_6 = 7$; its operands come from I2 and I4, and Equation (7.1) gives $D_6 \geq 1 + \max(7, 9, 11) = 12$, which is also the cycle in which the multiplier is released by I5, so $D_6 = 12$ and $W_6 = 16$. The table agrees with the full trace of Table 7.2.

Table 7.4: Issue, dispatch and write-result cycles of the running example.

Instruction	I_n	D_n	W_n	What determines the timing
I1 MUL R1, R2, R3	1	2	6	ready at issue
I2 ADD R4, R1, R5	2	7	9	$W_1 = 6$; adder busy with I3 until $W_3 = 7$
I3 SUB R6, R2, R5	3	4	7	ready at issue; bus taken in 6
I4 ADD R1, R5, R3	4	9	11	adder: I3, then the older I2
I5 DIV R2, R6, R3	5	8	12	$W_3 = 7$
I6 MUL R3, R4, R1	7	12	16	no station until $W_1 = 6$; $W_4 = 11$; multiplier until $W_5 = 12$
I7 SUB R5, R3, R2	8	17	19	$W_6 = 16$

Key point — Lesson summary

Tomasulo's algorithm combines register renaming with out-of-order execution, in which each instruction executes as soon as its operands are available. Issue moves instructions in program order from the instruction queue into reservation stations, taking each operand either as a value from the register file or as a tag from the RAT, and points the RAT entry of the destination to the station. A station captures values from the common data bus until its instruction is ready; dispatch then sends it to a free execution unit, choosing the oldest when several are ready. Write result broadcasts the value and tag, frees the station, and updates the register file and the RAT only if the result is not stale. One bus serves one result per cycle, the slower unit having priority. Loads and stores are performed in program order. With one step per cycle, the issue, dispatch and write-result cycles of every instruction follow from its operands, the available stations and units, and the bus. Tomasulo's algorithm does not support exceptions; the reorder buffer of the next lesson adds this.

ReOrder Buffer

Lecture	Lesson 8, videos 1–40
Textbook	H&P [1] §3.6, §3.8, §C.4; [16]
Keywords	exception, precise exception, phantom exception, reorder buffer, commit pointer, issue pointer, commit, branch misprediction recovery, register alias table, unified reservation station, superscalar, out-of-order execution

Tomasulo’s algorithm (Lesson 7) executes each instruction as soon as its operands are available and writes its result into the register file when the result is broadcast. Renaming keeps the computed values correct, but two events that every real processor must handle break the scheme: exceptions and branch mispredictions. In both cases instructions that follow the offending instruction in program order may already have overwritten registers, and these writes cannot be undone. This lesson introduces the reorder buffer, which lets instructions execute out of order but updates the registers strictly in program order. It traces the resulting processor in detail and ends with unified reservation stations, superscalar execution, and a precise account of which steps of an out-of-order processor actually work out of order.

8.1 Exceptions in out-of-order execution

An exception is an event, such as a division by zero or an access to an invalid memory address, that an instruction raises when it executes. The processor saves the address of the excepting instruction and transfers control to an exception handler, usually part of the operating system. When the handler has dealt with the cause, it may return to the saved address, so that the excepting instruction is executed again and the program continues after it. For the handler to work, and for the program to continue correctly after the return, the registers and memory must look as they would in a processor that executes one instruction at a time.

Definition 8.1 (Precise exception). An exception is a *precise exception* if, when the handler starts, every instruction that precedes the excepting instruction in program order has completed and updated the registers and memory, and neither the excepting instruction nor any instruction that follows it has changed them.

Tomasulo’s algorithm does not provide precise exceptions. Consider:

```
I1:  DIV R1, R2, R3      ; R3 = 0: division by zero
I2:  ADD R2, R4, R5      ; overwrites R2, read by I1
I3:  ADD R6, R6, R7      ; R6 <- R6 + R7
```

I2 and I3 do not depend on I1, so they execute and write R2 and R6 within a few cycles of being issued. The division takes many cycles, and in the model used here the divider reports the division by zero when the operation completes (Figure 8.1). By then both later results are in the register file, and three things go wrong.

- The handler sees values of R2 and R6 that no sequential execution stopped at I1 could produce.
- After the return, I1 is executed again and now reads the value of R2 written by I2, which in program order it must never see.
- I2 and I3 are also executed a second time. I2 merely recomputes the same value, but I3 adds R7 to R6 twice.

8.2 Branch mispredictions

Branch mispredictions cause the same problem. In the following program the branch

Source Lesson 8, videos 1–2; H&P §C.4, §3.6.

▷ 正確な例外

Source Lesson 8, video 3; H&P §3.6.

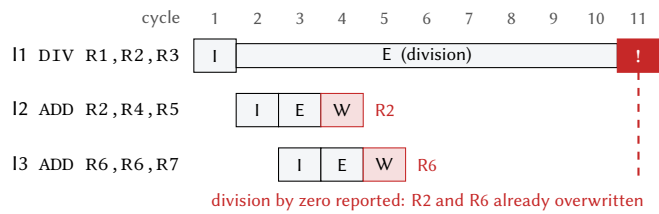


Figure 8.1: An exception under Tomasulo's algorithm (I: issue, E: execute, W: write result). I2 and I3 write R2 and R6 long before the division I1 reports the division by zero.

predictor lets the processor fetch and issue I3 to I6 while I2 waits for the result of the division:

```

I1:      DIV R1, R2, R3      ; takes many cycles
I2:      BEQ R1, R4, Skip    ; predicted not taken
I3:      ADD R5, R6, R7      ; I3-I6: executed only
I4:      LW  R8, 0(R5)       ; if not taken
I5:      MUL R9, R5, R5
I6:      SUB R6, R9, R7
...
Skip: ...

```

I2 needs R1 and cannot execute before the division completes, whereas I3 has its operands at once and I4 a cycle after I3. Under Tomasulo's algorithm R5 and R8 are written long before the branch is resolved. If the branch turns out to be taken, the instructions of the wrong path can be removed from the reservation stations and fetching can restart at Skip, but R5 and R8 have been overwritten with values that the program never computes, and their correct values are lost.

A second problem arises when an instruction on the wrong path raises an exception. The program uses R5 as an address only when the branch is not taken; when it is taken, the value R6 + R7 need not be a valid address, and the load I4 may raise an exception long before I2 executes.

Definition 8.2 (Phantom exception). An exception raised by an instruction that lies on a mispredicted path, and therefore is never executed in a sequential execution of the program, is called a *phantom exception*.

A phantom exception must not reach the handler: in program order, the instruction that raised it does not exist.

▷ ファントム例外

8.3 Correct out-of-order execution

Both problems have the same cause. An instruction changes the registers as soon as it has computed its result, before it is known that every earlier instruction has completed without an exception or a misprediction. The remedy is to separate computing a result from making it visible:

- instructions still *execute* out of order, as soon as their operands are ready;
- results are still *broadcast* out of order, so that waiting instructions receive them without delay;
- but results are *written into the registers* in program order, and only once it is certain that the instruction belongs to the program's execution.

Holding the results until they may be written requires a structure that remembers the program order of the instructions after they have been issued. The reservation stations cannot do this: an instruction leaves its reservation station as soon as its result is broadcast, and the stations are allocated in no particular order.

Key point

Execute out of order, broadcast out of order, but write the registers in program order.

Source Lesson 8, video 4; H&P §3.6; [16].

8.4 The reorder buffer

The structure that remembers the program order is a queue of the issued instructions.

Definition 8.3 (Reorder buffer). The *reorder buffer* (ROB) holds, in program order, every instruction that has been issued but has not yet passed the commit step of Definition 8.4, in which its result is written into the register file. Each entry has three fields:

Reg the destination register of the instruction;

Value its result, once computed;

Done a bit that is set when Value holds the result.

The lecture draws only these three fields. An entry must also record whether the instruction raised an exception or is a mispredicted branch, and the address of the instruction, which commit needs in order to take the exception or to restart fetching (Sections 8.5 and 8.6).

The ROB is a circular buffer managed by two pointers (Figure 8.2). The *issue pointer* names the entry that the next issued instruction receives, and it advances by one entry for every issued instruction. The oldest instruction that has not yet committed is found at the *commit pointer*. The entries from the commit pointer up to, but not including, the issue pointer are in use and hold instructions in program order; all others are free. Since equal pointers would describe both an empty and a full buffer, an occupancy count or a full bit distinguishes the two. In normal operation both pointers only advance, wrapping around from the last entry to the first; recovery moves the issue pointer back (Section 8.5). When every entry is in use, no instruction can be issued until the oldest one leaves the ROB. Every issued instruction receives an entry, including a branch, which writes no register: its entry keeps its place in the program order and, as shown below, records whether it was mispredicted.

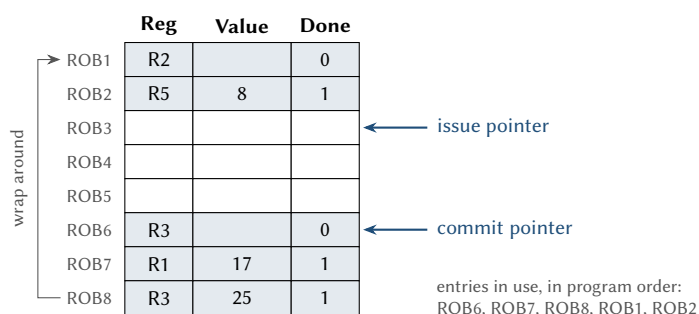


Figure 8.2: A reorder buffer with eight entries. The shaded entries, from the commit pointer up to the issue pointer, are in use; here this range wraps around from ROB8 to ROB1.

8.4.1 Issue, dispatch and write result

With a ROB, the three steps of Tomasulo's algorithm change as follows.

Issue The instruction needs a free reservation station (RS) and the free ROB entry at the issue pointer; if either is missing, issue waits. The ROB entry receives the destination register and a cleared Done bit, and the issue pointer advances. Each operand is then looked up in the RAT, whose entries now name ROB entries instead of RSs:

- if the RAT entry points to the register file, the value is copied from the register;
- if it names a ROB entry whose Done bit is set, the value is copied from that entry;
- otherwise the RS records the ROB entry as the tag of the operand and waits for a broadcast with that tag.

Finally, the RAT entry of the destination register is set to the new ROB entry, and the RS records that entry as the tag of its result.

Source Lesson 8, videos 5–10; H&P §3.6.

▷ リオーダーバッファ

▷ 発行ポインタ

▷ コミットポインタ

Dispatch As before, an instruction whose operands are all available starts executing. Its RS is freed at this point.

Write result The result is broadcast on the CDB together with its tag, the ROB entry. Waiting RSs capture it as before, and the ROB entry stores it and sets its Done bit. Neither the register file nor the RAT is changed.

The second lookup case is new. A result stays in the ROB from write result until commit, and during that time the RAT keeps naming the ROB entry; an instruction issued in that interval must find the value there, because the broadcast it would otherwise wait for has already happened.

The earlier release of RSs is also new. In Tomasulo's algorithm an RS must stay allocated until write result, because the station itself is the tag under which the result is broadcast. With a ROB the tag is the ROB entry, so the RS is no longer needed once the operation has started, and the same number of RSs can hold more waiting instructions.

8.4.2 Commit

A fourth step completes every instruction.

Definition 8.4 (Commit). In the *commit* step the processor examines the entry at the commit pointer. If its Done bit is set, the value is written into the destination register, and if the RAT entry of that register still names this ROB entry, the RAT entry is reset to point to the register file. The commit pointer then advances, which frees the entry.

If the instruction at the commit pointer is not done, nothing commits, even if later entries are: results enter the register file only in program order. The RAT entry is reset only when it still names the committing entry; otherwise a later instruction writes the same register, and the RAT must keep naming that instruction's entry (Section 8.7). Commit is also called *retirement*.

▷ コミット

8.4.3 Hardware organization

Figure 8.3 shows the organization of Lesson 7 with a ROB added. The CDB now delivers results to the RSs and to the ROB, the RAT names ROB entries, and the register file is written only by commit. An instruction passes through six steps: fetch, decode, issue, execute (from dispatch until the operation completes), write result and commit.

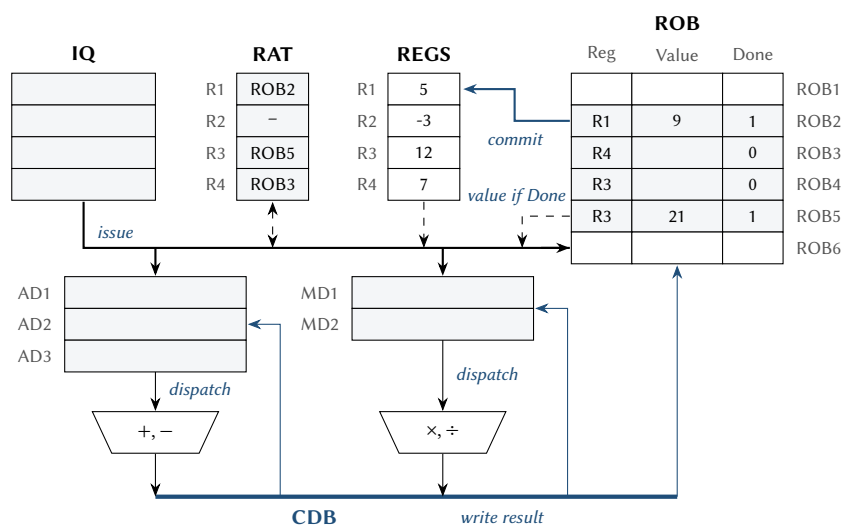


Figure 8.3: Tomasulo's organization with a reorder buffer. Issue allocates an RS and a ROB entry and renames the operands through the RAT; operand values come from the register file or from a Done ROB entry. Results go from the CDB to the RSs and the ROB; commit copies the value at the commit pointer into the register file.

Note 8.5. At any moment the most recent value of an architectural register, in the program order of the instructions issued so far, is in one of three places: in the register file, when the RAT points there; in a ROB entry whose Done bit is set; or not yet computed, in a ROB entry whose Done bit is clear. The register file itself always holds the values as of the last committed instruction.

8.5 Branch misprediction recovery

With a ROB, instructions on a mispredicted path cannot change the register file as long as they are removed before they commit. In the program of Section 8.2, I3 to I6 are issued while I2 waits for R1, and all of them except I6, which is still executing, write their results into their ROB entries. When I2 finally executes and the processor finds that the branch was mispredicted, the misprediction is recorded in the ROB entry of I2, and nothing else happens yet. Instructions before the branch are unaffected and continue to execute and commit.

When the branch reaches the commit pointer, every instruction before it has committed, so the register file holds exactly the values that a sequential execution has after the branch. Every instruction still in the processor follows the branch and is on the wrong path. The processor now performs *branch misprediction recovery*, which consists of four actions (Figure 8.4):

1. discard all ROB entries by setting the issue pointer equal to the commit pointer, which leaves the ROB empty;
2. reset every RAT entry to point to the register file;
3. free all RSs and abandon the operations in the execution units, whose results could only go to discarded ROB entries;
4. restart fetching at the correct successor of the branch.

A correctly predicted branch simply commits: the commit pointer advances and nothing else happens.

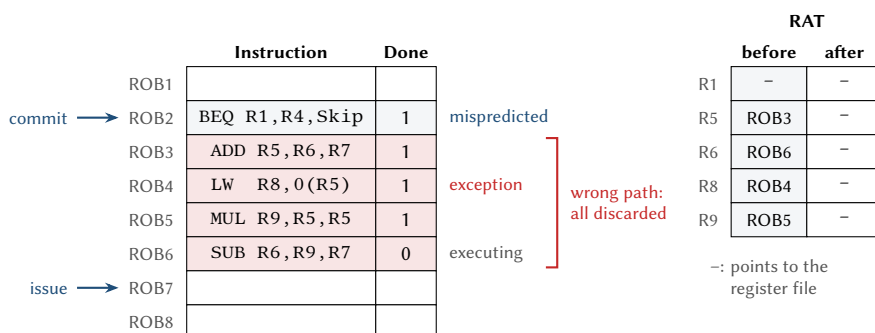


Figure 8.4: Recovery when the mispredicted branch I2 reaches the commit pointer. I1 has already committed; the ROB entries of I3 to I6 are discarded, including the exception recorded for I4, and all RAT entries return to the register file.

Note 8.6. Recovering at commit is the simplest scheme and the one used in the lecture. Real processors start recovery as soon as the mispredicted branch executes, discarding only the ROB entries after the branch and restarting fetching at once, so that no cycles are lost while the branch waits to commit (H&P §3.6). The RAT must then be brought back to its state at the branch, for example from a copy saved when the branch was issued.

8.6 Exceptions with a reorder buffer

Exceptions are handled by treating an exception as one more result of the instruction that raises it. When an instruction raises an exception during execution, the processor records the exception in the instruction's ROB entry instead of invoking the handler. When that entry reaches the commit pointer, the exception is taken: the processor

Source Lesson 8, videos 11–12; H&P §3.6.

▷ 分岐予測ミス回復

Source Lesson 8, videos 13–15; H&P §3.6.

discards the ROB entries, resets the RAT, and frees the RSs and execution units exactly as in misprediction recovery, saves the address of the excepting instruction, and starts fetching the exception handler.

This removes both problems of Sections 8.1 and 8.2.

Precise exceptions When the excepting instruction reaches the commit pointer, all earlier instructions have committed and no later instruction has written the register file. Stores likewise write memory only when they commit (Lesson 9), so memory is also unchanged by later instructions. The handler sees exactly the state of Definition 8.1; after the return, the later instructions are executed from that state.

No phantom exceptions An exception on a mispredicted path is recorded in an entry that follows the mispredicted branch. The branch commits first, and its recovery discards the entry together with the exception, which is never taken. In Figure 8.4 the exception of I4 disappears when I2 commits.

8.6.1 The outside view of execution

The ROB separates two notions of when an instruction is executed. Inside the processor, an instruction executes when an execution unit performs its operation, in any order. The programmer, the operating system and the exception handler, however, see only the register file, memory and the program counter, and for them an instruction is executed when it commits. Commits happen one instruction after another in program order, exactly as in a processor that completes each instruction before starting the next. An instruction whose result is in the ROB but has not committed has, seen from outside, not been executed yet; if its result is discarded, it has never been executed at all. The example of Section 8.8 ends with an instance.

8.7 RAT updates on commit

The conditional RAT update of Definition 8.4 is needed because a register may have several pending writers. Suppose the instructions in ROB2 and ROB5 both write R4, and ROB2 commits while ROB5 is still in the ROB. The RAT entry of R4 names ROB5, because that instruction was issued later.

- Commit writes the value of ROB2 into R4. This is the correct architectural value after ROB2, and it is the value an exception handler must see if an instruction between ROB2 and ROB5 raises an exception.
- The RAT entry must nevertheless keep naming ROB5. An instruction issued now follows ROB5 in program order and must use the value of ROB5, not the older value that has just entered the register file.

When ROB5 commits, the RAT entry of R4 still names ROB5 unless an even later writer has been issued in the meantime, and only then is it reset. The opposite mistake, leaving a RAT entry that names a committed entry, is equally wrong: the entry is freed at commit and may soon be given to an unrelated instruction.

8.8 A reorder buffer example

We now trace a complete program through a processor with a ROB. The processor has three RSs for addition and subtraction (AD1 to AD3), two for multiplication and division (MD1, MD2) and a ROB of eight entries. ADD and SUB take 1 cycle, MUL 4 cycles and DIV 6 cycles, and there are enough execution units that no ready instruction waits for one. The steps follow these conventions:

- one instruction is issued per cycle, into the free RS of the required kind with the lowest number;
- an RS freed by a dispatch in cycle C , and a ROB entry freed by a commit in cycle C , can be given to a new instruction from cycle $C + 1$;
- an instruction is dispatched no earlier than the cycle after its issue and the cycle after the broadcast of each operand it waits for;

Source Lesson 8, video 16; H&P §3.6.

Source Lesson 8, videos 17–30; H&P §3.6.

- an operation dispatched in cycle D with an execution time of k cycles executes in cycles D to $D + k - 1$ and writes its result in cycle $D + k$; there is one CDB, and in this example no two results are written in the same cycle;
- an instruction commits no earlier than the cycle after its write result, and at most one instruction commits per cycle.

Example 8.7. The program is

```

I1:  DIV R1, R2, R3      ; R1 <- R2 / R3
I2:  ADD R4, R5, R6      ; R4 <- R5 + R6
I3:  MUL R2, R4, R4      ; R2 <- R4 * R4
I4:  SUB R5, R1, R6      ; R5 <- R1 - R6
I5:  ADD R4, R4, R3      ; R4 <- R4 + R3
I6:  ADD R6, R2, R4      ; R6 <- R2 + R4

```

with initial values $R1 = 5$, $R2 = 48$, $R3 = 4$, $R4 = 2$, $R5 = 7$ and $R6 = 3$. All RAT entries point to the register file, and both ROB pointers are at ROB1.

Cycles 1–6: issue. One instruction is issued per cycle, I_k into ROB_k . I_1 copies 48 and 4 from the register file into MD1 and sets the RAT entry of $R1$ to ROB1. In cycle 2 I_1 is dispatched and frees MD1, and I_2 is issued into AD1 with the values 7 and 3. I_3 , issued in cycle 3, finds ROB2 in the RAT entry of $R4$; ROB2 is not done, so both operands of I_3 wait for ROB2. I_3 can use MD1 again, which under Tomasulo's algorithm alone would stay occupied by I_1 until cycle 8. I_4 is issued into AD1, freed by I_2 in cycle 3; its first operand waits for ROB1, its second is 3. In cycle 4 I_2 writes its result: ROB2 receives 10, and I_3 captures it. In cycle 5 I_5 is issued into AD2 and finds the RAT entry of $R4$ still naming ROB2, which is done, so it copies 10 from the ROB, not from the register file, which still holds the old value 2; I_5 then renames $R4$ to ROB5. In cycle 6 I_6 is issued into AD3, because AD2, freed by the dispatch of I_5 in the same cycle, can be used only from cycle 7; I_6 waits for ROB3 and ROB5. Tables 8.1 and 8.2 show the state at the end of cycle 6: AD1 holds I_4 and AD3 holds I_6 , all other RSs are free, and the register file is unchanged.

Table 8.1: ROB of Example 8.7 at the end of cycles 6 and 10. At the end of cycle 6 the commit pointer is at ROB1, at the end of cycle 10 at ROB3; the issue pointer is at ROB7 in both.

Entry	Instruction	Reg	Cycle 6		Cycle 10	
			Value	Done	Value	Done
ROB1	DIV R1, R2, R3	R1		0	committed	
ROB2	ADD R4, R5, R6	R4	10	1	committed	
ROB3	MUL R2, R4, R4	R2		0	100	1
ROB4	SUB R5, R1, R6	R5		0	9	1
ROB5	ADD R4, R4, R3	R4		0	14	1
ROB6	ADD R6, R2, R4	R6		0		0

Table 8.2: RAT and register file of Example 8.7. A dash means that the RAT entry points to the register file.

Register	Cycle 6		Cycle 10		End
	RAT	Value	RAT	Value	Value
R1	ROB1	5	–	12	12
R2	ROB3	48	ROB3	48	100
R3	–	4	–	4	4
R4	ROB5	2	ROB5	10	14
R5	ROB4	7	ROB4	7	9
R6	ROB6	3	ROB6	3	114

Cycles 7–8: results, but no commit. I5 writes 14 into ROB5 in cycle 7 and I1 writes 12 into ROB1 in cycle 8; I6 and I4 capture these values. Although ROB2 has been done since cycle 4 and ROB5 since cycle 7, nothing commits, because ROB1, at the commit pointer, is not done until the end of cycle 8.

Cycles 9–10: the first commits. In cycle 9 I1 commits: R1 becomes 12 and, since the RAT entry of R1 still names ROB1, it is reset to the register file. In the same cycle I3 writes 100 into ROB3 and I4 is dispatched. In cycle 10 I2 commits and R4 becomes 10, but the RAT entry of R4 names ROB5 and is left unchanged, as Section 8.7 requires. I4 writes 9 and I6 is dispatched.

Cycles 11–14: the remaining commits. I6 writes 114 in cycle 11. I3, I4, I5 and I6 commit in cycles 11 to 14, one per cycle. Each is the last writer of its register, so each resets its RAT entry. At the end the ROB is empty, all RAT entries point to the register file, and the registers hold R1 = 12, R2 = 100, R3 = 4, R4 = 14, R5 = 9 and R6 = 114, the values of a sequential execution.

At the end of cycle 10 the register file holds 10 in R4 while the RAT sends readers of R4 to ROB5, which holds 14. Both are right: 10 is the value of R4 after I2, the last committed instruction, and 14 is the value after I5, the last issued writer.

The example also illustrates the outside view of Section 8.6.1. Suppose that I4 raises an exception when it executes in cycle 9. The exception is recorded in ROB4, I2 and I3 still commit in cycles 10 and 11, and in cycle 12 I4 reaches the commit pointer and the exception is taken. The handler finds R1 = 12, R2 = 100 and R4 = 10, but not R4 = 14, although I5 wrote 14 into ROB5 in cycle 7, before I1 had even finished: from outside, execution stopped exactly before I4.

8.9 Timing with a reorder buffer

The cycle-by-cycle state of a ROB processor can be summarized by the cycle in which each instruction passes each step, as for Tomasulo's algorithm in Lesson 7, with one additional column for commit. We keep the notation of Lesson 7: I_n , D_n and W_n are the cycles of issue, dispatch and write result of instruction n , k_n is its execution time, and $P(n)$ is the set of instructions that produce its operands; C_n denotes its commit cycle. Under the conventions of Section 8.8, I_n is the first cycle after I_{n-1} in which an RS of the required kind and a ROB entry are free. An RS released by the dispatch of instruction m is free from cycle $D_m + 1$, and a ROB entry released by the commit of m from cycle $C_m + 1$. With enough execution units and no competition for the CDB, the other cycles follow from

$$D_n = 1 + \max\left(I_n, \max_{p \in P(n)} W_p\right), \quad W_n = D_n + k_n, \quad C_n = \max(W_n + 1, C_{n-1} + 1), \quad (8.1)$$

where the first equation is the bound of Equation (7.1) with a free execution unit. Unlike execution, commit depends on the preceding instruction even when the two share no data. The point at which RSs are released and the number of commits per cycle are design assumptions that change the table. If an RS is released only at write result, as in Lesson 7, it is free from cycle $W_m + 1$; if two instructions may commit in the same cycle, $C_n = \max(W_n + 1, C_{n-1}, C_{n-2} + 1)$.

Table 8.3 lists the result for the example. The Execute and Write result columns are out of order: I5 writes its result before I1, I3 and I4, which precede it. The Commit column is increasing. I2 and I5 commit six cycles after writing their results, held up by the slow division at the head of the ROB and, after it commits in cycle 9, by the limit of one commit per cycle: every later instruction commits exactly one cycle after its predecessor. This waiting delays no computation: I3 and I5 obtained the result of I2, and I6 those of I3 and I5, without waiting for any commit. The ROB delays only the moment at which results become part of the architectural state.

Source Lesson 8, videos 31–34; H&P §3.6.

Tip Fill in the table row by row in program order, each row from left to right, with Commit last.

Table 8.3: Timing of Example 8.7. The Execute column gives the cycles D_n to $D_n + k_n - 1$ in which the operation is performed.

	Instruction	Issue	Execute	Write result	Commit
I1	DIV R1, R2, R3	1	2–7	8	9
I2	ADD R4, R5, R6	2	3	4	10
I3	MUL R2, R4, R4	3	5–8	9	11
I4	SUB R5, R1, R6	4	9	10	12
I5	ADD R4, R4, R3	5	6	7	13
I6	ADD R6, R2, R4	6	10	11	14

The ROB can nevertheless limit execution when it is too small. With only four entries, I5 could not be issued before cycle 10, the cycle after I1 commits, and it would execute several cycles later than in Table 8.3 although its operands are available from cycle 5. A ROB must therefore be large enough to hold all instructions issued while a slow instruction waits to commit.

8.10 Unified reservation stations

In Figure 8.3 each kind of execution unit has its own group of RSs, and the size of each group is fixed at design time. Programs differ in their mix of operations, so a program with many additions can find all adder RSs occupied, and issue must wait, while the multiplier RSs are empty. Because instructions are issued in program order, even a multiplication that follows the waiting addition cannot use the free multiplier RSs.

Definition 8.8 (Unified reservation stations). A *unified reservation station* is a single pool of RSs shared by all operations: an instruction can be issued into any free RS of the pool.

Issue then waits only when all RSs are occupied, so the same number of RSs is used more effectively. The cost lies in dispatch: instead of choosing among the RSs of one group for one execution unit, the dispatch logic must find, for each free execution unit, the ready instructions in the whole pool whose operation that unit performs, and select among them.

8.11 Superscalar processors

Lesson 6 showed that the IPC of a processor is limited by the number of instructions it can handle per cycle (Section 6.8).

Definition 8.9 (Superscalar processor). A *superscalar* processor performs every step on more than one instruction per cycle: it fetches, decodes and issues several instructions, dispatches several to execution units, writes several results and commits several instructions in each cycle.

Every step needs its own hardware for this. Fetch and decode must handle several instructions at once; issue must rename several instructions in the same cycle, each seeing the RAT updates of the ones before it; dispatch needs several execution units; writing several results needs several CDBs, and every RS operand must compare its tag against all of them; commit must examine several entries at the commit pointer and write several registers.

The steps form a chain through which every instruction passes, so the processor sustains no more instructions per cycle than its most restrictive step handles. A processor that fetches, decodes and dispatches four instructions per cycle but issues three and commits two completes, in the long run, at most two instructions per cycle, however much ILP the program has. Widening one step pays off only if that step is the bottleneck.

Source Lesson 8, video 35; H&P ch. 3.

▷ 統合リザベーションステーション

Source Lesson 8, video 36; H&P §3.8.

▷ スーパースカラ

8.12 Terminology

The names used in this course for the steps of an out-of-order processor are not universal. Papers, textbooks and processor documentation use several alternatives (Table 8.4). Some of them call commit *complete*; in this chapter, by contrast, an operation completes when its execution ends.

Table 8.4: Names of the steps in this course and in other texts.

This course	Names in other texts
Issue	issue, allocate, dispatch
Dispatch	dispatch, execute, issue
Commit	commit, complete, retire, graduate

Source Lesson 8, video 37; H&P ch. 3.

Caution The same word can name different steps: in some texts *dispatch* is what this course calls issue, and *issue* what it calls dispatch.

8.13 What is out of order

A processor with out-of-order execution does not perform all of its steps out of order (Figure 8.5).

Fetch and decode take instructions in program order, along the predicted path.

Issue is in program order. Renaming requires it, because each instruction must see the RAT as left by all instructions before it, and ROB entries are allocated in program order.

Execute and write result are out of order: an instruction starts as soon as its operands are ready and finishes when its operation completes. These are the only steps in which a later instruction overtakes an earlier one.

Commit is in program order again; the ROB restores the order that execution relaxed.

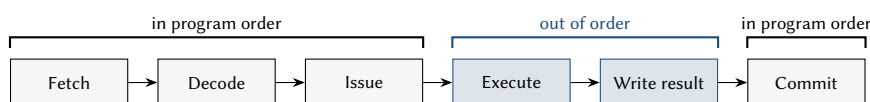


Figure 8.5: The steps of an out-of-order processor with a ROB. Only execution and write result take place out of program order.

Key point — Lesson summary

Tomasulo's algorithm writes results into the registers as soon as they are broadcast, so exceptions are not precise, mispredicted instructions corrupt registers, and wrong-path instructions can raise phantom exceptions. The reorder buffer keeps issued instructions in program order between a commit pointer and an issue pointer. The RAT names ROB entries, results are broadcast to the RSs and the ROB, RSs are freed at dispatch, and only commit, one ROB entry after another in program order, writes the register file and resets a RAT entry that still names the committing entry. A mispredicted branch or an excepting instruction is acted on when it commits: the ROB is emptied, the RAT points back to the register file, the RSs and execution units are cleared, and fetching restarts at the correct address. Timing tables gain a commit column that follows Equation (8.1). Unified reservation stations share one RS pool among all operations; a superscalar processor handles several instructions per cycle in every step and is limited by its narrowest step. Only execution and write result are out of order.

Memory Ordering

Lecture	Lesson 9, videos 1–14
Textbook	H&P [1] §3.5–3.6
Keywords	memory dependence, load, store, commit, load-store queue, LSQ, store-to-load forwarding, out-of-order load-store execution, in-order load-store execution, re-execution, reorder buffer, reservation station

With the reorder buffer of Lesson 8, an out-of-order processor handles every dependence that passes through registers and keeps exceptions precise. Loads and stores, however, also depend on one another through memory, and whether a load reads the location that an earlier store writes is known only once both addresses have been computed, which is why Lesson 7 performed loads and stores in program order. This lesson shows how they can execute out of order: stores write memory only at commit, a load-store queue keeps the uncommitted loads and stores in program order, and each load obtains its value either from memory or directly from the store that wrote it.

9.1 Dependences through memory

The processor of Lesson 8 deals with control dependences and with every dependence that passes through a register. Branch prediction (Lesson 4) removes most of the cost of control dependences, and the ROB cancels the instructions of a mispredicted path; renaming (Lesson 6) removes the false dependences between writes to the same register; the scheduling of Tomasulo’s algorithm (Lesson 7) makes each instruction wait until the register values it reads have been computed; and the ROB makes exceptions precise.

Loads and stores remain. They depend on one another through memory locations in the same way as instructions depend on one another through registers (Section 7.8):

```
SW  R1, 0(R2)      ; MEM[R2] <- R1
ADD R4, R5, R6      ; R4 <- R5 + R6
LW  R3, 0(R4)      ; R3 <- MEM[R4]
```

The load has a RAW dependence on the store exactly when R4 equals R2. Two properties distinguish such a dependence from a dependence through a register.

- It is not visible in the instructions themselves. The register names say nothing about the memory location, which is known only after the address has been computed, often several cycles after the instruction was issued.
- Nothing renames memory. The RAT maps register names only, so the processor must find, for every load, the store whose value the load has to read, or determine that there is none.

The rest of the lesson answers two questions: when a store actually writes memory, and where a load finds its value.

9.2 When memory is written

A store executes like any other instruction once its operands are available, but it does not write memory at that time. Executing the store computes the address and determines the value; the write itself takes place when the store commits. A write to memory can no more be undone than a write to a register. If a store wrote memory when it executed, an exception raised afterwards by an earlier instruction would let the handler see a location that, in program order, has not yet been written, and a store on a mispredicted path would change memory that the program never changes. At commit, in contrast, the store is known to belong to the execution of the program,

Source Lesson 9, videos 1–2; H&P §3.5–3.6.

Source Lesson 9, video 3; H&P §3.6.

and stores commit in program order.

Deferring the write creates a problem for loads. A load could read memory as soon as it has computed its address, but a store that precedes it in program order may have executed and still be waiting to commit, or may not even have computed its address yet. If that store writes the location the load reads, memory still holds the old value, and the load must instead obtain the value of the store. The processor must therefore keep, in program order, the address and value of every store that has not yet committed, and every load must consult them.

Key point

Stores execute out of order but write memory at commit, in program order. Until then their values exist only inside the processor, and loads must look for them there.

9.3 The load-store queue

The uncommitted loads and stores are kept in a queue that works like the ROB.

Definition 9.1 (Load-store queue). The *load-store queue* (LSQ) holds, in program order, every load and store that has been issued but has not yet committed. Each entry has four fields:

L/S whether the instruction is a load or a store;

Addr the memory address, once it has been computed;

Value for a store, the value to be written; for a load, the value it has obtained;

C set when the entry is complete: a load has obtained its value, or a store has recorded both its address and its value.

Like the ROB, the LSQ is a circular buffer with an issue pointer and a commit pointer (Figure 9.1). A load or store receives the entry at the issue pointer when it is issued, in addition to its ROB entry, and its LSQ entry is freed when it commits. Other instructions do not use the LSQ. A load records its address and then its value. A store records its address as soon as its base register is available and its value as soon as the value register is, in either order. Because a store keeps its entry until it commits and writes memory only then, the LSQ contains exactly the stores whose values are not yet in memory.

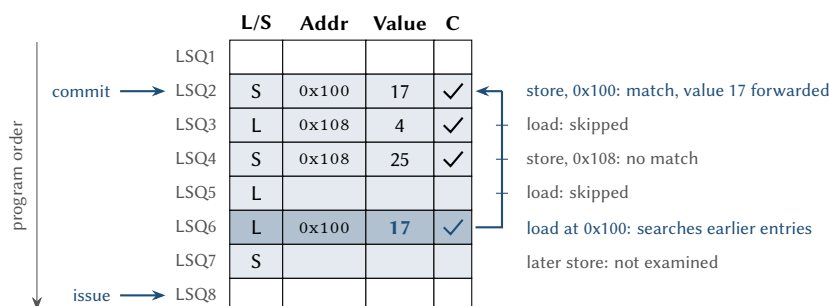


Figure 9.1: A load-store queue with eight entries; LSQ2 to LSQ7 are in use. The load in LSQ6 has computed the address 0x100. Its search skips loads and the store to 0x108 and takes the value 17 from the nearest earlier store to 0x100.

When a load has computed its address, it records the address in its LSQ entry and searches the earlier entries, from its own entry back towards the commit pointer. Loads are skipped, because reading a location does not change it; only stores matter. The search ends at the first store with the same address or at the commit pointer.

1. If it finds a store with the same address, this nearest matching store is the last write to the location before the load in program order, so its value is the one the load must read. The load copies the value from the store's LSQ entry, waiting for it if the store has not recorded its value yet, and does not access memory

In the pipeline of Lesson 3, the MEM stage performs loads and stores in program order.

Source Lesson 9, videos 4–5; H&P §3.6.

▷ ロードストアキュー

The ROB also holds the loads and stores, but a separate queue keeps the address searches short.

at all. Supplying a load with the value of an uncommitted store in this way is called *store-to-load forwarding*.

2. If it reaches the commit pointer without finding a matching store or a store with an unknown address, no uncommitted store writes the location before the load, so memory already holds the correct value, and the load reads memory.

If, before it ends, the search passes a store whose address has not been computed yet, that store may or may not write the location the load reads, and the processor cannot yet tell which. This case is the difficult one, and the next section presents the ways of dealing with it.

9.4 Out-of-order and in-order load-store execution

For a load whose search passes a store with an unknown address, there are three options.

1. Loads and stores execute in program order among themselves, as in Lesson 7.
2. The load waits until every earlier store has computed its address, so that its search never meets an unknown address.
3. The load executes anyway, and a wrong value is detected and corrected later.

The first two options make loads wait; the third does not, but needs a check. A store's address is unknown until its base register is available, which can take long when that register is computed by a slow instruction. In the following program the address of the store depends on a multiplication, whereas the address of the load is available almost at once:

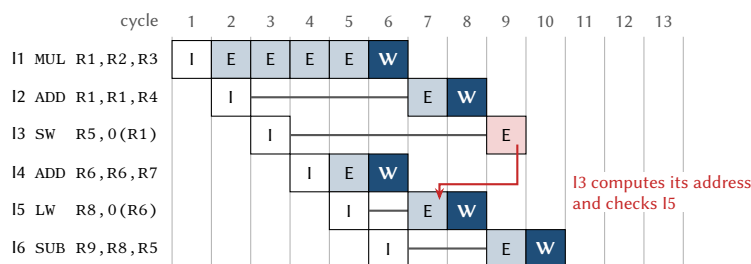
```

I1: MUL R1, R2, R3      ; R1 <- R2 * R3
I2: ADD R1, R1, R4      ; R1 <- R1 + R4 (an address)
I3: SW  R5, 0(R1)       ; MEM[R1] <- R5
I4: ADD R6, R6, R7      ; R6 <- R6 + R7 (an address)
I5: LW  R8, 0(R6)       ; R8 <- MEM[R6]
I6: SUB R9, R8, R5      ; R9 <- R8 - R5

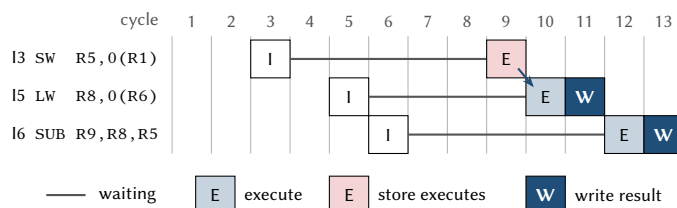
```

We use the timing conventions of Section 8.8. MUL takes 4 cycles; ADD, SUB, loads and stores take 1 cycle, in which a load computes its address and obtains its value and a store computes its address; the value R5 is available from the start. I1 writes its result in cycle 6 and I2 in cycle 8, so the store I3 executes, and its address becomes known, only in cycle 9. I4 writes its result in cycle 6, so the load I5 could execute in cycle 7 (Figure 9.2).

(a) out-of-order load-store execution



(b) in-order load-store execution



I issue — waiting E execute E store executes W write result

Figure 9.2: The program of Section 9.4 under two policies. (a) I5 executes before the store I3 knows its address; I3 checks I5 in cycle 9. (b) I5 waits until I3 has computed its address. I1, I2 and I4 behave as in (a).

▷ ストアロードフォワーディング

Source Lesson 9, videos 5–8; H&P §3.6.

9.4.1 Out-of-order load-store execution

Definition 9.2 (Out-of-order load-store execution). Under *out-of-order load-store execution* (option 3), a load executes as soon as its address can be computed, whether or not the earlier stores have computed theirs. Its LSQ search passes over stores with unknown addresses as if their addresses differed.

In the example I5 executes in cycle 7 and I6, which uses its value, in cycle 9. In return, every store must check its own address against the loads that have overtaken it. When I3 computes its address in cycle 9, it searches the later LSQ entries for loads with the same address whose C bit is set, that is, loads that have already obtained a value.

- If the address of I5 differs, nothing happens: I5 has obtained the correct value, and I6 writes its result in cycle 10.
- If the addresses are equal, I5 has obtained a value that I3 overwrites in program order, so every instruction that used this value, directly or indirectly, has computed a wrong result. The processor cancels I5 and all instructions after it and executes them again, starting by fetching I5. None of them has committed, since I3 precedes them, so the ROB discards them like the instructions of a mispredicted path (Section 8.5). When I5 executes again, its search finds I3, and it receives the value by forwarding.

9.4.2 In-order load-store execution

Definition 9.3 (In-order load-store execution). Under *in-order load-store execution* (option 1), loads and stores execute in program order among themselves: a load or store waits until every earlier load and store has executed, where a store counts as executed once its address is known. Other instructions still execute out of order.

In the example I5 waits for I3 and executes in cycle 10, and I6 executes in cycle 12 and writes its result in cycle 13. No load ever obtains a wrong value, and nothing is re-executed. But I5 waits three cycles also when its address differs from that of I3, and then the wait is pure loss. Option 2 gives the same timing here, because no load precedes I5; in general it waits less, since a load does not depend on an earlier load.

9.4.3 Comparison

Out-of-order load-store execution gains cycles whenever a load does not read a location written by an earlier store whose address is still unknown, and loses cycles when it does, because cancelling and re-executing the load and the instructions after it costs more than waiting would have. Options 1 and 2 delay every load that follows a store with an unknown address, whether the two conflict or not. In real programs few loads actually read what such a store writes, so out-of-order load-store execution performs better, and it is the policy that high-performance processors use.

Key point

In-order load-store execution never re-executes a load but makes loads wait for stores they usually do not depend on. Out-of-order load-store execution lets loads run at once; correctness then relies on each store checking the completed later loads when it computes its address.

9.5 Store-to-load forwarding

Store-to-load forwarding relates each load to the store whose value it must read, and a processor with out-of-order load-store execution uses this relation from both ends.

For a load the value comes from the nearest earlier store with the same address, the last store to write the location before the load in program order. An older store to the same address does not qualify: its value is overwritten by the nearer

▷ ロードストアのアウトオブオーダー実行

▷ ロードストアのインオーダー実行

Source Lesson 9, video 9; H&P ch. 3.

one. If the LSQ holds no earlier store with this address, the value comes from memory.

For a store the value goes to every later load with the same address, up to the next store with that address. The loads after that store receive the value of that store instead.

A load applies the first rule in its search (Section 9.3), and a store applies the second in its check of the completed later loads (Section 9.4). Figure 9.3 shows the relation for seven loads and stores to two addresses.

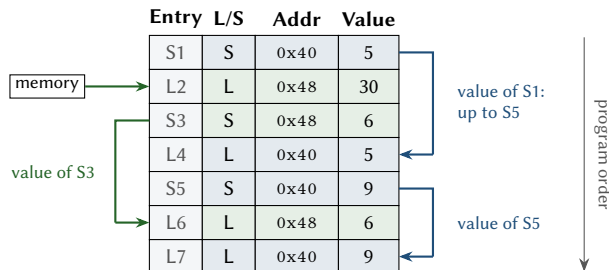


Figure 9.3: Store-to-load forwarding for seven loads and stores to 0x40 and 0x48, with the values of a sequential execution. L4 receives the value of S1, not of S5, which follows it; L7 receives the value of S5, which overwrites that of S1. L2 precedes S3 and reads memory.

9.6 A load-store queue example

We now trace the loads and stores of Figure 9.3 (Example 9.4) under out-of-order load-store execution.

Example 9.4. The loads and stores of a program are, in program order,

```

S1:  SW  R1, 0(R2)      ; MEM[0x40] <- R1
L2:  LW  R3, 8(R2)      ; R3 <- MEM[0x48]
S3:  SW  R4, 8(R10)     ; MEM[R10+8] <- R4
L4:  LW  R5, 0(R2)      ; R5 <- MEM[0x40]
S5:  SW  R6, 0(R7)      ; MEM[R7] <- R6
L6:  LW  R8, 8(R2)      ; R8 <- MEM[0x48]
L7:  LW  R9, 0(R2)      ; R9 <- MEM[0x40]

```

with $R2 = 0x40$, $R1 = 5$, $R4 = 6$ and $R6 = 9$ available from the start. $R10 = 0x40$ and $R7 = 0x40$ are computed by slow instructions that precede S3 and S5, so these two stores learn their addresses late, in the order of Table 9.1. Memory initially holds 20 at 0x40 and 30 at 0x48.

Table 9.1: Execution of Example 9.4. In each step one entry computes its address: a load searches the earlier stores; a store checks the later loads whose C bit is set.

Step	Entry	Addr	Search or check	Outcome
1	S1	0x40	no later load completed	value 5 recorded
2	L2	0x48	S1: other address	memory: 30
3	L4	0x40	S3: unknown; S1: match	from S1: 5
4	S3	0x48	L4: other address	no conflict
5	L6	0x48	S5: unknown; S3: match	from S3: 6
6	L7	0x40	S5: unknown; S3: other; S1: match	from S1: 5
7	S5	0x40	L6: other address; L7: match	L7 re-executed
8	L7	0x40	S5: match	from S5: 9

L4 and L6 pass over stores whose addresses are still unknown, and the later checks of S3 and S5 find no conflict with them. L7 also passes over S5, but in step 7 S5 computes 0x40 and finds L7 completed with this address, with no store to 0x40

Caution Search from the load backwards: a search forward from the commit pointer finds the oldest matching store.

Source Lesson 9, video 10; H&P §3.6.

between them. The value 5 that L7 obtained from S1 is overwritten by S5 in program order, so L7 and every instruction after it are cancelled and executed again, and in step 8 L7 receives 9 from S5.

After the entries commit in program order, memory holds 9 at 0x40 and 6 at 0x48, and the loads have delivered $R3 = 30$, $R5 = 5$, $R8 = 6$ and $R9 = 9$, the results of a sequential execution. With in-order load-store execution L4 would have waited for S3, and L6 and L7 for S5, and nothing would have been executed twice.

9.7 The LSQ, the ROB and the reservation stations

Loads and stores pass through the same steps as other instructions, and the LSQ adds work to each step (Table 9.2). A load or store is issued only when a ROB entry and an LSQ entry are both available; the LSQ entry takes the place of the reservation station that other instructions need with their ROB entry. A load broadcasts its value on the CDB under the name of its ROB entry, exactly like an arithmetic result. A store produces no register value, so nothing is broadcast, and its ROB entry is done as soon as the store has recorded both its address and its value.

Source Lesson 9, videos 11–14; H&P §3.6.

Table 9.2: The steps of a load and a store with an LSQ.

Step	Load	Store
Issue	needs a free ROB entry and LSQ entry (no RS); the base register is looked up in the RAT, and the RAT entry of the destination register is set to the load's ROB entry	needs a free ROB entry and LSQ entry (no RS); the base and value registers are looked up in the RAT; no RAT entry changes
Execute	computes the address and records it in the LSQ; searches the earlier stores; copies a forwarded value or reads memory; sets C	records the address when the base register is available and checks the later loads whose C is set; records the value when it is available; sets C once both are recorded
Write result	broadcasts the value on the CDB; the ROB entry receives it	nothing to broadcast
Commit	the value is written to the register file, and the RAT entry is reset if it still names this ROB entry; the ROB and LSQ entries are freed	the value is written to memory; the ROB and LSQ entries are freed

Both queues are in program order, so the load or store at the commit pointer of the ROB is also the oldest entry of the LSQ, and the two entries are freed together. A store leaves the LSQ exactly when its value is written to memory, so a later load always finds that value in one of the two places it looks: in the store's LSQ entry before the commit, in memory after it.

Key point — Lesson summary

Loads and stores depend on one another through memory addresses that are known only after execution, and memory is not renamed. Stores therefore write memory at commit, in program order. The load-store queue holds the uncommitted loads and stores in program order, with their addresses, values and completion bits. A load takes its value from the nearest earlier store with the same address by store-to-load forwarding, or from memory if there is none. Earlier stores with unknown addresses are handled either by in-order load-store execution, where the load waits for them, or by out-of-order load-store execution, where the load proceeds and a store that later computes a matching address has the load and all instructions after it re-executed; the latter is faster because such conflicts are rare. A load or store needs a ROB entry and an LSQ entry at issue, where other instructions need a ROB entry and an RS.

Compiler ILP

Lecture	Lesson 10, videos 1–20
Textbook	H&P [1] §3.2, Appendix H
Keywords	instruction-level parallelism, instructions per cycle, dependence chain, tree height reduction, instruction scheduling, in-order execution, if-conversion, loop unrolling, function call inlining, code bloat, instruction count, software pipelining, trace scheduling

Lessons 6–9 raised the instructions per cycle (IPC) of a processor with hardware: register renaming, out-of-order execution with a reorder buffer, and out-of-order load-store execution let the IPC approach the instruction-level parallelism (ILP) of the program. The hardware, however, can exploit only the parallelism that the program contains and that lies within its reach. This lesson shows how the compiler improves both: tree height reduction shortens the dependence chains that bound the ILP, instruction scheduling places independent instructions next to each other, and if-conversion, loop unrolling and function call inlining remove the branches, calls and returns that keep them apart.

10.1 How the compiler can raise IPC

The IPC of every processor is bounded by the ILP of the program it executes (Section 6.8). Two limits on the IPC are not determined by the processor alone, and the compiler influences both.

The ILP of the program The ILP is the number of instructions divided by the length of the longest chain of RAW dependences (Equations (6.1) and (6.2)). How long the chains are depends on how the compiler has translated the source program: the same computation can often be expressed by instructions that form shorter chains.

The parallelism a processor can find A processor looks for instructions to execute only among a limited number of instructions that follow the oldest unexecuted one. An out-of-order processor considers the instructions that fit into its reorder buffer (Lesson 8); an in-order processor considers only the next few and stops at the first one that must wait (Lesson 6). Independent instructions that lie farther apart in program order than this window cannot execute in the same cycle, however high the ILP. The compiler can place such instructions close together, and it can remove the branches, calls and returns that stand between them.

The compiler sees the whole program rather than a window of it, and it runs before the program executes, so it can afford analyses that a processor could never carry out within a clock cycle. Table 10.1 lists the techniques of this lesson.

Source Lesson 10, videos 1–2; H&P §3.2.

Table 10.1: Limits on the IPC and the compiler techniques that address them.

Limit	Compiler technique	Section
long dependence chains lower the ILP	tree height reduction	10.2
independent instructions lie far apart	instruction scheduling	10.3
branches separate instructions	if-conversion, trace scheduling	10.4, 10.7
loop branches separate iterations	loop unrolling, software pipelining	10.5, 10.7
calls and returns separate instructions	function call inlining	10.6

10.2 Tree height reduction

A compiler translates an arithmetic expression by way of its *expression tree*, in which every operation is a node whose children are its operands. Each operation becomes one instruction, and the instruction of a node reads the results of the instructions of its children. The longest chain of RAW dependences among these instructions therefore equals the *height* of the tree, the number of operations on the longest path from the root to a leaf, and on the ideal processor of Lesson 6 each level of the tree takes one cycle.

A straightforward translation follows the order in which the source program writes the operands. For a sum of six variables held in R1 to R6, with the result in R10, it produces

```
ADD R10, R1, R2      ; cycle 1
ADD R10, R10, R3     ; cycle 2
ADD R10, R10, R4     ; cycle 3
ADD R10, R10, R5     ; cycle 4
ADD R10, R10, R6     ; cycle 5
```

Every addition uses the result of the previous one. The tree degenerates into a chain of height 5 (Figure 10.1a): the five instructions take five cycles on any processor, and their ILP is 1.

Definition 10.1 (Tree height reduction). Rewriting an expression into an equivalent one whose expression tree has fewer levels, by using the associativity and commutativity of its operations, is called *tree height reduction*. The number of operations, and hence of instructions, does not change.

Example 10.2. Grouped as $((R1 + R2) + (R3 + R4)) + (R5 + R6)$, the same sum becomes

```
ADD R10, R1, R2      ; cycle 1
ADD R11, R3, R4      ; cycle 1
ADD R12, R5, R6      ; cycle 1
ADD R10, R10, R11     ; cycle 2
ADD R10, R10, R12     ; cycle 3
```

The three additions of pairs depend on nothing and execute in cycle 1, the sum of the first two pairs follows in cycle 2 and the final addition in cycle 3 (Figure 10.1b). There are still five instructions, now in three cycles: the ILP rises from 1 to $5/3 \approx 1.67$. The price is two further registers, R11 and R12, for intermediate results.

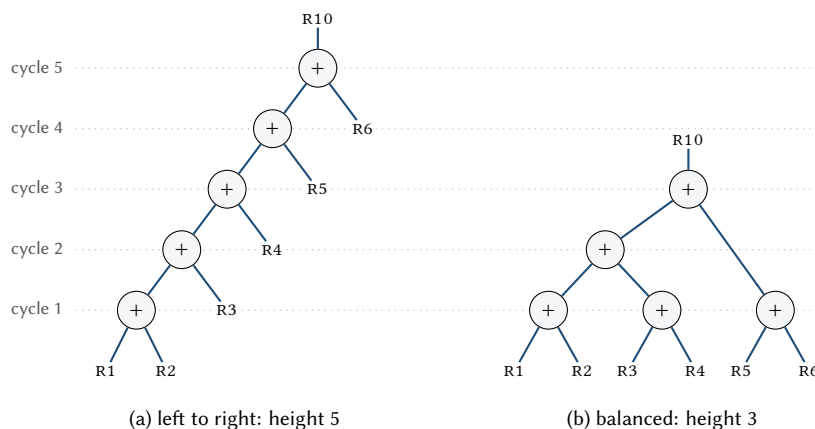


Figure 10.1: Expression trees of the sum of R1 to R6. (a) The left-to-right translation is a chain of five additions. (b) The balanced tree of Example 10.2 has height 3. Each level of additions executes in one cycle of the ideal processor.

For n operands combined by one associative operation, the left-to-right translation has height $n - 1$, whereas a balanced tree has height $\lceil \log_2 n \rceil$. Addition and multiplication are associative, and so are AND, OR and XOR. Subtraction is not, since

Source Lesson 10, videos 3–4; H&P Appendix H.

▷ 木の高さ削減

Caution Floating-point addition is not associative (rounding); compilers regroup it only when permitted.

$(a - b) - c \neq a - (b - c)$, but an expression with subtractions can still be regrouped when the signs are adjusted. Because $a - b = a + (-b)$, the terms can be grouped freely as long as each keeps its sign, and the signs inside a group that follows a minus sign are inverted: for example $a - b - c - d = (a - b) - (c + d)$, which reduces the height from 3 to 2.

Key point

Tree height reduction changes the dependences between instructions, not their number. It shortens the chain that bounds the ILP, which pays off on processors that can execute several instructions per cycle.

10.3 Instruction scheduling

A processor executes independent instructions in the same cycle only if it finds them. A compiler usually translates a program statement by statement, so the instructions of one statement are followed by those of the next. When each statement forms a chain, the independent instructions of different statements are separated by the rest of the chain. An out-of-order processor whose window covers both chains finds them anyway, but an out-of-order processor with a shorter window does not, and neither does an in-order processor, which executes the next instructions only up to the first one that must wait. Three compiler techniques make independent instructions easier to find: instruction scheduling, loop unrolling (Section 10.5) and trace scheduling (Section 10.7).

Definition 10.3 (Instruction scheduling). Reordering the instructions of a program, without changing its results, so that instructions that can execute in the same cycle are adjacent in program order is called *instruction scheduling*.

For an in-order processor, scheduling fills the cycles in which the processor would otherwise wait with instructions that do not depend on the waiting one. To schedule for a processor that executes up to w instructions per cycle, the compiler can determine for every instruction the cycle in which the ideal processor would execute it (Equation (6.2)) and emit the instructions in the order of these cycles, in groups of at most w . Every instruction then still follows the instructions whose results it reads, but the reordered program must also compute the same results as the original. A move that breaks a WAW or WAR dependence changes a value that some instruction uses; the compiler then either keeps the original order or compensates:

Destination register When a moved instruction would overwrite a register whose old value a later instruction still reads, it writes its result to another, otherwise unused register instead, and the instructions that read the result are changed accordingly.

Address offset When an instruction that adds a constant to a register, such as an index increment, moves ahead of an instruction that uses this register as the base of an address, the displacement of the address is reduced by the constant: `SW R3, 0(R1)` placed after `ADDI R1, R1, 4` becomes `SW R3, -4(R1)`.

Example 10.4. With a to h in $R1$ to $R8$, the statements $x = (a + b) \times c - d$ and $y = (e - f) \times g + h$ compile into

```
I1:  ADD R9, R1, R2      ; a + b
I2:  MUL R9, R9, R3      ; (a + b) * c
I3:  SUB R9, R9, R4      ; x <- (a + b) * c - d
I4:  SUB R10, R5, R6     ; e - f
I5:  MUL R10, R10, R7    ; (e - f) * g
I6:  ADD R10, R10, R8    ; y <- (e - f) * g + h
```

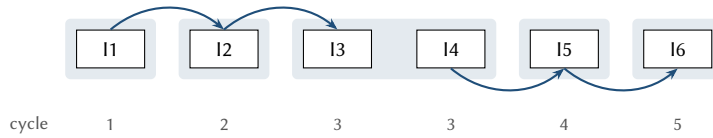
The instructions form two chains, I1–I3 and I4–I6, so the ideal processor executes I1 and I4 in cycle 1, I2 and I5 in cycle 2, and I3 and I6 in cycle 3: the ILP is 2. A two-wide in-order processor executes only I1 in cycle 1, because I2 must wait for it, and only I2 in cycle 2. In cycle 3 it executes I3 together with I4, but then I5 and I6 again need a

Source Lesson 10, videos 5–7; H&P §3.2.

▷ 命令スケジューリング

cycle each. It takes five cycles, an IPC of $6/5 = 1.2$ (Figure 10.2a). Scheduled in the order I1, I4, I2, I5, I3, I6, every pair of adjacent instructions is independent, and the same processor needs three cycles, an IPC of 2, equal to the ILP (Figure 10.2b). The two statements use different registers, so the reordering breaks no WAW or WAR dependence.

(a) statement by statement: 6 instructions in 5 cycles



(b) scheduled: 6 instructions in 3 cycles

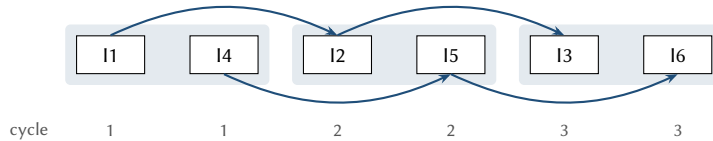


Figure 10.2: The program of Example 10.4 on a two-wide in-order processor. Arcs are RAW dependences; shaded groups execute in the same cycle. (a) In the order of the statements, the chain I1–I3 holds back I4. (b) After scheduling, both chains advance in every cycle.

Example 10.5. The loop

```
for (i = 0; i < n; i++)
    a[i] = a[i] + x;
```

adds x to each of the n elements of an integer array. With R1 pointing to the current element, R2 holding x , and R5 holding the address just past the last element, it compiles into

```
Loop: LW    R3, 0(R1)      ; R3 <- a[i]
      ADD   R3, R3, R2     ; R3 <- a[i] + x
      SW    R3, 0(R1)      ; a[i] <- R3
      ADDI  R1, R1, 4      ; next element
      BNE   R1, R5, Loop   ; repeat until the end
```

A four-wide in-order processor with perfect branch prediction executes the LW alone, because the ADD waits for it, and then the ADD alone, because the SW waits for it. The SW and the ADDI execute together, and the BNE, which waits for the ADDI, executes together with the LW of the next iteration. Every iteration takes three cycles, a CPI of $3/5 = 0.6$. Moved up behind the LW, the ADDI no longer waits behind the chain, and the SW, which now follows the increment, needs the displacement -4 :

```
Loop: LW    R3, 0(R1)
      ADDI  R1, R1, 4
      ADD   R3, R3, R2
      SW    R3, -4(R1)      ; R1 already advanced
      BNE   R1, R5, Loop
```

This is the order of the ideal cycles, with the branch kept at the end of the iteration. The LW and the ADDI now execute together, the ADD follows, and the SW, the BNE and the LW and ADDI of the next iteration fill all four places of the next cycle. Every iteration takes two cycles, a CPI of $2/5 = 0.4$.

Scheduling thus matters for in-order processors, for out-of-order processors whose window is short compared with the chains in the program, and for out-of-order processors whenever more instructions are ready than their width, since the order decides which of them execute first (Lesson 6).

10.4 Scheduling and if-conversion

Instruction scheduling can move instructions only within a stretch of code that is executed as a whole, and a conditional branch ends such a stretch. An instruction that follows the branch is executed on only one of its paths. Moving it ahead of the branch would execute it on the other path as well, and instructions of the two paths cannot be placed in the same group, because only one of the paths is executed. The compiler therefore cannot group instructions across a branch, even when the instructions before and after it are independent.

If-conversion (Lesson 5) removes this boundary for a small if-then-else. The converted code computes the condition and both parts unconditionally and selects the results with conditional moves, all in one branch-free sequence, which the compiler can schedule like any other: the instructions that compute the condition and the two parts do not depend on the selections, so they are scheduled into the first groups according to their own dependences, and the selections follow. The if-converted code of Example 5.4 already has this shape. Its first four instructions are independent of each other and the three conditional moves depend only on them, so a four-wide in-order processor executes all seven instructions in two cycles.

Source Lesson 10, videos 8–9; H&P §3.2.

10.4.1 Loops

A loop ends every iteration with a branch back to its beginning, and this branch cannot usefully be if-converted. Removing it would mean executing the body as many times as the loop could ever iterate, and each copy would need its own predicate telling whether its iteration should have been executed; the instructions that compute and apply these predicates would cost more than scheduling could gain. Loop branches are, moreover, predicted accurately, and predicating one would execute the code that follows the loop in every iteration (Table 5.1). The loop branch therefore stays, and it confines instruction scheduling to a single iteration: the scheduled body of Example 10.5, with two cycles per iteration on the four-wide in-order processor, is the best order for that processor. When the body is short, most of the independent instructions of the loop lie in different iterations, where the compiler cannot bring them together. Loop unrolling addresses this limitation.

10.5 Loop unrolling

In a loop with a short body, a large fraction of the instructions executed are loop overhead, and the work is spread over many iterations separated by the loop branch. In the loop of Example 10.5, three of the five instructions do the work; the ADDI and the BNE are overhead.

Definition 10.6 (Loop unrolling). Rewriting a loop so that each iteration of the new loop performs the work of k iterations of the original loop, by repeating the body k times with adjusted indices and advancing the index by k , is called *loop unrolling*. The number k is the *unrolling factor*: unrolling a loop once gives $k = 2$, unrolling it twice $k = 3$.

Unrolled with $k = 2$, for an even n , the loop becomes

```
for (i = 0; i < n; i += 2) {
    a[i]    = a[i]    + x;
    a[i+1] = a[i+1] + x;
}
```

and compiles into

```
Loop: LW    R3, 0(R1)      ; a[i] <- a[i] + x
      ADD   R3, R3, R2
      SW    R3, 0(R1)
      LW    R3, 4(R1)     ; a[i+1] <- a[i+1] + x
      ADD   R3, R3, R2
```

Source Lesson 10, videos 10–15; H&P §3.2.

▷ ループ展開

```

SW    R3, 4(R1)
ADDI  R1, R1, 8      ; two elements further
BNE   R1, R5, Loop

```

Each iteration now processes two elements with eight instructions, of which the index increment and the branch are executed once. The two copies of the body are adjacent and separated by no branch, so the compiler can schedule them together once the second copy keeps its intermediate result in a register of its own (Section 10.5.2).

If n is not a multiple of k , the compiler adds code, typically a copy of the original loop, for the remaining elements.

10.5.1 Instruction count and ILP

Unrolling does not affect the clock period, but it changes the other two factors of the iron law (Equation (2.3)). The original loop executes 5 instructions per element, the unrolled loop 8 per two elements, or 4 per element: the overhead instructions are executed half as often, and the instruction count falls by 20 %. With a factor k the loop executes $(3k + 2)/k$ instructions per element, which approaches 3, the work alone.

The ILP rises as well. On the ideal processor (Equation (6.2)) the ADDIs of successive iterations form a chain of one cycle per iteration. Every LW of an iteration reads R1 from the ADDI of the previous iteration and executes in the same cycle as the ADDI of its own iteration, and its ADD and SW follow in the next two cycles; the second write to R3 in the unrolled body is a WAW dependence that renaming removes. The ideal processor thus executes one iteration, or k elements, per cycle, and the ILP of a long loop approaches the number of instructions per iteration: 5 for the original loop, 8 for the unrolled one. This relies on iterations that depend on each other only through the index: in a loop that accumulates, such as $s = s + a[i]$, the additions to s form a chain of one cycle per element in either version.

10.5.2 Cycles per instruction

Whether the CPI falls as well depends on the processor. Table 10.2 compares four versions of the loop on the four-wide in-order processor of Example 10.5 and on the ideal processor, both with perfect branch prediction. The first two versions are those of Example 10.5. Unrolled but not scheduled, the loop still executes one LW–ADD–SW chain after the other: the second LW joins the first SW, the ADDI joins the second SW, and the BNE joins the LW of the next iteration, so every iteration of eight instructions takes five cycles. The CPI rises from 0.6 to $5/8 = 0.625$, although the time per element falls from 3 to 2.5 cycles with the instruction count. Scheduled in the order of the ideal cycles, the unrolled body becomes

```

Loop: LW    R3, 0(R1)
      LW    R4, 4(R1)      ; second copy uses R4
      ADDI  R1, R1, 8
      ADD   R3, R3, R2
      ADD   R4, R4, R2
      SW    R3, -8(R1)     ; R1 already advanced
      SW    R4, -4(R1)
      BNE   R1, R5, Loop

```

A loop branch is mispredicted about once per run, at the exit; a remainder loop adds at most one more.

Both compensations of Section 10.3 are needed. The second LW now precedes the ADD and the SW of the first copy, which still need R3, so the second copy computes its value in R4; and both stores follow the increment, so their displacements are reduced by 8. The processor executes both LWs and the ADDI in the first cycle and both ADDs in the second, and the two SWs, the BNE and the first LW of the next iteration fill the third. From then on the groups no longer coincide with the iterations, and in the steady state 16 instructions, two iterations, take 5 cycles: 1.25 cycles per element, a CPI of $5/16 = 0.3125$. Compared with the scheduled original loop, the time per element falls from 2 to 1.25 cycles, a speedup of 1.6, of which $5/4 = 1.25$ comes from the instruction count and $0.4/0.3125 = 1.28$ from the CPI.

On the in-order processor, unrolling thus lowers the CPI through the scheduling it makes possible: the loop branch no longer keeps the instructions of two iterations apart, and the compiler interleaves them. On the ideal processor the order of the

Table 10.2: The loop of Example 10.5 before and after unrolling with $k = 2$, each as compiled and scheduled: instructions per element, and cycles per element and CPI on two processors with perfect branch prediction.

Version	Instructions	Four-wide in-order		Ideal	
		Cycles	CPI	Cycles	CPI
Original	5	3.00	0.600	1.00	0.200
Scheduled	5	2.00	0.400	1.00	0.200
Unrolled	4	2.50	0.625	0.50	0.125
Unrolled and scheduled	4	1.25	0.3125	0.50	0.125

instructions does not matter. There the CPI falls because the ILP rises from 5 to 8, and together with the lower instruction count the time per element halves; an out-of-order processor with a large window profits in the same way, up to its width. A processor that completes one instruction per cycle has a CPI of 1 in every version and gains only the factor 1.25 from the instruction count. Loop unrolling therefore lowers the instruction count on every processor, and the CPI on processors that execute several instructions per cycle.

10.5.3 Downsides of unrolling

The body grows linearly with k , from 5 instructions to $3k + 2$, whereas the gain levels off: the instructions per element approach 3, and a real processor cannot raise its IPC beyond its width, however high the ILP. Going from $k = 4$ (14 instructions, 3.5 per element) to $k = 8$ (26 instructions, 3.25 per element) nearly doubles the body to execute only 7 % fewer instructions, an instance of the diminishing returns of Lesson 2.

Definition 10.7 (Code bloat). The growth of the program size caused by a transformation such as loop unrolling is called *code bloat*.

Larger code occupies more memory, and, as the lessons on caches will show, it causes more cache misses when its instructions are fetched, which can cancel the gain. Two further problems concern the number of iterations. When n is not a multiple of k , the remaining iterations need additional code. And unrolling presumes that the number of remaining iterations is known when an iteration of the unrolled loop starts. A loop whose exit is decided inside the body, such as a search that stops at the first zero element, cannot simply execute k copies of the body per iteration, because the later copies could perform iterations that the original loop would not; it needs additional exit checks. Compilers therefore unroll with small factors, and only loops whose bodies are short.

Key point

Loop unrolling executes the index increment and the loop branch once per k iterations of the original loop, which lowers the instruction count on every processor. It also lowers the CPI on processors that execute several instructions per cycle: on in-order processors because the compiler can schedule the instructions of several iterations together, on processors that find the parallelism themselves because the ILP of loops with independent iterations rises. Its costs are code that grows with k and extra code when the number of iterations is not a multiple of k or not known in advance.

▷ コード肥大化

10.6 Function call inlining

A call separates the instructions of the caller from those of the called function, much as a branch separates the two paths of an if-then-else. In addition, every call executes instructions that perform none of the function's work: the call and the return themselves, the copying of the arguments into the registers prescribed by the call-

Source Lesson 10, videos 16–18; H&P Appendix A.

ing convention and of the result out of them, and often the saving and restoring of registers on the stack.

Definition 10.8 (Function call inlining). Replacing a call by a copy of the body of the called function, adapted to the operands of the caller, is called *function call inlining*.

Example 10.9. The function

```
int sqdiff(int x, int y) { int d = x - y; return d * d; }
```

is called as `c = sqdiff(a, b)`, with *a*, *b* and *c* in R8, R9 and R10. The calling convention passes the arguments in R4 and R5 and returns the result in R2:

```

      ADD R4, R8, R0      ; x <- a
      ADD R5, R9, R0      ; y <- b
      CALL SqDiff
      ADD R10, R2, R0      ; c <- result
      ...
SqDiff: SUB R2, R4, R5      ; d <- x - y
      MUL R2, R2, R2      ; result <- d * d
      RET
```

Inlined, the call becomes

```

      SUB R10, R8, R9      ; c <- a - b
      MUL R10, R10, R10    ; c <- c * c
```

Each call executes seven instructions, the inlined code two. The argument and result copies also lengthen the dependence chain from *a* and *b* to *c*: on the ideal processor the call takes four cycles, the inlined code two (Figure 10.3).

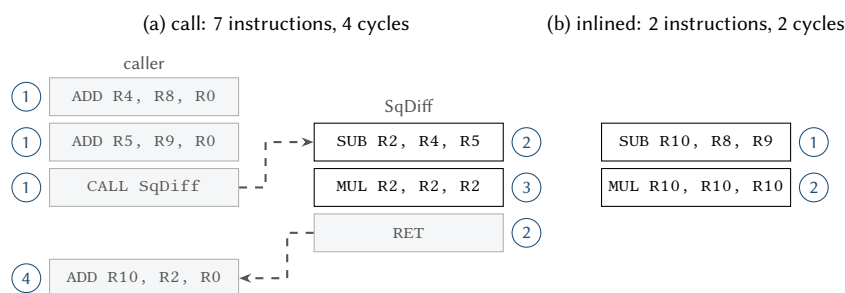


Figure 10.3: Example 10.9 before and after inlining. Shaded instructions only implement the call; dashed arrows are the transfers of control; circled numbers give the cycle in which the ideal processor executes each instruction.

Inlining thus lowers the instruction count, since the call, the return and the copying of arguments and results disappear, and it shortens dependence chains, since the body reads the caller's operands directly and delivers its result where the caller needs it. Moreover, the body becomes part of the caller's code: the compiler can schedule its instructions together with those of the caller and apply tree height reduction across the former call, and the processor no longer has to predict the call and the return (Lesson 4).

10.6.1 Code bloat from inlining

Inlining places a separate copy of the body at every call site, and for large functions called from many sites the code grows considerably.

Example 10.10. A function with a body of 20 instructions and a return is called from 10 sites, each of which spends 4 instructions on the call (two argument copies, the call and a result copy). Without inlining the code holds $10 \times 4 + 21 = 61$ instructions, and each call executes $4 + 21 = 25$. Inlined at all sites, the function and the call sequences disappear, but every site receives the 20 instructions of the body: 200 instructions, more than three times the code, to save 5 of the 25 instructions (20 %) of each call.

▷ インライン展開

For the function of Example 10.9, whose body has two instructions, the same 10 sites need $10 \times 4 + 3 = 43$ instructions without inlining and only $10 \times 2 = 20$ with it: the code shrinks, and each call executes 2 instructions instead of 7.

As with loop unrolling, the larger code causes more cache misses and can cancel the gain. Compilers therefore inline small functions, whose call overhead is large compared with their body, and functions called from a single site, whose one copy replaces the original; large functions called from many sites remain calls.

Key point

Function call inlining removes the call, the return and the passing of arguments and results, lowering the instruction count and shortening dependence chains, and it lets the compiler optimize the body together with the caller. It pays off for small functions; for large functions called from many sites, code bloat outweighs the gain.

10.7 Other compiler techniques

The techniques of this lesson are examples of a larger body of compiler transformations that raise the IPC. Two further techniques carry instruction scheduling across the boundaries that confine it; they are only outlined here. In *software pipelining* the compiler treats a loop like a pipeline: it schedules the instructions of successive iterations so that the iterations overlap, and every cycle combines independent instructions of several iterations while the results stay those of the original loop [17]. The second, *trace scheduling*, carries if-conversion further: the compiler uses a profile to select the most frequently executed path through several branches, schedules this path as one block of code, and adds checks that branch to compensation code, which corrects the results, when execution leaves the path [18]. Carried to its end, scheduling by the compiler leads to processors that leave the grouping of operations into parallel steps entirely to the compiler: the VLIW processors of Lesson 11.

Key point — Lesson summary

The IPC is limited by the ILP of the program and by the independent instructions a processor can find within its window; the compiler influences both. Tree height reduction regroups expressions so that their instructions form shorter chains, raising the ILP at the same instruction count. Instruction scheduling makes independent instructions adjacent, changing destination registers and address offsets where instructions move past each other; it helps in-order and small-window processors. Branches confine it, and if-conversion removes them from small if-then-else constructs. Loop unrolling executes the index increment and the loop branch once per k iterations, lowering the instruction count; it also lowers the CPI, because the compiler can schedule the instructions of several iterations together and because the ILP of loops with independent iterations rises. Function call inlining removes calls, returns and argument copying. Both cause code bloat, so compilers reserve them for short loops and small functions. Software pipelining and trace scheduling extend scheduling across loop iterations and across branches.

Source Lesson 10, videos 19–20; [17]; [18].

▷ ソフトウェアパイプラインニング

▷ トレーススケジューリング

VLIW

Lecture	Lesson 11, videos 1–9
Textbook	H&P [1] §3.7, Appendix H
Keywords	very long instruction word, VLIW, out-of-order superscalar, in-order superscalar, NOP, backward compatibility, irregular code, code bloat, full predication, branch hint, stop bit, Itanium, EPIC, digital signal processor

Lesson 10 showed that the compiler can raise the number of instructions a processor completes per cycle by rewriting the program: it shortens chains of dependences, places independent instructions next to each other, and removes branches and calls. In a very long instruction word (VLIW) processor this idea is taken to its end point: the compiler alone decides which operations execute in parallel and encodes the decision in the instructions, so that the hardware needs none of the dependence-tracking machinery of the out-of-order processors of Lessons 7–9. This lesson compares VLIW with superscalar designs, weighs its advantages against its disadvantages, and shows where the approach has failed and where it has succeeded.

11.1 Superscalar and VLIW processors

A superscalar processor (Lesson 8) executes up to N instructions per cycle. Designs differ in how many instructions they examine to find N that can execute together, and in how much of that search they leave to the compiler (Figure 11.1 and Table 11.1). An out-of-order design finds the parallelism of any program by itself, at the price of complex hardware for renaming, window scheduling and the reorder buffer (Lessons 6–8). An in-order design stops at the first instruction that must wait for another, so it needs the compiler to place independent instructions next to each other (Lesson 10). A VLIW processor executes, without any checking, one instruction of up to N operations that the compiler has made independent.

Source Lesson 11, videos 1–3; H&P §3.7; [19].

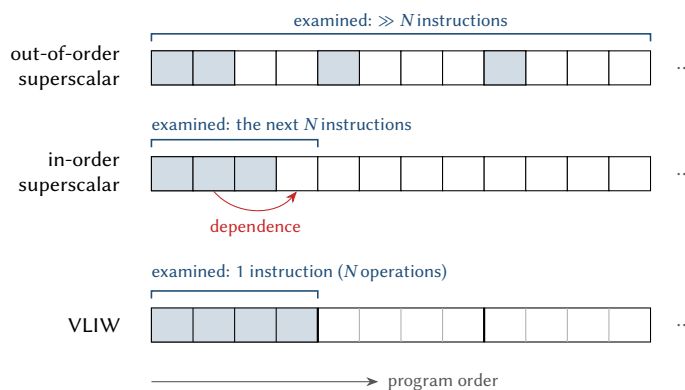


Figure 11.1: What each design examines to fill one cycle with up to $N = 4$ operations. Shaded boxes are executed in this cycle; the in-order design stops at the first instruction that depends on an earlier one in its group.

Definition 11.1 (Very long instruction word). A *very long instruction word* (VLIW) architecture is an instruction set architecture in which each instruction consists of up to N operations, each equivalent to one instruction of an ordinary ISA, that are executed in parallel in the same cycle. Choosing the operations that form an instruction, and ensuring that they are independent, is the task of the compiler; the processor performs no dependence checking.

▷ VLIW

An instruction that carries N operations is about N times as long as an ordinary one, which gives the architecture its name. Since it does the work of up to N ordinary

instructions, a VLIW processor that completes one instruction per cycle is comparable to a superscalar processor with an IPC of up to N (Lesson 6).

Table 11.1: Superscalar and VLIW processors that execute up to N operations per cycle.

	Out-of-order	In-order	VLIW
Executes per cycle	$\leq N$ instructions	$\leq N$ instructions	1 instr. ($\leq N$ ops)
Examines	$\gg N$ instructions	next N instructions	1 instruction
Hardware	complex	simpler	simplest
Compiler	can help	must help	does all the work

Example 11.2 (Packing operations into instructions). The fragment below adds four array elements and stores the sum. The comments list its RAW dependences (Lesson 3); the four loads only read R8 and are independent of each other.

```

I1: LW  R1, 0(R8)
I2: LW  R2, 4(R8)
I3: LW  R3, 8(R8)
I4: LW  R4, 12(R8)
I5: ADD R5, R1, R2    ; RAW with I1, I2
I6: ADD R6, R3, R4    ; RAW with I3, I4
I7: ADD R7, R5, R6    ; RAW with I5, I6
I8: SW  R7, 16(R8)    ; RAW with I7

```

A VLIW compiler takes the operations in program order and places each one in the earliest instruction that follows all the operations it depends on and still has a free slot; slots left over are filled with NOPs (Table 11.2). With four slots the eight operations occupy four instructions, hence four cycles, where an ordinary ISA needs eight instructions; with two slots they occupy five.

Table 11.2: The fragment of Example 11.2 compiled for VLIW ISAs with four and with two operation slots. Each row is one instruction; sizes assume 32-bit slots.

Instruction	4 slots	2 slots
1	I1, I2, I3, I4	I1, I2
2	I5, I6, NOP, NOP	I3, I4
3	I7, NOP, NOP, NOP	I5, I6
4	I8, NOP, NOP, NOP	I7, NOP
5	—	I8, NOP
NOP slots	8 of 16	2 of 10
Code size	512 bits	320 bits

11.2 Advantages and disadvantages

Leaving the search for parallelism to the compiler has three advantages.

- **The compiler has plenty of time.** An out-of-order processor must find independent instructions within a fraction of every clock cycle and only in its window; the compiler runs once and can analyze the whole program.
- **The hardware is simpler.** Without renaming, window scheduling and a reorder buffer, a VLIW processor needs less area and power for the same operations per cycle, so it can be more energy efficient.
- **Loops and regular code work well.** When arrays are processed in loops, the dependences are few and visible at compile time, and loop unrolling (Lesson 10) removes the branches between iterations, so that their independent operations fill the slots.

It also has four disadvantages.

- **Latencies are not known at compile time.** The compiler groups operations as if each completes within its cycle. If one takes longer, e.g. a load with a

Caution Instruction counts and CPI of a VLIW program are not comparable with those of an ordinary ISA; compare operations per cycle.

Source Lesson 11, videos 4–5; H&P §3.7, Appendix H.

cache miss (Lesson 12), all later operations wait for it, even independent ones, whereas an out-of-order processor keeps executing instructions that do not depend on it.

- **Irregular code is hard.** The compiler sees the program, not its execution. When it cannot prove two operations independent—a store and a load through pointers that may address the same location, or operations on either side of a data-dependent branch—it must place them in different instructions, whereas an out-of-order processor knows the actual addresses and predicts the branches at run time (Lessons 4 and 9).
- **Code bloat.** Whenever fewer than N independent operations are available, NOPs fill the slots and the program grows: the four-slot version in Table 11.2 is half NOPs and occupies 512 bits, against 256 bits for eight ordinary 32-bit instructions. Compaction (Section 11.3) reduces this overhead.
- **A new processor needs recompiled programs.** The grouping of operations and the number of slots are fixed at compile time, which undermines backward compatibility (below).

Definition 11.3 (Backward compatibility). A processor offers *backward compatibility* when it runs, unmodified, the programs compiled for earlier processors of its family.

For superscalar processors backward compatibility comes with a performance gain: a wider out-of-order processor finds more parallelism in an *old* binary without recompilation. A new VLIW processor with more slots either cannot run an old VLIW binary at all, because the instruction format has changed, or runs it with no more operations per cycle than the processor it was compiled for.

Example 11.4 (An old binary on a wider VLIW processor). A four-slot processor that still accepts the two-slot format executes the two-slot version of Example 11.2 in five cycles, as the old processor does; only the recompiled four-slot version takes four. The eight ordinary instructions, counted the same way, take five cycles on a two-wide and four on a four-wide out-of-order processor, without recompilation.

Key point

VLIW trades hardware complexity for dependence on the compiler. It pays off when the code is regular and compiled for the exact processor; it loses when parallelism or operation latencies become known only at run time, or when old binaries must run faster on new processors.

11.3 VLIW instructions

A VLIW instruction is a sequence of N operation slots (Figure 11.2). Each slot is encoded like an instruction of an ordinary ISA, with its own opcode from the full set of ordinary opcodes and its own operands.

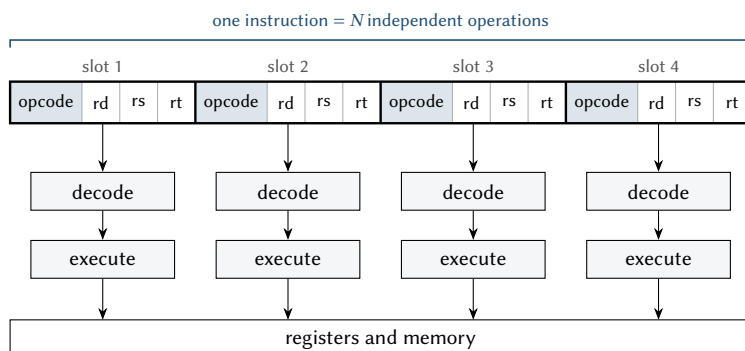


Figure 11.2: A VLIW instruction with four operation slots. Each slot is decoded and executed independently of the others; no hardware checks for dependences between slots.

The operations of one instruction must be independent: none may use the result of

▷ 後方互換性

Source Lesson 11, video 6; H&P Appendix H.

another, and no two may write the same location. The hardware does not check this, so correct grouping is the responsibility of the compiler. Operations in *consecutive* instructions may depend on each other, as I5 depends on I1 in Table 11.2.

The compiler exposes the parallelism with instruction scheduling, as in Example 11.2, helped by the other techniques of Lesson 10: tree height reduction shortens dependence chains so that more operations can share an instruction, and loop unrolling and function call inlining remove the branches, calls and returns that would separate operations. A VLIW processor has no renaming hardware, so the compiler itself removes false dependences (Lesson 3) by writing results into otherwise unused registers, of which the instruction set provides many.

Branches limit the grouping, since the operations after a branch execute only on one of its paths. A VLIW instruction set therefore provides good support for predication (Lesson 5), ideally full predication, so that the compiler can if-convert branches and group the operations of both paths. A branch that remains can carry a *branch hint*, with which the compiler tells the processor which way it expects the branch to go.

Fixed slots waste space whenever fewer than N independent operations are available. Compacted instructions store the operations one after another without NOPs, and a *stop bit* marks the last operation of each group that executes together. In the four-slot program of Table 11.2, I4, I6, I7 and I8 carry a stop bit; at 33 bits per operation the code occupies 264 bits instead of 512.

11.4 VLIW processors in practice

Two kinds of processors show where the VLIW approach fails and where it succeeds.

11.4.1 Itanium

The most prominent attempt to bring VLIW-style instructions to general-purpose computing was *Itanium*, the processor family Intel built for the IA-64 instruction set; its designers called the approach EPIC, explicitly parallel instruction computing [20]. The instruction set, however, accumulated many additional features whose hardware became very complex, removing the main advantage of VLIW. The compiler still found little parallelism in irregular code, and Itanium did not displace out-of-order processors.

11.4.2 Digital signal processors

A *digital signal processor* (DSP) is a processor specialized for digital signals such as audio, video and radio. This market plays to the strengths of VLIW (Section 11.2).

- Loops that apply the same computation to many samples dominate the code: regular code in which the compiler finds plenty of parallelism.
- Programs are usually compiled for the particular DSP of the device they run in, so backward compatibility of binaries matters little.
- DSPs sit in power-limited devices, where energy-efficient hardware pays off.

Key point – Lesson summary

An out-of-order superscalar processor examines many more than N instructions and needs no help from the compiler; an in-order one examines N and needs that help; a VLIW processor examines one instruction of up to N independent operations and relies on the compiler entirely. VLIW hardware is simple and energy efficient and performs well on regular loop code, but binaries must be recompiled for each new processor, irregular code yields little parallelism, a cache miss on one load delays all later operations, and NOPs bloat the code unless it is compacted with stop bits. Itanium failed in general-purpose computing; DSPs are where VLIW fits. Lesson 12 turns from instructions per cycle to memory.

Caution Independence is required only within an instruction, not between instructions.

▷ 分岐ヒント

▷ ストップビット

Source Lesson 11, videos 7–9; H&P §3.7, Appendix H.

▷ Itanium

IA-64 uses stops in the same way to end groups of independent instructions (Section 11.3).

▷ DSP

H&P lists the TI C6x DSPs as an example of the VLIW approach.

Cache Review

Lecture	Lesson 12, videos 1–45
Textbook	H&P [1] §2.1, §B.1–B.2
Keywords	locality principle, temporal locality, spatial locality, cache hit, cache miss, AMAT, miss penalty, block offset, tag, valid bit, direct-mapped cache, set-associative cache, LRU, dirty bit

Lessons 3–11 made processors complete ever more instructions per second, yet every instruction has to be fetched from memory, many also load or store data, and Lesson 1 showed that memory speed falls ever further behind processor speed (the memory wall). This lesson reviews the mechanism that bridges the gap: the cache, a small and fast memory that serves most accesses without waiting for the large and slow main memory, because programs access memory with locality. It covers how cache performance is measured, how a cache is organized and searched, which block is evicted on a miss, and how writes are handled; virtual memory (Lesson 13) and the advanced cache techniques of Lesson 14 build directly on this material.

12.1 The locality principle

Caches work because the memory accesses of a program are far from random.

Definition 12.1 (Locality principle). The observation that what a program does in the near future is likely to be close to what it has done in the recent past is called the *locality principle*.

For memory references, “close” has two meanings. Suppose a program has just accessed address X .

Definition 12.2 (Temporal and spatial locality). A program exhibits *temporal locality* when it is likely to access the same address X again soon. It exhibits *spatial locality* when it is likely to access addresses near X soon.

Both kinds follow from the way programs are structured. A loop executes the same instructions over and over (temporal locality), and those instructions are fetched from consecutive addresses (spatial locality). A variable updated in every iteration is accessed at the same address again and again; the elements of an array processed in order lie at consecutive addresses.

Example 12.3. The loop below copies $R6$ bytes from the buffer at address $R4$ to the buffer at address $R5$.

```

Loop:  LB   R3, 0(R4)      ; R3 <- source byte
       SB   R3, 0(R5)      ; destination byte <- R3
       ADDI R4, R4, 1      ; next source byte
       ADDI R5, R5, 1      ; next destination byte
       ADDI R6, R6, -1     ; one byte less to copy
       BNE  R6, R0, Loop   ; repeat for all bytes

```

The six instruction fetches have both kinds of locality: they go to consecutive addresses, and the same six addresses are fetched in every iteration. The data accesses form two streams, one through each buffer. Every byte is read or written exactly once, so the data has no temporal locality. It has spatial locality nevertheless: the accesses alternate between two buffers that may be far apart, but each of them is adjacent to the previous access to the same buffer. Spatial locality only requires accesses near *recently* accessed addresses, not near the one accessed last.

Source Lesson 12, videos 1–6; H&P §2.1; [21].

▷ 局所性の原理

▷ 時間的局所性
▷ 空間的局所性

12.2 Caches

Locality suggests how to give the processor fast access to a large memory. Going to main memory for every fetch, load and store is slow. Making all of main memory as fast as the processor is not possible either: fast memory is expensive per bit, and a larger memory is inherently slower to access. The remaining option is to keep a small amount of fast memory inside the processor itself and to place in it the data that was accessed recently, together with the data around it. By temporal and spatial locality, most accesses then find their data there.

Definition 12.4 (Cache). A *cache* is a small, fast memory that holds copies of the recently accessed parts of a larger and slower memory, so that most accesses can be served from the copies.

Every memory access of the processor first looks for its data in the cache (Figure 12.1). If the data is there, the access is a *cache hit*, and the data is read or written in the cache.

Otherwise the access is a *cache miss*: the data is obtained from main memory and copied into the cache, so that further accesses to it, and by spatial locality to its neighbours, will hit. Because the cache is consulted on every access, the lookup itself must be very fast, which is a second reason, besides cost, why the cache has to be small. A small cache cannot hold everything, and misses are therefore unavoidable.

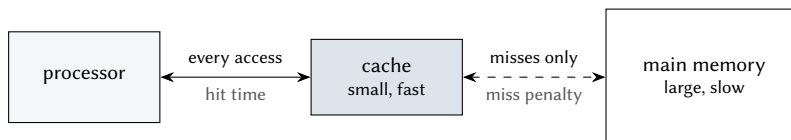


Figure 12.1: Every memory access goes to the cache; only misses go on to main memory.

12.3 Cache performance

The benefit of a cache is measured by the average time a memory access takes. The following quantities enter it.

Definition 12.5 (Hit rate, miss rate). The *hit rate* is the fraction of memory accesses that are cache hits; the *miss rate* is the fraction that are misses. Hit rate + miss rate = 1.

Definition 12.6 (Hit time, miss penalty). The *hit time* is the time needed to look up the cache and access the data in it. The *miss penalty* is the *additional* time a miss takes: fetching the data from main memory and placing it in the cache.

A miss also pays the hit time, since the cache has to be searched before the miss is detected; a miss therefore takes the hit time plus the miss penalty. Weighting both cases by their frequency gives the *average memory access time* (AMAT):

$$\begin{aligned} \text{AMAT} &= (1 - \text{miss rate}) \times \text{hit time} + \text{miss rate} \times (\text{hit time} + \text{miss penalty}) \\ &= \text{hit time} + \text{miss rate} \times \text{miss penalty}. \end{aligned} \quad (12.1)$$

A good AMAT requires three things.

- A low hit time: the cache must be small and simple.
- A low miss rate: the cache must be large enough, and organized cleverly enough, to hold the data the program is about to use.
- A low miss penalty. The miss penalty is dominated by the access time of main memory and is large; Lesson 14 reduces it with further levels of cache.

The first two goals conflict: a larger cache misses less often but takes longer to search. The best cache size balances them, and AMAT is the measure by which they are balanced.

Source Lesson 12, videos 7–9; H&P §B.1; [22].

▷ キャッシュ

▷ キャッシュヒット

▷ キャッシュミス

Source Lesson 12, videos 10–12; H&P §B.2.

▷ ヒット率

▷ ミス率

▷ ヒット時間

▷ ミスパナルティ

▷ 平均メモリアクセス時間

Caution The miss penalty is the *extra* time of a miss: subtract the hit time from the total time of a miss.

Example 12.7. Main memory adds a miss penalty of 60 ns. Three candidate caches are compared:

Cache	Hit time	Miss rate	AMAT
Small	1 ns	6 %	$1 + 0.06 \times 60 = 4.6$ ns
Medium	1.5 ns	3 %	$1.5 + 0.03 \times 60 = 3.3$ ns
Large	3 ns	2 %	$3 + 0.02 \times 60 = 4.2$ ns

The medium cache is best. The large cache misses less often, but its longer hit time costs more than the misses it saves. All three are an order of magnitude better than accessing main memory on every access.

Note 12.8. A cache pays off only if a hit is much faster than an access to main memory and misses are rare. With a hit time close to the access time of main memory, even a hit is about as slow as an access without a cache; with frequent misses, the miss penalty dominates. In both cases AMAT approaches or even exceeds the access time of main memory alone.

12.3.1 Caches in real processors

Real processors have more than one level of cache. The cache that is consulted first, and that directly serves the reads and writes of the processor, is the smallest and fastest; only its misses go on to further, larger cache levels (L2, L3; Lesson 14) and finally to main memory. This first level is called the *L1 cache*. L1 caches are typically 16–64 KB (1 KB = 2^{10} bytes): large enough that about 90 % of accesses hit, and small enough that a hit takes only one to three clock cycles.

Source Lesson 12, video 13; H&P §2.1.

▷ L1 キャッシュ

12.4 Blocks and lines

To determine quickly whether data is in the cache, the cache is organized as a table of fixed-size entries. Data is moved between main memory and the cache not byte by byte but in units called blocks.

Definition 12.9 (Cache block). A *cache block* is a run of consecutive bytes of memory that is copied into the cache, and evicted from it, as a unit. The number of bytes in a block is the *block size*.

The block size trades spatial locality against cache space.

- If blocks are very small (in the extreme, one byte), an access to the next byte, which spatial locality makes likely, misses again, and each byte needs its own bookkeeping.
- If blocks are very large, most of a block is data that will not be used soon. It occupies space in which other recently used data could have been kept, and fewer blocks fit into the cache.

Typical block sizes are 32–128 bytes. The number of blocks a cache can hold is its size divided by the block size: a 16 KB cache with 64-byte blocks holds $2^{14}/2^6 = 256$ blocks.

Source Lesson 12, videos 14–18; H&P §B.1.

▷ キャッシュブロック

▷ ブロックサイズ

12.4.1 Where blocks start

Blocks do not start at arbitrary addresses. If they did, a block covering addresses 100–227 and another covering 200–327 could both be in the cache, an address could lie in several cached blocks, and finding out whether an address is cached would require comparing address ranges. Instead, memory is divided into fixed, non-overlapping blocks that start at multiples of the block size: with 128-byte blocks, at addresses 0, 128, 256, ... (Figure 12.2). Every address then belongs to exactly one block.

Definition 12.10 (Cache line). A *cache line* is a slot in the cache that can hold one block. Memory is divided into blocks and the cache into lines; a line has the size of a block, and holds different blocks at different times.

▷ キャッシュライン

Block sizes, and therefore line sizes, are always powers of two. Only then can the block of an address and the position within the block be found by selecting bits of the address, as the next section shows, rather than by a division.

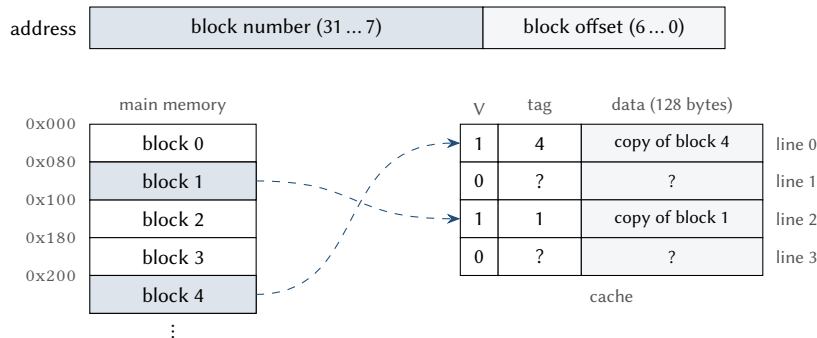


Figure 12.2: Main memory divided into 128-byte blocks and a cache of four lines. Lines 0 and 2 hold copies of blocks 4 and 1; lines 1 and 3 hold nothing valid.

12.5 Block number, tag and valid bit

With a block size of 2^k bytes, an address A splits into two fields.

Definition 12.11 (Block offset and block number). The low-order k bits of the address are its *block offset*, the position of the byte within its block. The remaining high-order bits are its *block number*, which identifies the block:

$$\text{block number} = \lfloor A/2^k \rfloor, \quad \text{block offset} = A \bmod 2^k. \quad (12.2)$$

Example 12.12. With 128-byte blocks ($k = 7$), the address $0x0040A7C6$ has block offset $0xC6 \& 0x7F = 0x46 = 70$ and block number $0x0040A7C6 \gg 7 = 0x814F$. The byte is at position 70 of block $0x814F$, which covers the addresses $0x0040A780$ to $0x0040A7FF$.

12.5.1 Tags

Since a line holds different blocks at different times, each line must record which block it currently holds. This record is the *tag* of the line: the block number of the block in the line, or, as the following sections show, the part of the block number that is not already implied by the position of the line. A lookup computes the block number of the address and compares it with the tags of the lines in which the block could be. A match is a hit, and the block offset then selects the byte within the line.

12.5.2 The valid bit

When the processor is turned on, the lines of the cache contain arbitrary bits, including arbitrary tags. A tag that happens to match a block number would produce a hit on data that was never loaded.

Definition 12.13 (Valid bit). The *valid bit* of a line indicates whether the line holds a block. All valid bits are cleared at start-up, and a line's valid bit is set when a block is loaded into it.

Emptying a line, or the whole cache, thus only requires clearing valid bits; the tags and data can be left as they are. The one exception is a block that has been written in the cache but not yet in main memory (Section 12.11): it must be written back first.

A lookup is a hit only if the line is valid and its stored tag matches the tag field of the address:

$$\text{hit} = V \wedge (\text{stored tag} = \text{tag of the address}). \quad (12.3)$$

Source Lesson 12, videos 19–23; H&P §B.1.

- ▷ ブロックオフセット
- ▷ ブロック番号

▷ タグ

▷ 有効ビット

As long as no part of the block number selects the line, as in this section, the tag field is the whole block number; the organizations of the following sections use some of its bits to select the line and keep only the remaining bits in the tag.

Key point

An address is a block number and an offset. The tag identifies which block a line holds, the valid bit whether it holds one at all, and the offset picks the byte. Where a block may be placed, the subject of the next section, decides *which* lines have to be searched.

12.6 Where a block can be placed

Caches differ in how many of their lines a given block may be placed in. This number is the *associativity* of the cache. It determines how many tags a lookup must compare, and how often different blocks compete for the same lines. Three organizations are distinguished (Figure 12.3):

Direct-mapped each block can be placed in exactly one line (Section 12.7).

Set-associative each block can be placed in any of N lines that form a set (Section 12.8).

Fully associative each block can be placed in any line of the cache (Section 12.9).

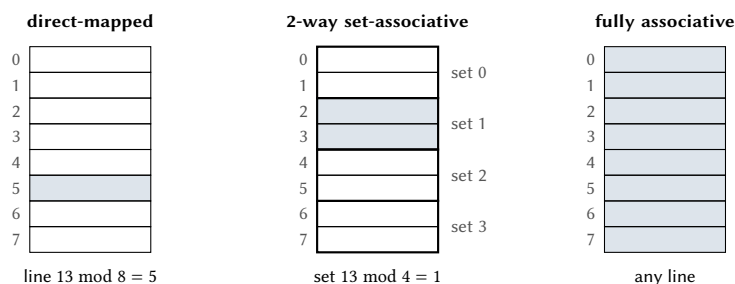


Figure 12.3: The lines in which block number 13 may be placed in an 8-line cache of each organization.

12.7 Direct-mapped caches

The simplest of the three organizations gives every block a single line of the cache. In a *direct-mapped cache* every block has exactly one line in which it can be placed, and that line is selected by the low-order bits of the block number.

Definition 12.14 (Index). The *index* is the group of address bits that selects where in the cache a block may be placed. In a direct-mapped cache with 2^n lines it consists of the n low-order bits of the block number.

The address is thus split into three fields (Figure 12.4): the offset in the k lowest bits, above it the index in the next n bits, and the tag in all remaining bits. The line stores only these tag bits, not the whole block number: all blocks that can be placed in line i have index i , so the index part of their block numbers is known from the line itself.

A lookup reads the one line selected by the index, compares its tag with the tag of the address, and checks the valid bit (Equation (12.3)).

Example 12.15. For a direct-mapped 64 KB cache with 128-byte blocks and 32-bit addresses there are $512 = 2^9$ lines: offset bits 6–0, index bits 15–7, tag bits 31–16. The address $0x0040A7C6$ of Example 12.12, block number $0x814F$, has index $0x814F \& 0x1FF = 0x14F = 335$ and tag $0x814F \gg 9 = 0x0040$. It hits if line 335 is valid and holds tag $0x0040$.

Source Lesson 12, video 24; H&P §B.1.

▷ 連想度

Source Lesson 12, videos 25–28; H&P §B.1.

▷ ダイレクトマップキャッシュ

▷ インデックス

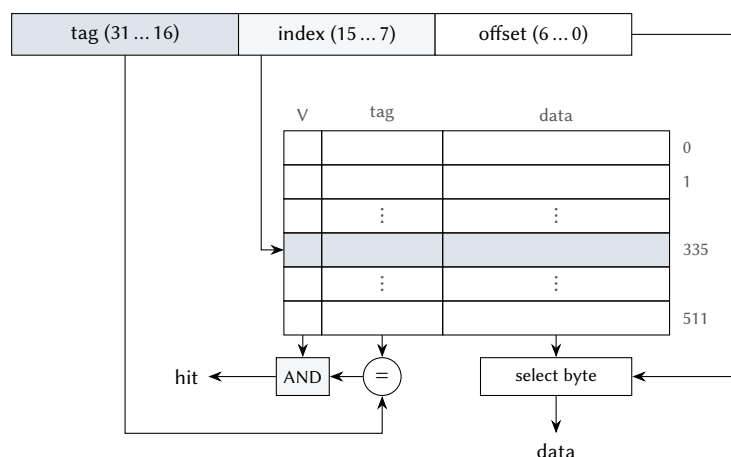


Figure 12.4: Lookup in a direct-mapped 64 KB cache with 128-byte blocks. The index selects one of 512 lines; a single comparator checks its tag.

12.7.1 Advantages and disadvantages

The strength of a direct-mapped cache is that only one line has to be examined. A single comparator suffices, so the lookup is fast, cheap and energy-efficient: the hit time is as low as it can be for a cache of a given size.

Its weakness is that blocks with the same index compete for a single line. If a program alternates between two such blocks, each access evicts the block the other access needs, and every access misses, even when the rest of the cache is empty. These conflicts raise the miss rate.

Example 12.16. An 8-line direct-mapped cache with 16-byte blocks uses offset bits 3–0, index bits 6–4 and tag bits 31–7. A program alternately accesses addresses 0x1000 (block 0x100) and 0x1084 (block 0x108). Both blocks have index 0, with tags 0x20 and 0x21. Each access replaces the block loaded by the previous one, so every access is a miss, however long the program continues.

12.8 Set-associative caches

A set-associative organization relaxes the one-line restriction.

Definition 12.17 (Set-associative cache). A *set-associative cache* divides its lines into *sets* of N lines each; a block can be placed in any line of one particular set. Such a cache is called N -way set-associative.

The index now selects a set rather than a line. A cache with 2^s sets has an s -bit index, and the tag again consists of all bits above the index. A lookup compares the tag of the address with the tags of all N lines of the selected set, in parallel, and hits if any valid line matches (Figure 12.5); the matching line supplies the data.

Example 12.18. Organize the 8 lines of Example 12.16 as a 2-way set-associative cache. It has 4 sets, so the index is bits 5–4 and the tag bits 31–6. Block 0x100 goes to set 0 with tag 0x40, and block 0x108 to set 0 with tag 0x42. The two blocks still share a set, but the set has two lines: after the first access to each block has missed and loaded it, every further access hits.

Compared with a direct-mapped cache of the same size, an N -way set-associative cache needs N comparators and a selection of the matching line's data, which makes the hit time longer, but it suffers fewer conflicts and so has a lower miss rate.

12.9 Fully associative caches

At the other extreme, a block may be placed anywhere.

Source Lesson 12, videos 29–31; H&P §B.1.

▷ セットアソシアティブキャッシュ

Source Lesson 12, videos 32–33; H&P §B.1.

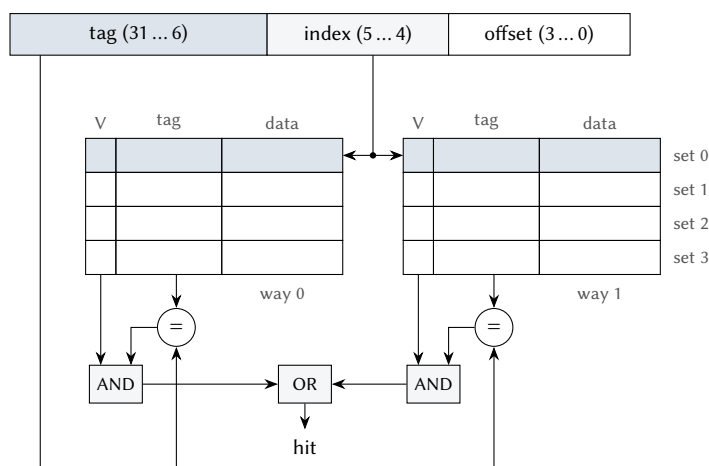


Figure 12.5: Lookup in a 2-way set-associative cache with four sets, the organization of Example 12.18. The index (here 0, as in that example) selects a set; each of its two lines hits if it is valid and its tag matches.

Definition 12.19 (Fully associative cache). In a *fully associative cache* any block can be placed in any line.

There is nothing to select, so there is no index: the tag is the entire block number, and a lookup compares it with the tag of every line in the cache. Two blocks never have to compete for a line while another line is free, but a comparator is needed for every line, which makes a large fully associative cache slow and expensive.

12.9.1 One scheme for all three

Direct-mapped and fully associative caches are the two extremes of set-associative caches. A direct-mapped cache is 1-way set-associative, with as many sets as lines; a fully associative cache has a single set containing all lines. For every organization, with addresses of a bits,

$$\begin{aligned} \text{offset bits} &= \log_2(\text{block size}), \\ \text{index bits} &= \log_2(\text{number of sets}) = \log_2(\text{lines}/N), \\ \text{tag bits} &= a - \text{index bits} - \text{offset bits}, \end{aligned} \quad (12.4)$$

where $N = 1$ for a direct-mapped cache (index bits = $\log_2(\text{lines})$) and $N = \text{lines}$ for a fully associative cache (0 index bits). For the 64 KB cache with 128-byte blocks of Example 12.15, the tag grows from 16 bits when direct-mapped to 25 bits when fully associative; Table 12.4 in Section 12.12 lists the intermediate cases.

Key point

Associativity trades hit time against miss rate. More ways per set means more tag comparisons on every access, but fewer conflicts, because a block is evicted only when every line of its set is occupied by another block.

12.10 Cache replacement

On a miss, the new block must be placed in one of the lines allowed for it. If one of those lines is not valid, that line is used. If all of them hold valid blocks, one block has to be evicted. A direct-mapped cache has no choice; in set-associative and fully associative caches the victim is chosen by the *replacement policy*.

Random evict a randomly chosen block of the set. Simple, but it ignores how the blocks are being used.

FIFO (first in, first out) evict the block that has been in the cache longest, regardless of whether it is still in use.

▷ フルアソシアティブキャッシュ

Tip A cache of C bytes has $a - \log_2(C/N)$ tag bits. Each doubling of N moves one bit from the index to the tag.

Source Lesson 12, videos 34–36; H&P §B.1.

▷ 置換アルゴリズム

LRU evict the block whose most recent access lies furthest in the past.

Definition 12.20 (Least recently used). The *least recently used* (LRU) policy evicts, among the blocks that may be replaced, the one that has not been accessed for the longest time.

LRU exploits temporal locality: a block accessed recently is likely to be accessed again soon, so the least recently used block is the one least likely to be needed.

12.10.1 Implementing LRU with counters

LRU can be implemented with a counter per line. In an N -way set-associative cache each line has a $\log_2 N$ -bit counter. At start-up the counters of each set are initialized to the values $0, 1, \dots, N - 1$, in any order, and the update rule below keeps it that way: every value occurs exactly once in each set. The line with counter 0 is the least recently used and the one with counter $N - 1$ the most recently used. On every access to a line whose counter is c :

1. every counter in the set that is greater than c is decremented;
2. the counter of the accessed line is set to $N - 1$.

On a miss, the victim is the line with counter 0; once the new block is in it, the access is treated like any other and the counters are updated as above. Table 12.1 traces one 4-way set.

Table 12.1: LRU counters of one 4-way set holding blocks A–D, shown as block:counter for each of its four lines.

Access	Result	Line 0	Line 1	Line 2	Line 3
(initial)		A:0	B:1	C:2	D:3
C	hit	A:0	B:1	C:3	D:2
A	hit	A:3	B:0	C:2	D:1
E	miss, B evicted	A:2	E:3	C:1	D:0
D	hit	A:1	E:2	C:0	D:3
B	miss, C evicted	A:0	E:1	B:3	D:2

The cost of this implementation is $\log_2 N$ bits per line and, more importantly, activity on every access: even a hit may change all N counters of the set. For highly associative caches this costs considerable hardware and energy, which motivates the cheaper approximations of LRU in Lesson 14.

12.11 Write policy

Writes raise two questions that reads do not.

1. *Is a block brought into the cache on a write miss?* A cache that brings it in follows the *write-allocate policy*; a no-write-allocate cache writes the value to main memory only. Most caches are write-allocate: by locality, a block that has just been written is likely to be read or written again soon.
2. *When does main memory receive the written value?* Here two policies exist.

Definition 12.21 (Write-through and write-back). Under the *write-through policy* a write updates the block in the cache and immediately also the block in main memory.

Under the *write-back policy* a write updates only the block in the cache; main memory is updated when the block is evicted from the cache.

Write-through keeps main memory always up to date, but every write then pays the access time of main memory, which is what the cache was meant to avoid. Caches therefore use write-back: many writes to a block while it is in the cache cost at most a single write to main memory, at eviction. A write-back cache is normally also write-allocate; otherwise every write to a block that is not in the cache would go to main memory, however often the block is written.

▷ LRU

Caution Update the counters on *every* access, hits included; updating them only on loads gives FIFO.

Source Lesson 12, videos 37–40; H&P §B.1.

▷ ライトアロケート方式

▷ ライトスルー方式

▷ ライトバック方式

Writing every evicted block back would, however, also write blocks that were only read and are identical to their copies in memory.

Definition 12.22 (Dirty bit). The *dirty bit* of a line indicates whether its block has been written since it was loaded. It is cleared when a block is loaded and set when the block is written. On eviction, a dirty block is written to main memory; a clean block is simply discarded.

Example 12.23. Blocks X and Y map to the same line of a direct-mapped cache, which is initially invalid. Table 12.2 follows a sequence of accesses under write-back. The two consecutive writes to Y cost a single memory write, at step 6, and evicting Y at step 8 costs none, because Y was not written after it was loaded at step 7. Write-back needs 3 memory writes in total; write-through would need one per write, i.e. 4.

Table 12.2: A single line under the write-back policy (Example 12.23). D is the dirty bit after the access.

Step	Access	Result	Memory operations	Line after
1	read X	miss	read X	X, D=0
2	write X	hit	—	X, D=1
3	read Y	miss	write X back, read Y	Y, D=0
4	write Y	hit	—	Y, D=1
5	write Y	hit	—	Y, D=1
6	write X	miss	write Y back, read X	X, D=1
7	read Y	miss	write X back, read Y	Y, D=0
8	read X	miss	read X	X, D=0

12.12 Putting it together

A cache is fully described by a few design choices, from which its structure follows.

Cache size and block size give the number of lines and the offset bits.

Associativity gives the number of sets, hence the index and tag bits (Equation (12.4)).

Replacement policy LRU adds a $\log_2 N$ -bit counter to every line (none for direct-mapped caches).

Write policy write-back adds a dirty bit to every line.

Every line thus stores a valid bit, a dirty bit, an LRU counter and a tag in addition to its data. Figure 12.6 shows how these are used on one access. A hit reads or writes the data and updates the LRU counters. A miss first chooses a victim, writes it back if it is valid and dirty, and loads the new block; the access then proceeds as a hit.

Example 12.24. Consider the 2-way set-associative cache of Example 12.18 (16-byte blocks, index bits 5–4, tag bits 31–6), now with LRU replacement and the write-back policy. Table 12.3 follows its set 1 through five accesses; both lines of the set are valid throughout. The first access hits and only updates the counters. The second misses, and its victim is way 1, whose counter is 0; that block is dirty and is written back before the new block is loaded. The fourth access is a write miss whose victim is clean, so nothing is written back, and the loaded block becomes dirty at once. The last access evicts the block with tag 0x40, which the third access has written, and writes it back.

Example 12.25. Table 12.4 works out the organization of a 64 KB cache with 128-byte blocks, 32-bit addresses, LRU replacement and write-back, for several associativities. For the 8-way cache, the address 0x0040A7C6 (block number 0x814F) has index $0x814F \& 0x3F = 15$ and tag $0x814F \gg 6 = 0x205$. Its 512 lines need $512 \times 24 = 12288$ bits of metadata, i.e. 1536 bytes, about 2.3 % of the 64 KB of data.

Lesson 13 applies the same ideas—blocks, tags and replacement—to the translation of virtual addresses, and Lesson 14 turns to techniques that lower the hit time, the

▷ ダーティビット

Source Lesson 12, videos 41–45; H&P §B.1.

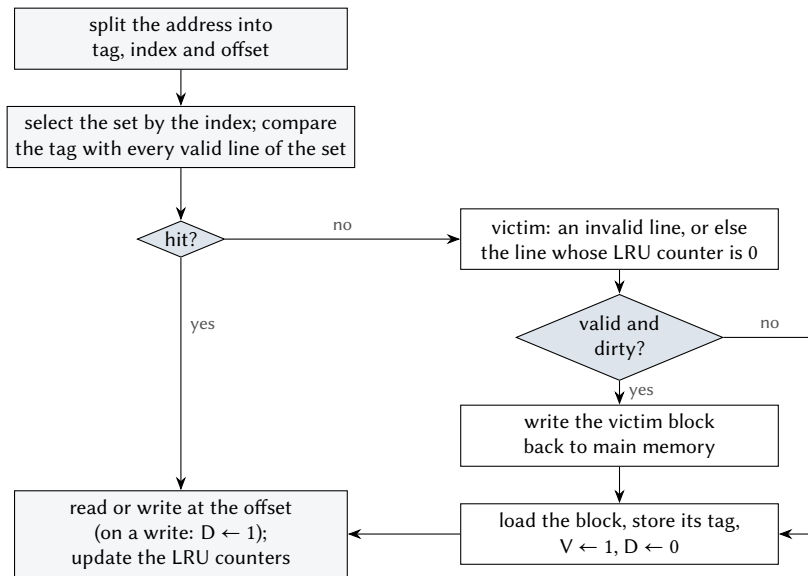


Figure 12.6: One memory access in a set-associative cache with LRU replacement and the write-back policy.

Table 12.3: Set 1 of a 2-way set-associative cache with LRU replacement and write-back (Example 12.24): tag, dirty bit and LRU counter of each way after each access. All tags, including those of the memory operations, are hexadecimal.

Access	Result	Way 0			Way 1			Memory operations
		Tag	D	LRU	Tag	D	LRU	
(initial)		40	0	0	43	1	1	
read 0x1018	hit	40	0	1	43	1	0	—
read 0x1054	miss	40	0	0	41	0	1	write back 43, load 41
write 0x101C	hit	40	1	1	41	0	0	—
write 0x1098	miss	40	1	0	42	1	1	load 42
read 0x1058	miss	41	0	1	42	1	0	write back 40, load 41

Table 12.4: Address fields and metadata bits per line (valid + dirty + LRU counter + tag) of a 64 KB cache with 128-byte blocks and 32-bit addresses.

Organization	Sets	Offset	Index	Tag	Metadata per line
Direct-mapped	512	7	9	16	$1 + 1 + 0 + 16 = 18$
2-way	256	7	8	17	$1 + 1 + 1 + 17 = 20$
8-way	64	7	6	19	$1 + 1 + 3 + 19 = 24$
Fully associative	1	7	0	25	$1 + 1 + 9 + 25 = 36$

miss rate and the miss penalty of the basic cache described here.

Key point – Lesson summary

Programs access memory with temporal and spatial locality, so a small, fast cache of recently used blocks serves most accesses. Its benefit is measured by $AMAT = \text{hit time} + \text{miss rate} \times \text{miss penalty}$ (Equation (12.1)). Memory is divided into power-of-two blocks placed into cache lines; an address splits into tag, index and offset, and a line hits when it is valid and its tag matches. Direct-mapped, set-associative and fully associative caches differ in how many lines a block may occupy, trading hit time against miss rate. On a miss LRU evicts the least recently used block, and the write-back policy with a dirty bit writes a block to memory only when a modified block is evicted.

Virtual Memory

Lecture	Lesson 13, videos 1–27
Textbook	H&P [1] §B.4–B.5
Keywords	virtual memory, physical memory, page, page frame, page table, page fault, virtual page number, page offset, multi-level page table, page size, internal fragmentation, TLB, TLB miss, page table walk

Lesson 12 treated every address produced by the processor as a location in main memory. In a real system the addresses that programs use are *virtual*: every program sees a large address space of its own, and the operating system and the processor together map the parts of it that the program actually uses onto the memory that is installed. This lesson explains why virtual memory is needed, how page tables translate virtual addresses into physical ones, how large these tables are and how multi-level page tables keep them small, and why translation would slow down every memory access if the processor did not cache recent translations in a translation lookaside buffer, which Lesson 14 combines with the caches of Lesson 12.

13.1 Why virtual memory

Hardware and software see memory differently. To the hardware, memory is what is installed in the machine: an array of as many bytes as the memory modules provide. To a program, memory is an address space of 2^{32} bytes on a 32-bit processor or 2^{64} bytes on a 64-bit processor, and all of it belongs to that program: code, data and stack are placed at whatever addresses the program and its compiler choose, regardless of how much memory is installed and of which other programs are running at the same time.

The two views disagree in two respects.

- The address space of a single program is much larger than the installed memory. A 32-bit address space is 4 GB, which a machine may or may not have; a 64-bit address space is 16 EB (1 EB = 2^{60} bytes), many orders of magnitude more than any machine has.
- Many programs run at the same time, and each of them uses the same addresses, as if it owned the entire address space.

Definition 13.1 (Virtual memory). The mechanism that gives every program the view of a large memory of its own, and maps the parts of that memory the program actually uses onto the memory installed in the machine, is called *virtual memory*.

The operating system decides where each part of each program is kept, and the processor applies these decisions to every address on every memory access. Programs can thus be written without knowing the size of memory or the other programs they will run with, a program cannot access the memory of another unless the operating system arranges it, and together the programs can use more memory than is installed, the excess being kept on disk (Section 13.4).

13.2 Physical memory and virtual address spaces

The two views correspond to two kinds of addresses.

Definition 13.2 (Physical memory). The memory actually installed in the machine is its *physical memory*. Its bytes are numbered consecutively, from 0 up to the size of physical memory minus one, and the number that identifies a byte is its *physical address*.

The size of physical memory is determined by the installed modules, not by the

Source Lesson 13, videos 1–3; H&P §B.4; [23].

▷ 仮想記憶

Source Lesson 13, videos 4–5; H&P §B.4.

▷ 物理メモリ

▷ 物理アドレス

width of the processor's addresses. A 32-bit machine can address 4 GB but may have only 2 GB installed; a 64-bit machine has perhaps tens of gigabytes, a tiny fraction of the 2^{64} bytes it could address. In the examples of this lesson a physical address has just as many bits as the installed memory requires: 31 bits for 2 GB.

Definition 13.3 (Virtual address space). The *virtual address space* of a program is the full range of addresses the program can use, from 0 to $2^n - 1$ on an n -bit processor, independently of physical memory. An address in it is a *virtual address*.

All addresses a program works with are virtual: the program counter, the pointers stored in its data, and the addresses the processor computes for loads and stores. A program uses only a few regions of its virtual address space (Figure 13.1, left and right):

- its *code* and its global *data* at low addresses;
- above them the *heap*, which grows toward higher addresses as the program allocates memory dynamically;
- the *stack* at the top of the address space, which grows toward lower addresses.

Between the heap and the stack lies a huge range of addresses that the program never uses. Every running program has its own virtual address space laid out in this way, so different programs use the same virtual addresses for entirely different contents.

13.3 Pages and page tables

A mapping recorded separately for every virtual address would need as many table entries as there are addresses. Virtual memory therefore maps fixed-size ranges of addresses, just as a cache moves blocks rather than individual bytes.

Definition 13.4 (Page, page frame). The virtual address space is divided into consecutive ranges of equal size, each called a *page*. Physical memory is divided into ranges of the same size, each called a *page frame*, or frame for short. The size is a power of two, typically 4 KB, and any page fits exactly into any frame.

Definition 13.5 (Page table). The *page table* of a program records, for every page of its virtual address space, whether the page is in physical memory and, if it is, which frame holds it.

The operating system maintains one page table for every running program and updates it whenever it places one of the program's pages into a frame or removes one. The page table of the running program is found through a register set by the operating system and is read to translate the program's addresses, usually by the processor itself and in some systems by the operating system (Section 13.10). Figure 13.1 illustrates the resulting mapping for two programs.

- Any page may be placed in any frame. Pages that are adjacent in a virtual address space need not be adjacent in physical memory, and the frames of different programs are interleaved.
- Pages that a program never uses are not mapped and occupy no frame; only the used regions of the address space consume physical memory.
- The page tables of two programs may map pages to the same frame. This lets programs share memory, for example a single copy of the code of a program that is running twice.
- Not every mapped page has to be in physical memory; the others are on disk (Section 13.4).

A page table describes *every* page of the virtual address space, used or not; in its simplest form (Section 13.6) it has an entry for each. The number of pages it describes is therefore set by the size of the virtual address space and the size of a page; the size of physical memory only determines how many frames an entry can refer to.

▷ 仮想アドレス空間

▷ 仮想アドレス

Source Lesson 13, videos 6–7; H&P §B.4.

▷ ページ

▷ ページフレーム

▷ ページテーブル

Caution A page table is indexed by virtual page, not by frame. With less physical memory there are fewer frames, but the page table must still describe just as many pages.

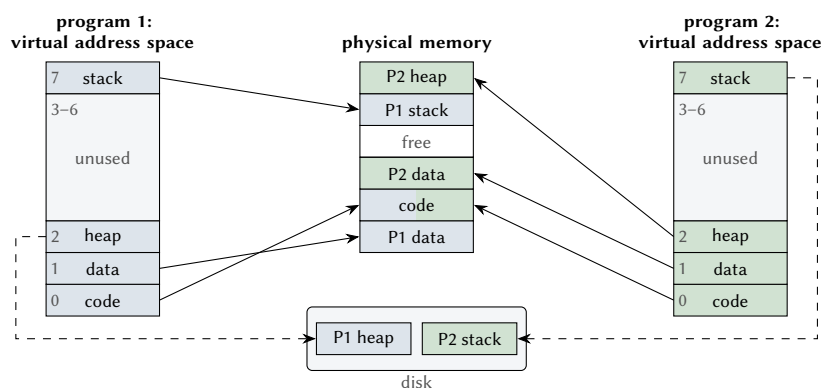


Figure 13.1: The virtual address spaces of two programs (pages 0–7, low addresses at the bottom) mapped onto six frames of physical memory. Page 0 of both programs is mapped to the same frame; each program has one page on disk (dashed); the unused pages 3–6 are not mapped.

13.4 Page faults

Together, the running programs usually map more pages than physical memory has frames. The pages that do not fit are kept on disk, and physical memory holds the pages that have been used recently.

Definition 13.6 (Page fault). An access to a page that the page table does not map to a frame causes a *page fault*: an exception that transfers control to the operating system.

If the page is on disk, the operating system handles the page fault in three steps.

1. It chooses a frame for the page. If no frame is free, it evicts a page that has not been used recently; if that page has been written since it was loaded, it is first written back to disk. The page table of the evicted page is updated to show that the page is no longer in memory.
2. It reads the faulting page from disk into the frame and updates the page table to map the page to this frame.
3. It restarts the instruction that caused the fault, which now finds its page in memory; this requires precise exceptions (Lesson 8).

A disk access takes milliseconds, i.e. millions of clock cycles, so a page fault is enormously more expensive than a cache miss. Page faults must be rare, and it pays to let the operating system choose the page to evict with a careful algorithm in software.

Not every page fault involves the disk: if the address lies outside every region that the program has been allocated, there is nothing to load, and the operating system treats the access as an error, usually by terminating the program.

Note 13.7. Physical memory acts as a cache of the pages on disk. Pages correspond to the blocks of Lesson 12, frames to lines, page faults to cache misses, and writing only modified pages back to disk to a write-back cache with a dirty bit. Placement is fully associative, since any page may be placed in any frame, yet no tags have to be searched, because the page table is indexed directly by the page.

13.5 Address translation

Because pages start at multiples of the page size, a virtual address splits into two fields, just like the block number and block offset of a cache address.

Definition 13.8 (Virtual page number, page offset). With pages of 2^k bytes, the low-order k bits of a virtual address form its *page offset*, the position of the byte within its page. The remaining high-order bits form its *virtual page number* (VPN), which identifies the page.

Source Lesson 13, video 8; H&P §B.4.

▷ ページフォールト

Source Lesson 13, videos 9–10; H&P §B.4.

▷ ページオフセット
▷ 仮想ページ番号

Frames are numbered in the same way: the frame that starts at physical address $j \times 2^k$ has the *physical frame number* (PFN) j . The page table maps the VPN to the PFN, and translation replaces the one by the other while keeping the offset (Figure 13.2):

$$\begin{aligned} \text{VPN} &= \lfloor \text{VA} / 2^k \rfloor, & \text{offset} &= \text{VA} \bmod 2^k, \\ \text{PFN} &= \text{page table}[\text{VPN}], & \text{PA} &= \text{PFN} \times 2^k + \text{offset}, \end{aligned} \quad (13.1)$$

where VA and PA are the virtual and physical addresses. The offset passes through unchanged because a page and its frame have the same size and the byte keeps its position within them; in binary, the physical address is the PFN bits followed by the offset bits. The VPN has k bits fewer than a virtual address and the PFN k bits fewer than a physical address, so the two numbers differ in width whenever virtual and physical addresses do. If the page table holds no PFN for the page, the access causes a page fault instead.

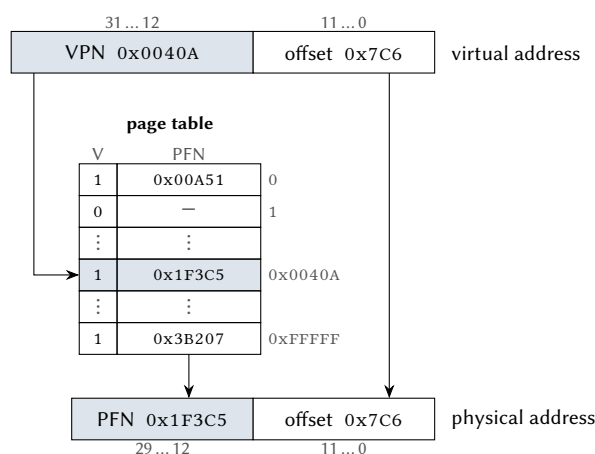


Figure 13.2: Translation of the virtual address 0x0040A7C6 with 4 KB pages and 30-bit physical addresses (Example 13.9). The VPN selects a page table entry; its PFN replaces the VPN, and the offset is copied. An entry with $V = 0$ would cause a page fault.

Example 13.9. A 32-bit processor with 4 KB pages ($k = 12$) has 1 GB of physical memory, hence 30-bit physical addresses. The page offset is bits 11–0, the VPN bits 31–12 (20 bits), and a PFN has 18 bits. The virtual address 0x0040A7C6 has VPN 0x0040A and offset 0x7C6. The page table of Figure 13.2 maps page 0x0040A to frame 0x1F3C5, so the physical address is $0x1F3C5 \times 2^{12} + 0x7C6 = 0x1F3C57C6$. The virtual address 0x000017FF lies in page 0x00001, which the same page table marks as not in memory: an access to it causes a page fault.

Key point

Translation replaces the virtual page number by a physical frame number and keeps the page offset. The page table, maintained by the operating system, supplies the frame number; if it holds none for the page, the access causes a page fault.

▷ 物理フレーム番号

Tip With 4 KB pages and addresses written in hexadecimal, the last three digits are the offset; translation replaces the digits in front of them.

13.6 The size of a flat page table

The simplest page table is a single array.

Definition 13.10 (Flat page table). A *flat page table* is an array with one entry for every virtual page, indexed directly by the VPN; its element for a page is the *page table entry* (PTE) of that page.

A PTE holds the PFN of its page and a few status bits: a valid bit that is set only if the page is in physical memory, and bits that record how the page may be accessed

Source Lesson 13, videos 11–12; H&P §B.4.

▷ 単一レベルページテーブル

▷ ページテーブルエントリ

(read, write, execute). A PTE is typically about as large as an address, 4 bytes on a 32-bit processor and 8 bytes on a 64-bit processor, which leaves room for the PFN and the status bits. Since every virtual page has an entry,

$$\text{size of a flat page table} = \frac{\text{size of the virtual address space}}{\text{page size}} \times \text{PTE size.} \quad (13.2)$$

Two quantities are absent from Equation (13.2). The size of physical memory only affects how many bits of an entry the PFN occupies. The amount of memory the program actually uses does not matter at all, because unused pages have entries too. Every running program needs a page table of this size.

Example 13.11. With 32-bit virtual addresses, 4 KB pages and 4-byte entries, a flat page table has $2^{32}/2^{12} = 2^{20}$ entries and occupies $2^{20} \times 4$ bytes = 4 MB for each running program, even for a program that uses only a few kilobytes. With 64-bit virtual addresses, 4 KB pages and 8-byte entries it would have 2^{52} entries and occupy 2^{55} bytes (32 PB), more than any memory or disk holds. Even if only 48 of the 64 address bits are used, the flat page table occupies $2^{36} \times 8 = 2^{39}$ bytes, i.e. 512 GB per program.

13.7 Multi-level page tables

A flat page table is large because it has entries for the whole virtual address space, although programs use only the bottom and the top of it. A multi-level page table avoids storing entries for the unused range in between.

Definition 13.12 (Multi-level page table). A *multi-level page table* is a tree of small tables. The VPN is split into one field per level. The first field indexes the outer page table, whose entries point to tables of the next level; the entries of the tables of the last level, the inner page tables, hold PFNs. An entry whose part of the address space is not used at all is marked invalid, and no tables exist below it.

With two levels, a virtual address is split into an outer index p_1 , an inner index p_2 and the page offset (Figure 13.3), and the translation takes two steps:

1. entry p_1 of the outer page table gives the location of an inner page table;
2. entry p_2 of that inner page table gives the PFN, which is combined with the offset as in Equation (13.1).

If either entry is invalid, no PFN is found and the access causes a page fault. The tables are usually sized so that each fits exactly into one page.

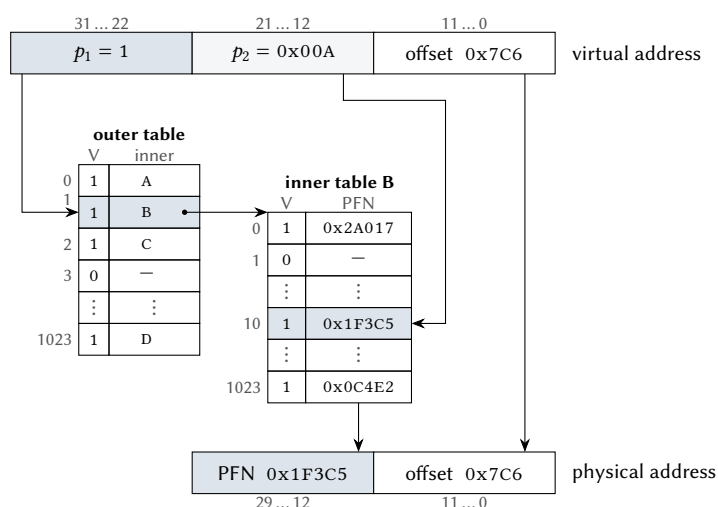


Figure 13.3: Two-level translation of the virtual address 0x0040A7C6 (Example 13.13). Outer entry 1 points to inner table B. Entries A, C and D point to inner tables that are not drawn; invalid outer entries, such as entry 3, have no inner table at all.

Source Lesson 13, videos 13–17; H&P §B.4–B.5.

▷ 多段ページテーブル

Example 13.13. The 20-bit VPN of Example 13.9 is split into a 10-bit outer index (bits 31–22) and a 10-bit inner index (bits 21–12). With 4-byte entries every table has 2^{10} entries and occupies 4 KB, exactly one page. The virtual address 0x0040A7C6 has VPN 0x0040A = 0000000001 0000001010₂, hence $p_1 = 1$ and $p_2 = 0x00A = 10$. Outer entry 1 points to inner table B, and entry 10 of B holds PFN 0x1F3C5. The physical address is 0x1F3C57C6, as with the flat page table: a multi-level page table stores the same mapping in a different form.

13.7.1 Size of a multi-level page table

The outer page table always exists, but an inner page table exists only for a part of the address space in which at least one page is used. With the organization of Example 13.13, each inner table covers 2^{10} pages, i.e. 4 MB of the virtual address space.

Example 13.14. A program uses 10 MB at the bottom of its 32-bit address space for code, data and heap (addresses 0x00000000–0x009FFFFF) and 1 MB at the top for its stack (0xFFFF0000–0xFFFFFFFF). With the two-level page table of Example 13.13, the bottom 10 MB lie in the ranges of outer entries 0, 1 and 2, of 4 MB each, and the stack in the range of outer entry 1023. The page table consists of the outer table and four inner tables, $5 \times 4 \text{ KB} = 20 \text{ KB}$, instead of the 4 MB of a flat page table.

A two-level page table is larger than a flat one only if every outer entry needs an inner table, i.e. if at least one page is used in each of the 1024 ranges of 4 MB. It then occupies $4 \text{ KB} + 1024 \times 4 \text{ KB} \approx 4.004 \text{ MB}$, more than the flat table by the size of the outer table. Real programs use a few contiguous regions of their address space and need only a few inner tables.

13.7.2 More levels for 64-bit addresses

Two levels do not suffice for 64-bit address spaces. A 4 KB table of 8-byte entries has 2^9 entries, so each level of tables that fit into one page can consume only 9 bits of the VPN. With 48-bit virtual addresses the VPN has 36 bits. A two-level page table with 9-bit inner tables would need a 27-bit outer index and an outer table of $2^{27} \times 8 \text{ bytes} = 1 \text{ GB}$ for every program. Splitting the 36 bits into four 9-bit fields instead yields a four-level page table in which every table fits into one page; x86-64 processors use this organization. Each further level makes room for 9 more bits of virtual address.

Example 13.15. In such a four-level page table, a table of the last level covers 2^9 pages, i.e. 2 MB of the address space; a table of the third level covers 2^9 times as much, 1 GB; a table of the second level 512 GB; and the single table of the first level all 2^{48} bytes (256 TB). A program that uses 8 MB at the bottom and 8 MB at the top of its address space needs, for each end, four tables of the last level and one of each of levels 3 and 2, and all of them hang below the single first-level table: $1 + 2 \times (4 + 1 + 1) = 13$ tables, or $13 \times 4 \text{ KB} = 52 \text{ KB}$, whereas its flat page table would occupy 512 GB (Example 13.11).

Key point

A page table must describe the entire virtual address space, but a multi-level page table needs storage only for the parts that are used. The price is one table read per level on every translation.

13.8 Choosing the page size

How large pages should be is a compromise between two effects.

- **Larger pages** mean fewer pages: fewer PTEs and a smaller page table, and, as Section 13.10 shows, more memory covered by each translation that the processor caches.
- **Smaller pages** waste less memory in partly used pages.

The *page size*, the number of bytes in a page and in a frame, must balance the two.

Source Lesson 13, video 18; H&P §B.4.

▷ ページサイズ

Definition 13.16 (Internal fragmentation). Memory is mapped a whole page at a time. A region whose size is not a multiple of the page size therefore leaves part of its last page unused, and no other program can use that part. The memory lost in this way is *internal fragmentation*.

A region of arbitrary size leaves on average half a page unused, so internal fragmentation grows in proportion to the page size.

Example 13.17. A program on a 32-bit processor with 4-byte PTEs has three regions (code and data, heap, stack), each ending in a partly used page. Table 13.1 compares three page sizes. Going from 4 KB to 4 MB pages shrinks the flat page table by a factor of 2^{10} , from 4 MB to 4 KB, but raises the expected internal fragmentation by the same factor, from 6 KB to 6 MB.

Table 13.1: Page table size and expected internal fragmentation for the program of Example 13.17 (half a page per region).

Page size	Pages	Flat page table	Internal fragmentation
4 KB	2^{20}	4 MB	$3 \times 2 \text{ KB} = 6 \text{ KB}$
64 KB	2^{16}	256 KB	$3 \times 32 \text{ KB} = 96 \text{ KB}$
4 MB	2^{10}	4 KB	$3 \times 2 \text{ MB} = 6 \text{ MB}$

Pages are much larger than cache blocks (Lesson 12), because every page needs its own PTE and every page brought in from disk costs a disk access. The usual page size is 4 KB. Processors also support much larger pages, such as 2 MB and 1 GB on x86-64, which the operating system can use for large regions that a program fills completely, so that internal fragmentation hardly matters.

13.9 Memory access time with translation

Every memory access a program makes, whether it fetches an instruction, loads or stores, uses a virtual address, which must be translated before physical memory can be accessed. The page table is itself kept in memory, so translation consists of memory reads. With a two-level page table, the processor performs a load such as `LW R1, 4(R2)` in four steps.

1. It adds 4 to R2 to obtain the virtual address and takes its VPN.
2. It computes the physical address of the outer PTE (the outer table address plus p_1 times the PTE size) and reads the PTE.
3. It computes the address of the inner PTE in the same way from the table address in the outer PTE and p_2 , and reads the PTE.
4. It appends the offset to the PFN and accesses the data at this physical address through the cache.

Each PTE read is an ordinary memory access that goes through the caches and may hit or miss; if PTEs are not kept in the cache, each costs a full main-memory access. With a page table of L levels, every access of the program turns into $L + 1$ memory accesses. If PTE reads have the same average memory access time (AMAT) as data accesses,

$$\text{access time with translation} \approx (L + 1) \times \text{AMAT}. \quad (13.3)$$

Example 13.18. A cache has a hit time of 2 cycles, a miss rate of 5 % and a miss penalty of 40 cycles, so $\text{AMAT} = 2 + 0.05 \times 40 = 4$ cycles (Equation (12.1)). Without translation a memory access takes 4 cycles on average; with a flat page table it takes $2 \times 4 = 8$ cycles, and with a four-level page table $5 \times 4 = 20$ cycles. Individual accesses vary widely: a load whose three upper-level PTEs hit (3×2 cycles), whose last-level PTE misses ($2 + 40$ cycles) and whose data hits (2 cycles) takes 50 cycles, 25 times as long as the cache hit it would be without translation.

The few upper-level entries are read by every translation and are usually cached; the numerous last-level entries miss more often. Either way, without a faster way to

▷ 内部断片化

Source Lesson 13, videos 19–21; H&P §B.4.

translate, virtual memory would slow down every instruction fetch, load and store by a large factor.

13.10 The translation lookaside buffer

A program that accesses a page is likely to access the same page again soon, so the translations it needs at any time are few and are used over and over. The processor therefore caches them.

Definition 13.19 (Translation lookaside buffer). A *translation lookaside buffer* (TLB) is a small, fast cache of recent translations. Each entry holds a valid bit, a tag taken from the VPN (the whole VPN in a fully associative TLB), the PFN to which that page is mapped, and the access bits of its PTE. If the VPN of an address matches a valid entry, the PFN is taken from the entry and no page table is read.

A TLB is a cache in the sense of Lesson 12, but it can be much smaller than the data cache, for three reasons.

- It stores only translations, not data; an entry takes a few bytes.
 - One entry serves a whole page rather than a single cache block, so a few entries cover the memory a program is actively using.
 - It stores the final result of the translation, not the entries of the individual levels.
- A TLB hit takes a single lookup, however many levels the page table has.

Example 13.20. A 128 KB data cache with 64-byte blocks has 2048 lines. With 4 KB pages, $128/4 = 32$ TLB entries cover as much memory as the cache holds, and 2048 are needed if every cached block lies in a different page; spatial locality keeps the actual need far below this upper bound.

13.10.1 TLB misses

A *TLB miss* occurs when no valid TLB entry matches the VPN of an address. The translation must then be obtained from the page table.

Definition 13.21 (Page table walk). A *page table walk* reads the page table entries of an address level by level, from the outer page table down to the entry that holds the PFN, as described in Section 13.7.

After a TLB miss, the walk finds the PFN, the translation is written into the TLB, replacing an existing entry if the TLB is full, and the access proceeds (Figure 13.4).

The TLB must agree with the page tables: when the operating system changes a mapping, e.g. by evicting a page, it invalidates the TLB entry of the page, and when the processor switches to another program, the entries of the previous program are invalidated or tagged with a program identifier.

The walk can be performed in two ways.

In software the TLB miss raises an exception, and a handler in the operating system walks the page table and writes the translation into the TLB. The hardware is simple and the page tables may be organized in any way, but every TLB miss pays for an exception. MIPS processors handle TLB misses this way.

In hardware the processor contains a page table walker, logic that reads the PTEs and fills the TLB itself without involving the operating system. TLB misses are handled much faster, but the page table must have exactly the format that the hardware expects. High-performance processors, such as those of the x86 family, walk the page table in hardware.

Key point

Every memory access needs a translation. The TLB makes the common case, a TLB hit, a single fast lookup; a page table walk, in hardware or by the operating system, is needed only on a TLB miss, and the page-fault handler only when the walk finds no PFN.

Source Lesson 13, videos 22–24; H&P §B.4.

▷ TLB

▷ TLB ミス

▷ ページテーブルウォーク

Caution A TLB miss is not a page fault: the walk usually finds a valid PTE.

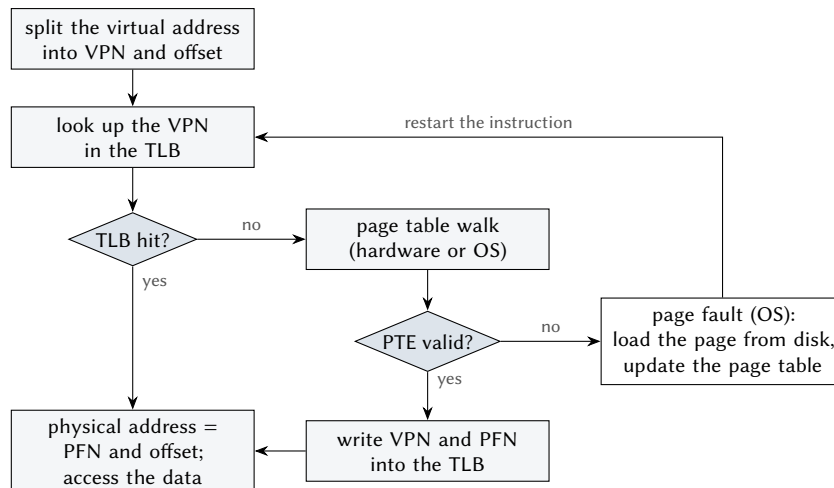


Figure 13.4: Translation on a memory access. A TLB hit supplies the PFN at once; a TLB miss requires a page table walk; an invalid PTE causes a page fault; once the page is in memory, the instruction is restarted.

13.11 TLB organization and performance

The organization of a TLB follows from its role.

Associativity Every TLB miss costs a walk of several memory reads, so TLBs are fully associative or highly set-associative; for so few entries the comparators are affordable, and translations do not compete for entries.

Size The TLB should hit about as often as the cache, so it must cover the pages of the data the cache holds (Example 13.20); this takes about 64 to 512 entries. Consulted on every access, it must also be about as fast as an L1 cache hit, which keeps it from growing larger.

Levels If that does not cover enough memory, a larger and slower second-level TLB with hundreds to thousands of entries is placed behind the first, as caches are organized in levels (Lesson 14). It is looked up only on a miss in the first, and a walk is needed only if both TLBs miss.

Example 13.22. The processor of Example 13.18 (AMAT of 4 cycles for every memory read, four-level page table) is given a TLB that is looked up in 1 cycle and misses on 1 % of the accesses. Every access pays for the lookup and for the data access, and one in a hundred also for a walk of $4 \times 4 = 16$ cycles, so an access takes on average $1 + 0.01 \times 16 + 4 = 5.16$ cycles instead of 20. Translation now costs 1.16 cycles instead of 16, mostly for the lookup, which Lesson 14 overlaps with the cache lookup.

Key point — Lesson summary

Virtual memory maps the pages of each program's virtual address space onto frames of physical memory and keeps the rest on disk; an access to a page without a frame is a page fault. Translation replaces the virtual page number by a frame number from the page table and keeps the offset (Equation (13.1)). Multi-level page tables avoid the entries a flat table spends on unused address space (Equation (13.2)); the page size trades table size against internal fragmentation. Every level adds a memory read to each access, so a small, highly associative TLB caches translations, and the page table is walked only on a TLB miss.

Source Lesson 13, videos 25–26; H&P §B.4.

Advanced Caches

Lecture	Lesson 14, videos 1–40
Textbook	H&P [1] §2.3, §B.1, §B.3
Keywords	pipelined cache, VIPT cache, aliasing, way prediction, NMRU, pseudo-LRU, conflict miss, prefetching, loop interchange, miss-under-miss support, MSHR, cache hierarchy, global miss rate, inclusion property

Lesson 12 described the basic cache and measured it by the average memory access time, and Lesson 13 added the address translation that every memory access needs in a system with virtual memory. This lesson collects the techniques that make real caches fast. They are organized by the term of the average memory access time that they reduce: the hit time (pipelined caches, the placement of the cache relative to the TLB, way prediction and cheap replacement policies), the miss rate (larger blocks, prefetching and loop interchange) and the miss penalty (servicing overlapping misses and, above all, multilevel cache hierarchies).

14.1 Improving cache performance

The quantity to be improved is the average memory access time (AMAT) of Lesson 12:

$$\text{AMAT} = \text{hit time} + \text{miss rate} \times \text{miss penalty}. \quad (14.1)$$

Each of the three terms offers its own line of attack, and Table 14.1 lists the techniques of this lesson by the term they target. Few of them are free. A technique that lowers one term often raises another—a smaller cache has a shorter hit time but a higher miss rate—and it is worthwhile only if AMAT as a whole decreases.

Table 14.1: Techniques of this lesson, grouped by the term of Equation (14.1) they reduce.

Term	Techniques	Sections
Hit time	pipelined caches, VIPT caches, way prediction, NMRU and PLRU replacement	14.2–14.6
Miss rate	larger blocks, prefetching, loop interchange	14.7–14.10
Miss penalty	miss-under-miss support, cache hierarchies	14.11–14.14

14.2 Pipelined caches

The most direct way to lower the hit time is to keep the cache small and simple, for example direct-mapped: fewer lines have to be decoded and fewer tags compared. A small cache, however, misses more often, and a direct-mapped cache suffers more conflicts, so the time saved on hits is easily lost again on misses. The techniques of this and the next four sections reduce the hit time, or its cost, without giving up the miss rate.

Lesson 12 noted that an L1 hit takes one to three clock cycles. If a lookup that needs several cycles occupied the cache until it completed, each memory access would have to wait for the previous one: the hit time seen by an access would be its own lookup time plus this wait, and consecutive loads and stores would stall the processor. A cache whose lookup fits into one cycle has no such problem. An L1 cache that takes two or three cycles is usually pipelined.

Definition 14.1 (Pipelined cache). A *pipelined cache* divides a lookup into stages separated by latches, so that a lookup takes several clock cycles but a new lookup can start in every cycle.

Source Lesson 14, videos 1–2; H&P §2.3.

Source Lesson 14, videos 3–4; H&P §2.3.

▷ パイプライン化キャッシュ

For a set-associative cache the lookup divides naturally into three steps, each of which can become a stage (Figure 14.1):

1. the index selects the set, and the valid bits and tags of its lines are read;
2. the tags are compared with the tag of the address, which decides between hit and miss and identifies the matching way, and the read of the data begins;
3. the data read completes, the data of the matching way is selected, and the block offset picks the requested bytes out of the block, which are delivered.

A miss is known at the end of the second stage. Pipelining does not reduce the number of cycles one lookup takes. It prevents that number from limiting the rate of accesses: while one access compares its tags, the next one already reads its set, exactly as instructions overlap in the pipeline of Lesson 3, and the clock period needs to accommodate only the slowest stage instead of the whole lookup.

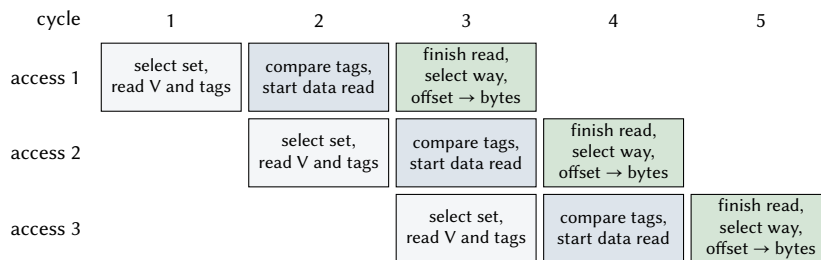


Figure 14.1: Three consecutive accesses to a cache whose lookup is split into three stages. Each access takes three cycles, but a new one starts in every cycle.

Example 14.2. The lookup logic of an L1 cache has a delay of 0.75 ns, and the other stages of the processor allow a clock period of 0.3 ns. Without pipelining, either the clock period grows to 0.75 ns or more for the whole processor, or each access occupies the cache for three cycles and memory accesses can start only in every third cycle. Split into three stages of about 0.25 ns each, which leaves room for the latch overhead, the lookup fits the 0.3 ns clock: the data arrives three cycles after the access starts, and an access can start in every cycle.

14.3 Address translation and the hit time

With virtual memory (Lesson 13) the processor issues virtual addresses, whereas data in main memory is located by physical addresses. The cache can be looked up after the translation, before it, or partly in parallel with it (Figure 14.2), and the choice affects the hit time.

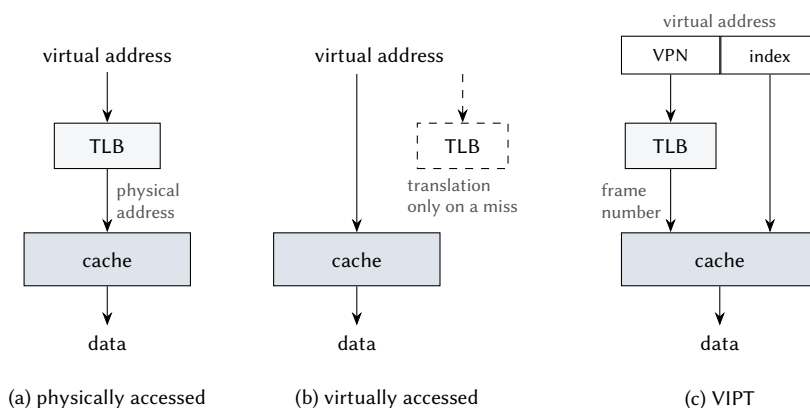


Figure 14.2: Three ways to combine the TLB with the cache lookup. In (b) translation is needed only on a miss, so the TLB is not in the path of a hit (it is still consulted for the permissions); in (c) it translates the virtual page number while the index already selects the set.

Source Lesson 14, videos 5–7; H&P §B.3.

14.3.1 Physically accessed caches

A cache that is looked up with the physical address is a *physically accessed cache*. Every access first translates its virtual address in the TLB, and only the resulting physical address is used for the lookup (Figure 14.2a). The hit time therefore includes the translation:

$$\text{hit time} = \text{TLB lookup time} + \text{cache lookup time.} \quad (14.2)$$

The TLB is small and fast, but its delay is paid on every access, hits included, and it adds to exactly the quantity we are trying to reduce.

14.3.2 Virtually accessed caches

Definition 14.3 (Virtually accessed cache). A *virtually accessed cache* takes both the index and the tag from the virtual address. The address is translated only on a miss, when the block has to be fetched from main memory.

On a hit the TLB is not in the path, and the hit time is just the cache lookup time (Figure 14.2b). This advantage comes with problems.

- **Permissions.** The TLB entry of a page also records whether the page may be read, written or executed, and the permission has to be checked on every access. In practice the TLB is therefore consulted on hits as well.
- **Context switches.** Each process has its own virtual address space, so the same virtual address denotes different data in different processes. When the operating system switches the processor from one process to another (a context switch), the blocks in the cache are identified by virtual addresses of the old process and must not be found by the new one. Either the cache is flushed by clearing all valid bits, or every tag is extended with an identifier of the address space (H&P §B.3). Context switches occur many times per second, and with flushing the process that runs next starts each time with an empty cache and a burst of misses.
- **Aliasing**, the subject of Section 14.4.

14.3.3 Virtually indexed, physically tagged caches

A combination of the two approaches keeps the short hit time and avoids the flushes.

Definition 14.4 (VIPT cache). A *virtually indexed physically tagged cache* (VIPT cache) takes the index from the virtual address and the tag from the physical address.

The index bits of the virtual address select the set at once (Figure 14.3). While the valid bits and tags of the set are being read, the TLB translates the virtual page number into the physical frame number, and the tags of the set, which are physical, are compared with the frame number from the TLB.

- The hit time is essentially the cache lookup time. The TLB lookup runs in parallel with the selection of the set and only has to deliver the frame number by the time the tags are compared, which a small TLB usually manages.
- The cache need not be flushed on a context switch. A tag names the physical memory its block came from, and that remains correct whichever process is running.
- The TLB is consulted on every access anyway, so the permissions are checked at no extra cost.

14.4 Aliasing

Virtual memory allows the same physical memory to appear at more than one virtual address.

Definition 14.5 (Aliasing). The situation in which two or more virtual addresses, in the same or in different address spaces, are mapped to the same physical address is

▷ 物理アドレスキャッシュ

▷ 仮想アドレスキャッシュ

▷ 仮想インデックス物理タグキャッシュ

Source Lesson 14, videos 8–11; H&P §B.3.

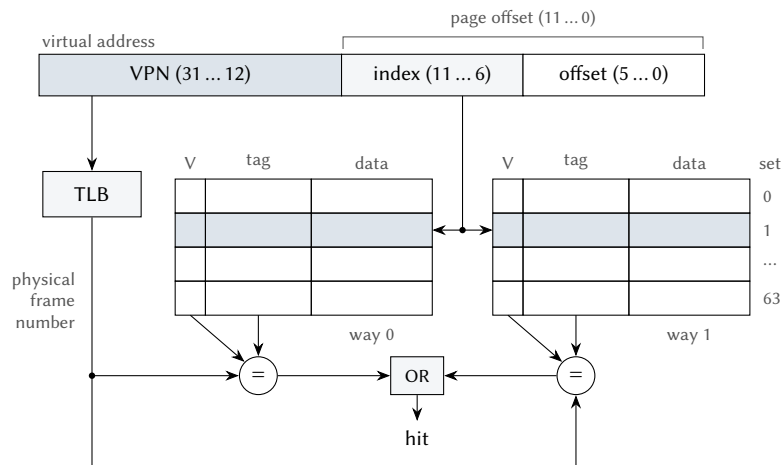


Figure 14.3: Lookup in a 2-way VIPT cache with 4 KB pages, 64-byte blocks and 64 sets (8 KB). The index comes from the page offset of the virtual address; the tags are physical frame numbers and are compared with the output of the TLB. A line hits if its valid bit is set and its tag matches.

called *aliasing*. The virtual addresses are then aliases of each other.

Aliasing is common: processes that share memory see the same frames, usually at different virtual addresses, and a single process can have one frame mapped at two addresses. In a virtually accessed cache two aliases are simply two different addresses. They generally have different indexes and tags, so the cache can hold two copies of the same physical block, one per alias, and nothing connects the copies.

Example 14.6. The virtual addresses V1 and V2 are aliases of the physical address P, and in a virtually accessed cache they have different indexes. The memory at P contains the value 5. Table 14.2 follows four accesses. The last one reads 5 through V2, although 7 has been written to the same memory through V1. With the write-through policy main memory would hold 7, but the stale copy for V2 would still be returned.

Table 14.2: Two aliases in a virtually accessed cache with the write-back policy (Example 14.6): contents of the two cached copies and of main memory after each access.

Step	Access	Result	Copy for V1	Copy for V2	Memory at P
1	read V1	miss	5	—	5
2	read V2	miss	5	5	5
3	write 7 to V1	hit	7	5	5
4	read V2	hit, returns 5	7	5	5

To be correct, every write would have to find all other cached copies of the same physical block and update or invalidate them, and the cache has no efficient way of locating them. Aliasing is thus a further problem of virtually accessed caches.

14.4.1 Aliasing in VIPT caches

In a VIPT cache the tag is physical, so two copies of a block would carry the same tag. They can nevertheless end up in different sets, because the index is taken from the virtual address. Whether that is possible depends on where the index bits lie. Translation replaces the virtual page number by the frame number and leaves the page offset unchanged; aliases therefore always have the same page offset, although their virtual page numbers differ.

- If the index includes bits of the virtual page number, two aliases can select different sets, the cache can hold two copies, and the problem of Example 14.6 remains.
- If the index consists of page-offset bits only, aliases always select the same set.

▷ エイリアシング

There they find a single line with the physical tag of their block, so at most one copy exists and aliasing is harmless. The index bits of the virtual address are then also those of the physical address, and the cache behaves exactly like a physically accessed cache, only faster.

14.4.2 Avoiding aliasing

A VIPT cache is thus free of aliasing when its block offset and index fit into the page offset:

$$\text{offset bits} + \text{index bits} \leq \text{page offset bits.} \quad (14.3)$$

The left side is $\log_2(\text{sets} \times \text{block size})$ and the right side $\log_2(\text{page size})$, so the condition reads $\text{sets} \times \text{block size} \leq \text{page size}$. Multiplying both sides by the associativity N gives a bound on the cache size:

$$\text{cache size} \leq N \times \text{page size.} \quad (14.4)$$

The block size has cancelled out: only the associativity and the page size limit a VIPT cache.

Example 14.7. With 4 KB pages the page offset has 12 bits. A VIPT cache with 64-byte blocks (6 offset bits) can use at most 6 index bits, i.e. 64 sets. With 2 ways this is the 8 KB cache of Figure 14.3; with 4 ways it is 16 KB. A 2-way cache of 16 KB would need 128 sets, and its seventh index bit would be bit 12 of the virtual address, the lowest bit of the virtual page number, so aliases could be placed in different sets.

14.4.3 Real VIPT caches

The page size is fixed by the architecture and the operating system, and is typically 4 KB. By Equation (14.4), an L1 cache that is to be both VIPT and free of aliasing can therefore grow only by becoming more associative: a direct-mapped cache is limited to 4 KB, a 32 KB cache needs 8 ways. The Intel Core 2, Nehalem, Sandy Bridge and Haswell processors all have 8-way L1 data caches of 32 KB, exactly at this bound. Real L1 caches are consequently more associative than their hit time alone would justify, which makes the techniques of the next two sections important.

Key point

A VIPT cache indexes with the virtual address while the TLB translates in parallel, and compares physical tags. It has nearly the hit time of a virtually accessed cache, needs no flush on a context switch, and is immune to aliasing as long as $\text{cache size} \leq N \times \text{page size}$.

14.5 Way prediction

Associativity moves the hit time and the miss rate in opposite directions. More ways per set mean fewer conflicts and a lower miss rate—and for a VIPT cache a larger permissible size (Equation (14.4))—but a lookup has to compare more tags and select among more lines, so the hit time grows. A direct-mapped cache has the shortest hit time and the highest miss rate. The goal is the miss rate of a set-associative cache with the hit time of a direct-mapped one.

Definition 14.8 (Way prediction). A set-associative cache with *way prediction* guesses which line of the selected set holds the requested block and first checks only that line. If its tag matches, the access completes in the hit time of a direct-mapped cache; otherwise the other lines of the set are checked as in an ordinary set-associative cache.

The first check of an N -way cache of size C examines one line per set, so it behaves like a direct-mapped cache of size C/N with the same sets, and the hit rate of such a cache estimates how often the guess is right. If the guess is the line accessed most

Source Lesson 14, videos 12–15; H&P §2.3.

▷ ウェイ予測

recently in the set, the estimate is exact: that line holds the block accessed most recently in the set, which is precisely the block a direct-mapped cache of size C/N would hold. A wrong guess costs the quick check plus the normal lookup, and a miss is detected only after the normal lookup as well. If p is the fraction of accesses in which the predicted line hits, t_1 the time to check one line and t_N the lookup time of the whole set, the average lookup time is

$$t = pt_1 + (1 - p)(t_1 + t_N) = t_1 + (1 - p)t_N. \quad (14.5)$$

Example 14.9. An L1 cache is followed by an L2 cache that gives it a miss penalty of 20 ns. Three L1 designs are compared. The 4 KB direct-mapped cache is the subset of the 8-way cache that way prediction checks first, so the predicted line hits in $p = 80\%$ of all accesses, and the remaining 20% (hits in other lines and all misses) take $0.8 + 1.5 = 2.3$ ns.

L1 cache	Lookup time	Miss rate	AMAT
4 KB direct-mapped	0.8 ns	20 %	$0.8 + 0.20 \times 20 = 4.8$ ns
32 KB 8-way	1.5 ns	5 %	$1.5 + 0.05 \times 20 = 2.5$ ns
32 KB 8-way, predicted	0.8 or 2.3 ns	5 %	$1.1 + 0.05 \times 20 = 2.1$ ns

By Equation (14.5) the average lookup time with prediction is $0.8 + 0.2 \times 1.5 = 1.1$ ns. Way prediction keeps the miss rate of the 8-way cache and brings its average lookup time close to that of the direct-mapped cache.

14.6 Replacement policy and hit time

The replacement policy affects the hit time as well as the miss rate. Two of the policies of Lesson 12 mark the extremes.

Random replacement does nothing on a hit and picks a victim at random on a miss. It adds nothing to the hit time, but it sometimes evicts a block that is about to be used, so its miss rate is higher.

LRU has a lower miss rate, but it keeps a $\log_2 N$ -bit counter in every line, and an access, hit or miss, may change the counters of all N lines of the set. This activity on every access costs logic, energy and hit time, the more so the higher the associativity.

The policies below approach the miss rate of LRU with far less bookkeeping.

14.6.1 Not most recently used

Definition 14.10 (NMRU). The *not most recently used* (NMRU) policy records for each set only which line was accessed most recently, and on a miss evicts one of the other lines, chosen at random.

By temporal locality, the most recently used block is the one most likely to be accessed next. Evicting it is the most harmful choice random replacement can make, and NMRU excludes exactly that choice. The cost is one $\log_2 N$ -bit pointer per set, and an access changes at most this pointer: an 8-way cache needs 3 bits per set instead of the $8 \times 3 = 24$ bits of LRU. NMRU misses somewhat more often than LRU, because its victim may be a block that was used only a little less recently than the most recent one.

14.6.2 Pseudo-LRU

Definition 14.11 (PLRU). The *pseudo-LRU* (PLRU) policy keeps one bit per line, initially 0. An access sets the bit of its line; if this sets the last bit of the set that was still 0, the bits of all other lines are cleared. A miss evicts a line whose bit is 0, chosen at random.

The bit of a line tells whether the line has been accessed since the last reset. The bit of the most recently used line is always set, so the candidates of PLRU are

Source Lesson 14, videos 16–19; H&P §B.1.

▷ NMRU

▷ 疑似 LRU

always among those of NMRU, and PLRU is at least as good as NMRU. How closely it approximates LRU depends on how many bits are set. Right after a reset only the most recently used line is excluded, as in NMRU. With $N - 1$ bits set, the only candidate is the line whose last access lies furthest in the past, which is exactly the choice of LRU. The cost is N bits per set, and an access sets a single bit, except at a reset.

Example 14.12. A 4-way set holds the blocks A, B, C and D in lines 0–3; D was used most recently, and the PLRU bits were last reset by the access to D. Table 14.3 follows the set under the three policies. At the miss on E, LRU must evict A, NMRU may evict A, C or D, and PLRU may evict A or C; here all three evict A. If the next access, to F, also misses, LRU evicts C, NMRU may evict B, C or D, and PLRU, whose only line with bit 0 is line 2, also evicts C.

Table 14.3: State kept by LRU, NMRU and PLRU for one 4-way set (Example 14.12): the LRU order from least to most recently used block, the most recently used line recorded by NMRU, and the PLRU bits of lines 0–3.

Access	Result	Lines 0–3	LRU order	NMRU	PLRU bits
(initial)		A B C D	A B C D	3	0 0 0 1
B	hit	A B C D	A C D B	1	0 1 0 1
D	hit	A B C D	A C B D	3	0 1 0 1
B	hit	A B C D	A C D B	1	0 1 0 1
E	miss	E B C D	C D B E	0	1 1 0 1

Key point

Work done on every access lengthens every hit. Way prediction checks one line before the others, and NMRU and PLRU replace the counters of LRU by one pointer per set (NMRU) or one bit per line (PLRU); each keeps most of the miss-rate benefit of associativity or LRU at a fraction of the hit-time cost.

14.7 Causes of misses

Reducing the miss rate starts with knowing why misses occur. Misses are classified into three kinds, often called the three Cs.

Definition 14.13 (Compulsory, capacity and conflict misses). A *compulsory miss* is a miss on the first access to a block; it occurs in any cache, even an infinitely large one.

A *capacity miss* is a miss that would occur even in a fully associative cache of the same size: the cache cannot hold all the blocks the program is using, and the block has been evicted for lack of space.

A *conflict miss* is a miss that occurs only because of limited associativity: a fully associative cache of the same size would still hold the block, but in the actual cache it was evicted by another block competing for the same set.

The three counts can be measured by simulating the same sequence of accesses on three caches. The misses of an infinite cache are the compulsory misses; a fully associative cache of the actual size adds the capacity misses; the actual cache adds the conflict misses.

Example 14.14. For one program, an infinite cache misses 1500 times, a fully associative 32 KB cache 6000 times, and the actual 4-way 32 KB cache 7800 times. Of the 7800 misses, 1500 are compulsory, $6000 - 1500 = 4500$ are capacity misses and $7800 - 6000 = 1800$ are conflict misses.

The classification points to the remedies. A larger cache reduces capacity misses, and a higher associativity or a better replacement policy reduces conflict misses, but all of these lengthen the hit time. The following sections present techniques that

Source Lesson 14, video 20; H&P §B.3; [24].

▷ 初期参照ミス

▷ 容量性ミス

▷ 競合性ミス

reduce the miss rate without lengthening hits.

14.8 Larger cache blocks

For a cache of a given size, a larger block size affects the miss rate in two opposite ways.

- Where spatial locality is good, the data around an accessed address is used soon. A larger block brings more of it into the cache with a single miss, and later accesses to that data, which would otherwise have missed, now hit.
- Where spatial locality is poor, most of a larger block is never used. The cache holds fewer blocks, hence fewer of the blocks that are actually needed, and capacity and conflict misses increase.

As the block size grows, the miss rate therefore first falls and then rises again (Figure 14.4). The minimum lies at a larger block size for a larger cache, for example at about 64 bytes for a 4 KB cache and at about 256 bytes for a 256 KB cache: a large cache has room for many blocks even when they are large, so the space wasted on unused data hurts it less.

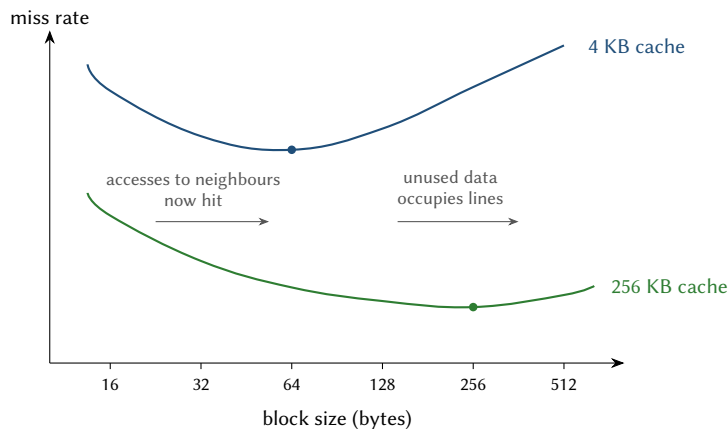


Figure 14.4: Miss rate as a function of the block size for a 4 KB and a 256 KB cache (shape only). The dots mark the best block sizes.

14.9 Prefetching

A miss can be avoided altogether if the block is brought into the cache before the program asks for it.

Definition 14.15 (Prefetching). Loading a block into the cache before it is accessed, because it is expected to be accessed soon, is called *prefetching*.

Prefetching is a guess, and its effect depends on whether the guess is right.

- A correct guess turns the first access to the block into a hit. The fetch from memory still takes place, but earlier, while the processor is doing other work, so the miss penalty is not paid by the access.
- A wrong guess wastes the fetch. Worse, the prefetched block has taken the place of another block, which may be needed later and then misses: the wrong guess *pollutes* the cache.

Prefetching pays off only if most guesses are right.

14.9.1 Prefetch instructions

The program often knows which data it will access next. A *prefetch instruction* is an instruction of the ISA that asks the processor to bring the block containing a given address into the cache; it changes no data. The compiler or the programmer places it in front of accesses that are known in advance, typically in loops over arrays [25]:

Source Lesson 14, videos 21–22; H&P §B.3.

Source Lesson 14, videos 23–26; H&P §2.3.

▷ プリフェッチ

▷ プリフェッチ命令

```

for (i = 0; i < n; i++) {
    prefetch(&a[i + D]);    /* D iterations ahead */
    sum = sum + a[i];
}

```

The difficulty is the prefetch distance D .

- If D is too small, the prefetch has not completed when the loop reaches element $i + D$; the access still waits for the block, and little is gained.
- If D is too large, the block arrives long before it is used. It occupies a line in the meantime and may be evicted again before it is needed.

The right distance is about the time a fetch takes divided by the time of one iteration. Both depend on the machine, so a program compiled with a distance that suits one processor prefetches too late or too early on another.

Example 14.16. A prefetched block is in the cache 50 ns after the prefetch instruction, and one iteration of the loop above takes 4 ns. The distance must be at least $50/4 = 12.5$, so a distance of 16 leaves a small margin. On a processor that executes each iteration in 2 ns with the same memory, the loop reaches element $i + 16$ only 32 ns after its prefetch, 18 ns before the block arrives. The loop then has to wait for memory: roughly every 16th access waits 18 ns, so 16 iterations take about 50 ns instead of 32 ns, and the faster processor runs the loop at only about 3.1 ns per iteration. It needs a distance of at least 25.

Caution A prefetch distance is tuned for one machine. A faster processor with the same memory needs a larger distance.

14.9.2 Hardware prefetching

In *hardware prefetching* the processor itself observes the sequence of memory accesses and guesses which blocks will be needed. The program need not be changed, and the guesses adapt to the machine the program runs on. Three designs exploit different access patterns (Figure 14.5).

Stream buffer A *stream buffer* [26] is a sequential prefetcher. When blocks are accessed in order, it fetches the next few blocks into a small buffer beside the cache, from where an access takes them without a memory access, and it fetches further blocks as the buffer is used up.

Stride prefetcher A *stride prefetcher* detects accesses that advance by a fixed distance, the *stride*, as when a loop visits every k -th element of an array, and prefetches the address one or more strides ahead.

Correlating prefetcher A *correlating prefetcher* records, for a block, which block was accessed after it, and when the block is accessed again prefetches the recorded successor. It handles patterns without a regular spacing, such as a linked list whose nodes are scattered in memory and that is traversed repeatedly.

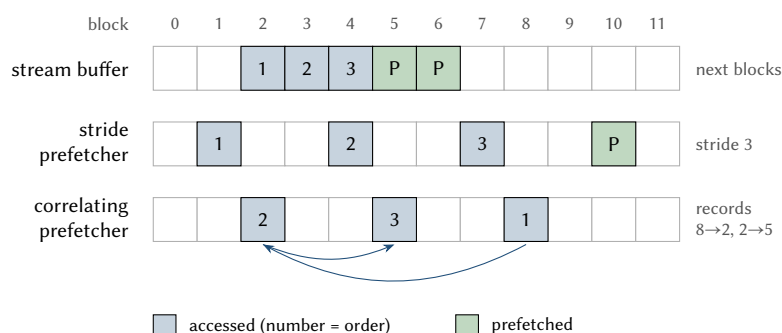


Figure 14.5: Access patterns exploited by the three hardware prefetchers. After the three accesses shown, the stream buffer fetches the following blocks, the stride prefetcher the block one stride ahead, and the correlating prefetcher block 2 the next time block 8 is accessed.

▷ ハードウェアプリフェッチ

▷ ストリームバッファ

▷ ストライドプリフェッチャ

▷ 相関プリフェッチャ

14.10 Loop interchange

The compiler can also lower the miss rate by changing the order in which a program accesses its data. C stores a two-dimensional array row by row: $m[j][i]$ and $m[j][i+1]$ are adjacent, whereas $m[j+1][i]$ lies a whole row further on. The following loop nest walks the array column by column:

```
double m[2000][1000];
for (i = 0; i < 1000; i++)
    for (j = 0; j < 2000; j++)
        m[j][i] = 2 * m[j][i];
```

Each access of the inner loop jumps 1000 elements, 8000 bytes, beyond the previous one. There is no spatial locality: every access of the inner loop touches a different block, and the 2000 blocks visited by one pass of the inner loop are far more than an L1 cache can hold, so nearly every access misses.

Definition 14.17 (Loop interchange). The compiler transformation that exchanges the order of nested loops, so that the innermost loop runs over the last index of the array and the data is accessed sequentially, is called *loop interchange*.

After interchange the outer loop runs over j and the inner loop over i . The inner loop now visits adjacent elements, one block serves several consecutive accesses, and a sequential or stride prefetcher can anticipate the next block. The compiler may interchange the loops only if it can prove that the result is unchanged. Here each iteration reads and writes only its own element, so the iterations can be executed in any order.

Key point

Prefetching can avoid even the miss on the first access to a block, but it helps only if its guesses are both correct and timely. Loop interchange attacks misses from the program side, by making the order of accesses match the layout of the data in memory.

14.11 Overlapping misses

The miss penalty can be larger than the time needed to fetch a block. An out-of-order processor (Lesson 7) has many instructions in progress, and several of its loads and stores may access memory within a few cycles. Fetching a missing block takes many cycles, and in the meantime another access may miss as well.

Definition 14.18 (Overlap miss). An *overlap miss* is a cache miss that occurs while an earlier miss is still being serviced.

After a load misses, an out-of-order processor continues with other instructions. The load cannot commit, however, so the processor eventually runs out of ROB entries or reservation stations and stops; a second miss issued before that point overlaps the first. How much it costs depends on the cache.

- A *blocking cache* does nothing else while a miss is being serviced. A later access is not even looked up until the earlier miss completes, so an overlapping miss pays the remaining time of the earlier miss in addition to its own penalty (Figure 14.6a).
- A *non-blocking cache* serves hits while a miss is in progress (*hit-under-miss*) and can also start further misses. Several requests are then in progress in memory at the same time, which exploits *memory-level parallelism*, provided that memory can service requests in parallel.

Definition 14.19 (Miss-under-miss support). A cache with *miss-under-miss support* can service several misses at the same time: it keeps track of each miss in progress and sends their requests to the next level of the memory hierarchy concurrently.

Source Lesson 14, video 27; H&P §2.3.

▷ ループ交換

Source Lesson 14, videos 28–30.

▷ オーバーラップミス

▷ ブロッキングキャッシュ

▷ ノンブロッキングキャッシュ

▷ ミスアンダーミス対応

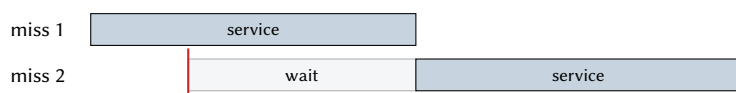
Each miss in progress is recorded in a *miss status handling register* (MSHR). An access that misses in the cache first compares its block address with the MSHRs.

No match The access is a new miss. It allocates a free MSHR, which records the block and the instruction to be woken up, and the request is sent to memory.

Match The block is already on its way; this is a *half-miss*. No second request is sent, and the instruction is only added to the matching MSHR.

When the data arrives, it is placed in the cache, all instructions recorded in the MSHR are woken up, and the MSHR is released. Already two MSHRs give a large improvement over a blocking cache, four are better still, and because memory accesses take so long, even 16 to 32 MSHRs continue to help.

(a) blocking cache



(b) miss-under-miss support

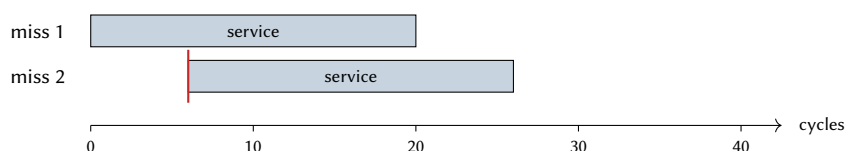


Figure 14.6: Two misses with a miss penalty of 20 cycles; the second occurs 6 cycles after the first (vertical line). (a) In a blocking cache the second miss takes 34 cycles instead of 20. (b) With miss-under-miss support each miss takes 20 cycles.

Miss-under-miss support costs additional hardware. It pays off in processors that issue many memory accesses concurrently, above all out-of-order processors.

14.12 Cache hierarchies

The miss penalty of an L1 cache is dominated by the access time of main memory. It can be reduced with the same idea that led to the L1 cache in the first place: place another cache between L1 and main memory. A miss in L1 then looks up the *L2 cache*, which is larger and slower than L1 but still much faster than main memory, and only misses in L2 go further.

Definition 14.20 (Cache hierarchy). A *cache hierarchy* consists of several levels of cache, L1, L2, L3, ..., each looked up only when the previous level misses (Figure 14.7). The level immediately before main memory is the *last-level cache* (LLC).

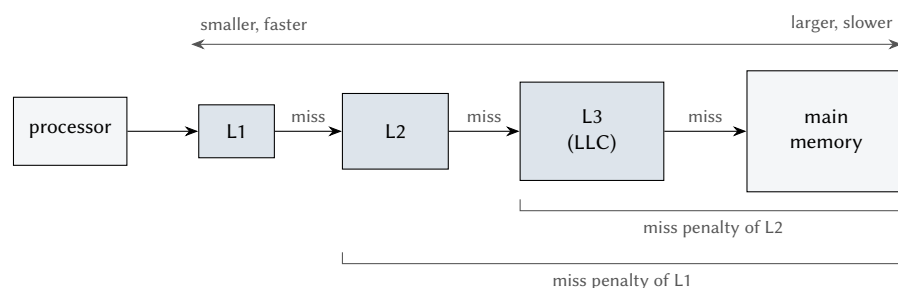


Figure 14.7: A three-level cache hierarchy. The miss penalty of each level is the average access time of the levels behind it.

▷ ミス状態管理レジスタ

▷ ハーフミス

Source Lesson 14, videos 31–34; H&P §B.3.

▷ L2 キャッシュ

▷ キャッシュ階層

▷ 最終レベルキャッシュ

14.12.1 AMAT of a cache hierarchy

The miss penalty of a level is the average time it takes to obtain the block from the levels behind it, which is the AMAT of the rest of the hierarchy. Applying Equation (14.1) level by level gives

$$\begin{aligned} \text{AMAT} &= h_1 + m_1 \times P_1, \\ P_1 &= h_2 + m_2 \times P_2, \\ &\vdots \\ P_{\text{LLC}} &= \text{access time of main memory}, \end{aligned} \quad (14.6)$$

where h_i is the hit time, m_i the miss rate and P_i the miss penalty of level i . The miss rate m_i is counted over the accesses that reach level i (Section 14.13).

14.12.2 L1 versus L2

In Equation (14.6) the hit time h_1 of L1 is paid by every access, whereas the hit time h_2 of L2 is paid only by the fraction m_1 of accesses that miss in L1. L1 is therefore designed for a short hit time, with the techniques of the previous sections, while L2 can trade a longer hit time for a lower miss rate.

Example 14.21. Main memory takes 50 ns per access. A 32 KB cache has a hit time of 1 ns and a miss rate of 10 %; a 256 KB cache has a hit time of 5 ns and a miss rate of 3 %. Assume that in a hierarchy the 256 KB cache misses the same accesses as it would alone, i.e. $0.03/0.10 = 30\%$ of the accesses that reach it.

Configuration	AMAT
No cache	50 ns
32 KB cache only	$1 + 0.10 \times 50 = 6$ ns
256 KB cache only	$5 + 0.03 \times 50 = 6.5$ ns
L1 32 KB, L2 256 KB	$1 + 0.10 \times (5 + 0.30 \times 50) = 3$ ns

The hierarchy is twice as fast as either cache alone. It combines the short hit time of the small cache, paid by every access, with the low miss rate of the large one, whose longer hit time is paid by only 10 % of the accesses.

14.13 Local and global miss rates

In Example 14.21 the L2 cache misses 30 % of the accesses that reach it, three times the rate of L1. This does not make L2 a poor cache. L2 sees only the accesses that L1 has missed, which are the ones with the weakest locality; the accesses that are easy to serve never get past L1. The miss rate of a lower level must therefore be stated with its reference.

Definition 14.22 (Local and global rates). The *local hit rate* of a cache is the number of its hits divided by the number of accesses that reach it, and its *local miss rate* is $1 - \text{local hit rate}$.

The *global miss rate* of a cache is the number of its misses divided by the number of all memory accesses of the processor, and its *global hit rate* is $1 - \text{global miss rate}$.

For L1 the local and global rates coincide. For a lower level the accesses that reach it are the misses of the level above, so the global miss rate of level n is the product of the local miss rates:

$$\text{global miss rate}_n = \prod_{i=1}^n \text{local miss rate}_i. \quad (14.7)$$

The miss rates in Equation (14.6) are local. The global miss rate of a level says what fraction of all accesses has to go beyond it, which makes it the right measure for comparing levels and hierarchies.

Source Lesson 14, videos 35–37; H&P §B.3.

▷ 局所ヒット率
▷ 局所ミス率

▷ 大域ミス率
▷ 大域ヒット率

Definition 14.23 (Misses per 1000 instructions). The metric *misses per 1000 instructions* (MPKI) counts the misses of a cache per 1000 executed instructions.

Like the global miss rate, MPKI counts all misses of a level, but relative to the instructions instead of the memory accesses:

$$\text{MPKI} = 1000 \times \text{global miss rate} \times \frac{\text{memory accesses}}{\text{instruction}}. \quad (14.8)$$

It thus states directly how often per instruction a program pays the miss penalty of that level.

Example 14.24. A program executes 1 000 000 instructions with 1 500 000 memory accesses. L1 misses 90 000 times and L2 27 000 times.

- L1: local and global miss rate $90\,000/1\,500\,000 = 6\%$; MPKI $90\,000/1000 = 90$.
- L2: local miss rate $27\,000/90\,000 = 30\%$ (local hit rate 70 %); global miss rate $27\,000/1\,500\,000 = 1.8\% = 0.06 \times 0.30$ (global hit rate 98.2 %); MPKI 27.

Key point

AMAT is computed with local miss rates. Levels and hierarchies are compared with global miss rates or MPKI, because a low local hit rate is normal for a level that sees only the misses of the level above.

14.14 The inclusion property

A block in L1 may or may not also be present in L2. Three relationships between the levels are possible:

Inclusion every block in L1 is also in L2.

Exclusion no block in L1 is also in L2.

Neither some blocks in L1 are also in L2 and others are not.

Definition 14.25 (Inclusion property). A cache hierarchy has the *inclusion property* if every block in a level is also present in the next, larger level.

It might seem that a larger L2 with LRU replacement keeps every block of L1 automatically. It does not. Hits in L1 never reach L2, so a block that is used often, and therefore stays in L1, looks unused to the LRU counters of L2; eventually it becomes the least recently used block of L2 and is evicted there, while it remains in L1.

Example 14.26. L1 holds 2 blocks and L2 holds 3 blocks; both are fully associative with LRU replacement and initially empty, and a block that misses in both levels is loaded into both. Table 14.4 follows the accesses P, Q, P, R, P, S. The later accesses to P hit in L1, so L2 last sees P at the first access. At the miss on S, P is the least recently used block of L2 and is evicted, although it is still in L1.

Table 14.4: A two-level hierarchy with LRU replacement in both levels (Example 14.26). Contents are listed from least to most recently used; the last access breaks inclusion.

Access	L1	L2	L1 after	L2 after
P	miss	miss	P	P
Q	miss	miss	P Q	P Q
P	hit	—	Q P	P Q
R	miss	miss	P R	P Q R
P	hit	—	R P	P Q R
S	miss	miss	P S	Q R S

Inclusion therefore has to be enforced if it is wanted. One way is an *inclusion bit* in every line of L2, set while the block is also in L1; L2 never chooses a line with this bit set as its victim.

▷ 1000 命令当たりミス数

Source Lesson 14, videos 38–40; H&P §B.3; [27].

▷ 包含性

This lesson completes the discussion of caches in a single processor (Lesson 19 returns to caches in multiprocessors). Lesson 15 turns to the memory technology from which caches and main memory are built.

Key point – Lesson summary

$AMAT = \text{hit time} + \text{miss rate} \times \text{miss penalty}$, and each term has its own techniques. The hit time is reduced by pipelining the lookup, by VIPT caches, which index with the virtual address while the TLB translates and avoid aliasing if cache size $\leq N \times \text{page size}$, by way prediction, and by the cheap replacement policies NMRU and PLRU. Misses are compulsory, capacity or conflict misses; the miss rate is reduced by a suitable block size, by prefetching (prefetch instructions, stream buffers, stride and correlating prefetchers) and by loop interchange. The miss penalty is reduced by miss-under-miss support, which tracks the misses in progress in MSHRs, and by cache hierarchies, whose AMAT is computed level by level with local miss rates, while global miss rates and MPKI describe how many accesses go past a level. With LRU in each level, inclusion does not hold automatically.

Memory

Lecture	Lesson 15, videos 1–14
Textbook	H&P [1] §2.2
Keywords	SRAM, DRAM, memory cell, word line, bit line, sense amplifier, destructive read, trench cell, refresh, row decoder, RAS and CAS, fast page mode, front-side bus, northbridge

Lessons 12–14 treated main memory as a large, slow device hidden behind the caches, and Lesson 1 showed that the gap between processor and memory speed, the memory wall, keeps widening. This lesson looks inside the memory itself. It describes the two technologies from which memories are built, static RAM for caches and dynamic RAM for main memory, down to the circuit that stores a single bit; it then shows how DRAM cells are organized into a chip and accessed, and how the chip is connected to the processor. Together these explain why main memory is slow and why it is not simply built with the faster technology of the caches.

15.1 Memory technologies

Caches and main memory are both random-access memories.

Definition 15.1 (Random access). A memory provides *random access* if any location can be read or written directly through its address, in about the same time for every location.

The opposite is sequential access (Lesson 16), in which a medium such as a magnetic tape has to be moved past all the data that lies between the current position and the requested one. Random-access memory (RAM) is built in two technologies, named after what happens to a stored bit while the memory is powered.

Definition 15.2 (SRAM and DRAM). In *static random-access memory* (SRAM) a stored bit keeps its value for as long as the memory is powered. In *dynamic random-access memory* (DRAM) a stored bit loses its value after a short time, even with power on, unless it is periodically rewritten.

The names say nothing about what happens without power: both technologies are volatile and lose their entire contents when power is removed.

The two technologies differ in the circuit that stores a bit, and this difference determines their speed and cost. SRAM uses several transistors per bit, six in the usual design (Section 15.2); DRAM uses a single transistor (Section 15.3). SRAM therefore takes more chip area per bit and is more expensive per bit, but it is faster, because its cell actively drives the bit lines, whereas a DRAM cell only holds a small charge that has to be sensed, written back and refreshed. DRAM packs more bits into the same area and is cheaper per bit, but slower. Caches, which must be fast and can be small, are built from SRAM; main memory, which must be large and cheap, is built from DRAM.

15.2 The SRAM cell

A memory stores its bits in a two-dimensional array of identical circuits. The circuit that stores one bit is called a *memory cell*. The cells are arranged in rows and columns and are reached through two sets of wires that cross the array.

Definition 15.3 (Bit line and word line). A *bit line* runs along a column of cells and carries the value that is read from or written to a cell of the column. A *word line* runs along a row of cells; when it is activated, it connects every cell of the row to the bit

Source Lesson 15, videos 1–3; H&P §2.2.

▷ ランダムアクセス

▷ SRAM

▷ DRAM

Source Lesson 15, video 4; H&P §2.2.

▷ メモリセル

▷ ビット線

▷ ワード線

line of its column.

In an SRAM cell (Figure 15.1a) the bit is held by two inverters connected in a loop, the output of each driving the input of the other. If the first inverter outputs 1, the second outputs 0, which in turn keeps the first at 1. The loop is stable in either of its two states and holds it for as long as the inverters are powered, which is what makes SRAM static. An access transistor, whose gate is connected to the word line, connects one node of the loop to the bit line.

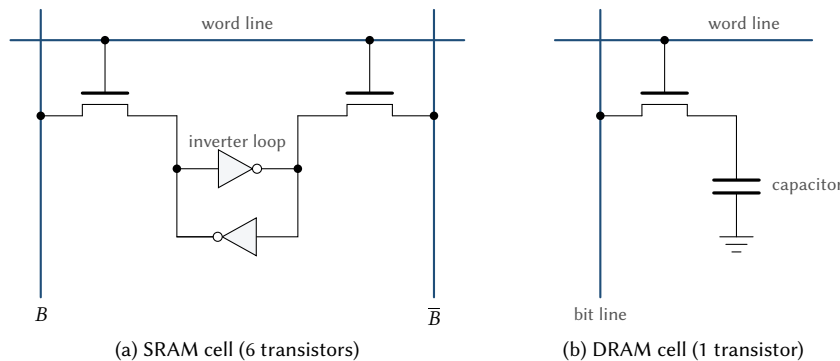


Figure 15.1: One memory bit. (a) An SRAM cell holds the bit in a loop of two inverters, whose nodes are connected to the complementary bit lines B and $\bar{B}$ by two access transistors. (b) A DRAM cell holds the bit as charge on a capacitor behind a single access transistor.

Write The bit line is driven strongly to the new value and the word line is activated. The bit line overpowers the small inverter that drives the node, the loop settles in the state that matches the bit line, and it keeps this state after the word line has been deactivated.

Read The word line is activated and the loop drives the bit line. The transistors of the inverters are small, because the cell has to be small, whereas the bit line is a long wire attached to every cell of the column. A single cell can change the voltage of the bit line only slowly.

To speed up the read, the other node of the loop, which holds the complement of the bit, is connected through a second access transistor to a second bit line, $\bar{B}$. When the word line is activated, the cell pulls the two bit lines in opposite directions, and the value is found by comparing them.

Definition 15.4 (Sense amplifier). A *sense amplifier* detects a small voltage change on a bit line, or a small difference between a pair of bit lines, and amplifies it to a full logic 0 or 1.

▷ センスアンプ

With a complementary pair, the sense amplifier only compares the two bit lines. The cell moves them in opposite directions, so the sign of their difference is clear after a very small swing and no reference voltage is needed; a single bit line would have to move much further before its value could be decided reliably. The pair helps writes as well, since the value and its complement act on both sides of the loop at once. The resulting cell has six transistors, two in each inverter and two access transistors, and it needs a word line, two bit lines and the supply connections of its inverters.

15.3 The DRAM cell

A DRAM cell stores its bit as electric charge on a capacitor (Figure 15.1b). A single access transistor, controlled by the word line, connects the capacitor to the bit line; a charged capacitor represents 1 and a discharged one 0. To write, the bit line is set to the value and the word line is activated, so that the capacitor charges or discharges through the transistor. When the word line is deactivated, the transistor turns off and isolates the charge.

A transistor that is off is not a perfect switch, however. A small current still flows

Source Lesson 15, videos 5–6; H&P §2.2; [28].

through it—the same imperfection that causes static power (Lesson 1)—and the charge of a stored 1 gradually leaks away until the cell can no longer be distinguished from a 0. This is what makes DRAM dynamic: a bit survives only if it is rewritten before too much of its charge has been lost.

Definition 15.5 (Refresh). Reading the bits of DRAM cells and writing them back with full charge before the charge has leaked away is called *refresh*. Every cell must be refreshed periodically, whether or not its data is being used.

Reading a DRAM cell is delicate as well. The bit line is first set to a voltage halfway between 0 and 1, and then the word line is activated. The capacitor shares its charge with the bit line, whose voltage rises slightly if the cell held 1 and falls slightly if it held 0. The capacitor of a cell is tiny compared with the capacitance of the long bit line, so the change is very small, and a sense amplifier is needed to detect it. Once the charge has been shared, the capacitor is left near the intermediate voltage: the stored value is gone.

Definition 15.6 (Destructive read). A *destructive read* is a read that destroys the stored value. Every read of a DRAM cell is therefore followed by writing the value that was read back into the cell.

An SRAM read is not destructive, because the inverter loop keeps driving its nodes and restores their values by itself.

The capacitor also raises a problem of area. Built flat on the surface of the chip, a capacitor large enough to be sensed reliably would occupy far more area than the transistor, and much of the density advantage of a one-transistor cell would be lost.

Definition 15.7 (Trench cell). A *trench cell* is a DRAM cell whose capacitor is formed in a deep, narrow trench etched vertically into the silicon next to the access transistor (Figure 15.2).

The trench gives the capacitor a large plate area while using little of the chip surface. The other common design, used in most current DRAMs, stacks the capacitor above the access transistor; either way the cell occupies little more area than the transistor. It also needs fewer wires than an SRAM cell: one bit line instead of two, and no supply connections for inverters. Table 15.1 summarizes the comparison of the two cells.

Table 15.1: SRAM and DRAM compared.

	SRAM	DRAM
Storage element	inverter loop	capacitor
Transistors per bit	6	1
Bit lines per cell	2	1
Keeps its value when powered	yes	only if refreshed
Read	non-destructive	destructive, then write-back
Speed	fast	slow
Area and cost per bit	high	low
Used for	caches	main memory

Key point

DRAM stores a bit as a tiny charge behind one transistor, which makes it dense and cheap but slow: every read destroys the charge and must be followed by a write-back, and leakage forces a periodic refresh. SRAM avoids both at the price of six transistors per bit.

15.4 Memory chip organization

The cells of a memory chip form an array of 2^r rows and 2^c columns (Figure 15.3), and an address of $r + c$ bits is split into a row address of r bits and a column address of c

▷ リフレッシュ

▷ 破壊読み出し

▷ トレンチセル

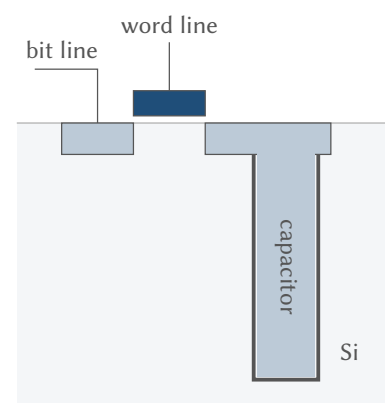


Figure 15.2: Cross-section of a trench cell (schematic).

bits, for instance the upper and the lower bits of the address. A read of one bit of a DRAM chip proceeds in four steps.

1. The *row decoder* turns the row address into the activation of exactly one of the 2^r word lines.
2. Every cell of the selected row is connected to its bit line. One sense amplifier per bit line detects the values, and the whole row is stored in a row latch.
3. The *column multiplexer* uses the column address to select one bit of the latch and delivers it to the data output.
4. The read was destructive for every cell of the row, so the contents of the latch are written back into the row over the bit lines.

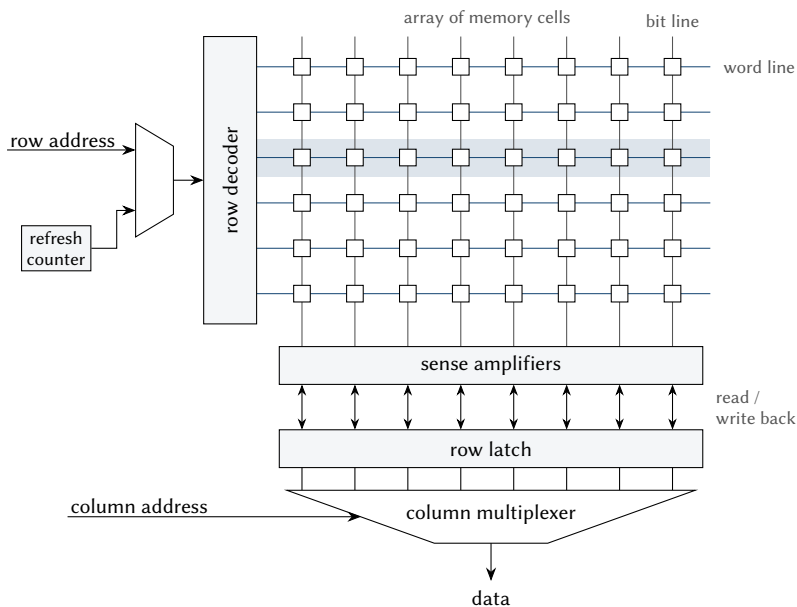


Figure 15.3: Organization of a DRAM chip. The row decoder activates the word line selected by the row address or by the refresh counter (Section 15.5); the sense amplifiers read the whole row into the row latch, the column multiplexer selects one bit, and the latch is written back into the row.

A write uses the same path. The row is read into the latch as above, the bit selected by the column address is replaced in the latch by the value to be written, and the write-back then stores the modified row. In a DRAM chip a write is thus a read whose latch is changed before it is written back.

Note 15.8. A row latch that is written back when the row is closed is the simplified model used throughout this lesson. In real DRAM chips the sense amplifiers themselves latch the row: while the word line is still active they drive the bit lines to full levels, which restores the cells of the row, and a bit written into the latch reaches its cell at once. Closing the row turns the word line off and prepares the bit lines for the next read.

The two parts of the address take very different amounts of time. Selecting a row involves decoding, driving a long word line and waiting for the sense amplifiers to detect tiny charges on long bit lines. Selecting a column only switches a multiplexer between bits that are already in the latch. Sections 15.6 and 15.7 exploit this difference.

Note 15.9. An SRAM chip is organized in the same way, with a row decoder, sense amplifiers and a column multiplexer. Its reads are not destructive, however, so no write-back follows a read, and its cells need no refresh.

Source Lesson 15, video 7; H&P §2.2.

▷ 行デコーダ

▷ 列マルチプレクサ

15.5 Refresh

Every row of a DRAM chip has to be refreshed at least once in every interval T , typically a few tens of milliseconds. A refresh uses the normal read path: it is a read (Section 15.4) whose latch is written back without using the output.

A read by the program refreshes the row it touches, but such reads cannot be relied upon, because some rows are not read for long periods or not at all. All rows are therefore refreshed systematically. A refresh counter holds a row number. Periodically the row decoder is given the value of the counter instead of the row address (Figure 15.3); that row is read and written back, and the counter advances to the next row, wrapping around after the last one. With R rows, each of which must be refreshed once per interval T , the chip must refresh one row, and advance the counter, at least once in every interval of length

$$t_{\text{row}} = \frac{T}{R}. \quad (15.1)$$

While a row is being refreshed, the chip cannot serve a read or a write, and an access that arrives in the meantime has to wait. Refresh is thus one more reason why DRAM is slow, and its cost grows with the number of rows.

Example 15.10. A DRAM chip has 8192 rows whose cells must be refreshed every 64 ms. By Equation (15.1) the chip must refresh one row in every $64 \text{ ms}/8192 \approx 7.8 \mu\text{s}$, i.e. 128 000 rows per second. If refreshing one row takes 50 ns, the chip spends $128\,000 \times 50 \text{ ns} = 6.4 \text{ ms}$ of every second, or 0.64 % of the time, on refresh. A chip with four times as many rows and the same T spends 2.56 %.

15.6 Address multiplexing

A chip with 2^{r+c} cells needs $r + c$ address bits, and every bit that is sent in parallel costs a pin on the package and a wire to the chip. The row address and the column address are not needed at the same time, however. The row address is needed first, to start the slow selection of the row; the column address is needed only once the row is in the latch. DRAM chips therefore receive both parts over the same pins, one after the other, and two control signals tell the chip which part is present.

Definition 15.11 (RAS and CAS). When the *row address strobe* (RAS) is asserted, the chip takes the value on the address pins as the row address, activates the word line and reads the row into the latch. When the *column address strobe* (CAS) is asserted, the chip takes the value on the address pins as the column address and selects that bit of the latch.

On a read the selected bit appears at the data output. If a write-enable signal is asserted together with CAS, the bit in the latch is instead replaced by the value at the data input. Deasserting RAS ends the access: the latch is written back and the row is closed (in the model of Note 15.8). Figure 15.4a shows two accesses of this kind.

Multiplexing reduces the address pins from $r + c$ to $\max(r, c)$, half as many when rows and columns are equally many, which makes the package smaller and cheaper and reduces the wiring between the chip and the memory controller that drives it (Section 15.8). A chip with 2^{28} cells in 16 384 rows and 16 384 columns, for example, needs 14 address pins instead of 28.

15.7 Fast page mode

In the sequence of Figure 15.4a every access opens a row, uses one column and closes the row again, even when the next access is to the same row. Once a row has been opened, however, all its bits are in the latch, and reaching another bit of the row requires only a new column address.

Definition 15.12 (Fast page mode). In *fast page mode* a row, once opened with RAS,

Source Lesson 15, videos 8–9; H&P §2.2.

Source Lesson 15, video 10; H&P §2.2.

▷ 行アドレスストローブ

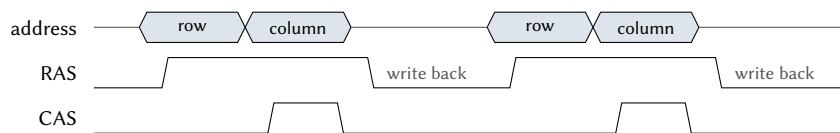
▷ 列アドレスストローブ

In real chips RAS and CAS are active low; Figure 15.4 draws asserted signals high.

Source Lesson 15, videos 11–12.

▷ ファストページモード

(a) two accesses to the same row



(b) fast page mode

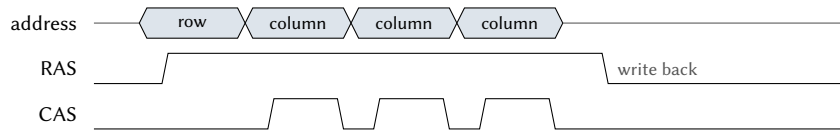


Figure 15.4: Signals of a DRAM chip with multiplexed address pins. (a) Each access opens the row with RAS, selects a column with CAS and closes the row. (b) In fast page mode (Section 15.7) the row is opened once, three columns are accessed, and the row is written back when RAS is deasserted.

stays open while several column addresses are applied with successive CAS pulses, each reading or writing one bit of the row. The row is written back only once, when RAS is finally deasserted (Figure 15.4b).

Fast page mode pays for the expensive steps—selecting and sensing the row, and writing it back—once for a whole group of accesses, and for each access only the quick selection of a column. Accesses to nearby addresses, which spatial locality (Lesson 12) makes common, usually differ only in the lower bits of the address and therefore share a row. The controller that drives RAS and CAS decides how long a row stays open. It can keep the row open as long as the following accesses use the same row, and it must close the row before it accesses another row or refreshes one. The savings therefore depend on how many consecutive accesses share a row.

Example 15.13. In a DRAM chip, opening a row (RAS, decoding and sensing) takes 30 ns, selecting a column with CAS 5 ns, and closing the row (write-back) 10 ns. Five accesses arrive in the order of Table 15.2, where the pairs give (row, column). Without fast page mode every access opens and closes its row, which takes $5 \times 45 \text{ ns} = 225 \text{ ns}$. With fast page mode, rows are opened and closed three times, and the table adds up to 145 ns. Had the access to row 22 come last, the four accesses to row 7 would have shared a single opening, for a total of 105 ns.

Table 15.2: The accesses of Example 15.13 in fast page mode.

Access	Steps	Time (ns)
read (7, 12)	open row 7, CAS column 12	35
read (7, 13)	CAS column 13	5
write (7, 14)	CAS column 14 with write enable	5
read (22, 3)	close row 7, open row 22, CAS column 3	45
read (7, 15)	close row 22, open row 7, CAS column 15	45
—	close row 7	10
Total		145

15.8 Connecting DRAM to the processor

The time the processor waits for main memory is not just the access time of the DRAM chips. Between the two lies a path through other chips and buses, shown in its traditional form in Figure 15.5a.

Definition 15.14 (Front-side bus and northbridge). The *front-side bus* (FSB) connects the processor to the *northbridge*, a chip that links the processor with main memory,

Caution This “page” is a row of the chip, not a virtual-memory page (Lesson 13).

Source Lesson 15, videos 13–14.

- ▷ フロントサイドバス
- ▷ ノースブリッジ

with fast devices such as the graphics adapter, and with the southbridge, to which the slower I/O devices are attached (Lesson 16).

The northbridge contains the *memory controller*. It translates each memory access that arrives over the FSB into the signals of the DRAM chips: it splits the address into the row and the column address and drives RAS, CAS and write enable over the memory bus. A memory access on a cache miss therefore travels from the processor over the FSB to the northbridge and from there over the memory bus to the DRAM chips, and the data returns the same way. Each of these steps adds latency, and the FSB carries all other traffic between the processor and the devices behind the northbridge as well.

▷ メモリコントローラ

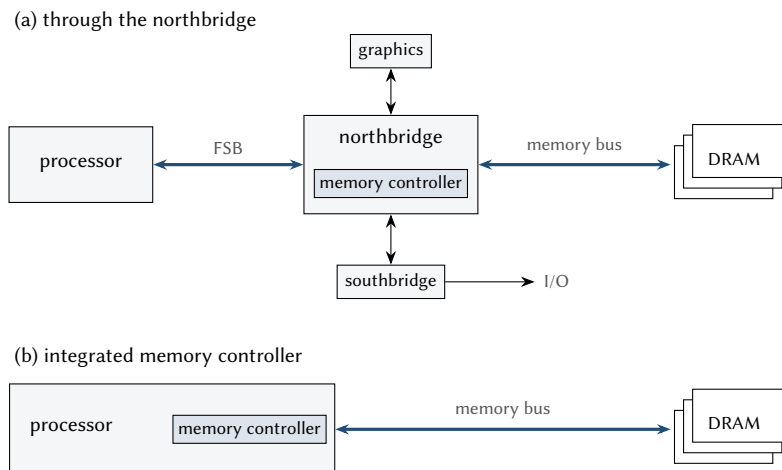


Figure 15.5: The path between processor and DRAM. (a) The memory controller in the northbridge is reached over the front-side bus. (b) With the memory controller integrated into the processor, the memory bus is attached to the processor directly.

Newer processors integrate the memory controller on the processor chip and attach the memory bus to the processor directly (Figure 15.5b). The FSB and the northbridge then disappear from the path to memory, which lowers the latency of every access to main memory. Even so, a DRAM access remains far slower than the processor, for the reasons collected in this lesson, and the caches of Lessons 12–14 remain indispensable. Lesson 16 turns to storage devices, which are slower still but keep their data without power.

Key point — Lesson summary

RAM is built as SRAM, whose six-transistor cell holds a bit in an inverter loop, or as DRAM, whose one-transistor cell (for example a trench cell) holds it as charge on a capacitor. SRAM is fast and expensive and is used for caches. DRAM is dense and cheap and is used for main memory, but its reads are destructive and are followed by a write-back, and leakage forces every row to be refreshed, so that one row must be refreshed in every interval T/R . A DRAM chip selects a row with the row decoder, reads it with the sense amplifiers into a latch and selects a bit with the column multiplexer. The row and column addresses share the address pins and are marked by RAS and CAS, and fast page mode serves several accesses to one row with a single opening. A memory access traditionally travels over the front-side bus to the memory controller in the northbridge and on over the memory bus; newer processors integrate the memory controller and drive the memory bus directly.

Bibliography

- [1] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. 6th ed. Morgan Kaufmann, 2017.
- [2] Gordon E. Moore. “Cramming More Components onto Integrated Circuits”. In: *Electronics* 38.8 (1965).
- [3] William A. Wulf and Sally A. McKee. “Hitting the Memory Wall: Implications of the Obvious”. In: *ACM SIGARCH Computer Architecture News* 23.1 (1995), pp. 20–24.
- [4] Standard Performance Evaluation Corporation. *SPEC Benchmarks*. URL: <https://www.spec.org/> (visited on 09/14/2026).
- [5] Gene M. Amdahl. “Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities”. In: *AFIPS Conference Proceedings* 30 (1967), pp. 483–485.
- [6] Johnny K. F. Lee and Alan Jay Smith. “Branch Prediction Strategies and Branch Target Buffer Design”. In: *Computer* 17.1 (1984), pp. 6–22.
- [7] James E. Smith. “A Study of Branch Prediction Strategies”. In: *Proceedings of the 8th Annual International Symposium on Computer Architecture (ISCA)*. 1981, pp. 135–148.
- [8] Tse-Yu Yeh and Yale N. Patt. “Two-Level Adaptive Training Branch Prediction”. In: *Proceedings of the 24th Annual International Symposium on Microarchitecture (MICRO)*. 1991, pp. 51–61.
- [9] Scott McFarling. *Combining Branch Predictors*. WRL Technical Note TN-36. Digital Equipment Corporation, Western Research Laboratory, 1993.
- [10] David R. Kaeli and Philip G. Emma. “Branch History Table Prediction of Moving Target Branches Due to Subroutine Returns”. In: *Proceedings of the 18th Annual International Symposium on Computer Architecture (ISCA)*. 1991, pp. 34–42.
- [11] J. R. Allen et al. “Conversion of Control Dependence to Data Dependence”. In: *Proceedings of the 10th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. 1983, pp. 177–189.
- [12] Scott A. Mahlke et al. “A Comparison of Full and Partial Predicated Execution Support for ILP Processors”. In: *Proceedings of the 22nd Annual International Symposium on Computer Architecture (ISCA)*. 1995, pp. 138–150.
- [13] Robert M. Keller. “Look-Ahead Processors”. In: *ACM Computing Surveys* 7.4 (1975), pp. 177–195.
- [14] David W. Wall. “Limits of Instruction-Level Parallelism”. In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 1991, pp. 176–188.
- [15] Robert M. Tomasulo. “An Efficient Algorithm for Exploiting Multiple Arithmetic Units”. In: *IBM Journal of Research and Development* 11.1 (1967), pp. 25–33.
- [16] James E. Smith and Andrew R. Pleszkun. “Implementing Precise Interrupts in Pipelined Processors”. In: *IEEE Transactions on Computers* 37.5 (1988), pp. 562–573.
- [17] Monica Lam. “Software Pipelining: An Effective Scheduling Technique for VLIW Machines”. In: *Proceedings of the ACM SIGPLAN 1988 Conference on Programming Language Design and Implementation (PLDI)*. 1988, pp. 318–328.
- [18] Joseph A. Fisher. “Trace Scheduling: A Technique for Global Microcode Compaction”. In: *IEEE Transactions on Computers* C-30.7 (1981), pp. 478–490.
- [19] Joseph A. Fisher. “Very Long Instruction Word Architectures and the ELI-512”. In: *Proceedings of the 10th Annual International Symposium on Computer Architecture (ISCA)*. 1983, pp. 140–150.

- [20] Michael S. Schlansker and B. Ramakrishna Rau. "EPIC: Explicitly Parallel Instruction Computing". In: *Computer* 33.2 (2000), pp. 37–45.
- [21] Peter J. Denning. "The Locality Principle". In: *Communications of the ACM* 48.7 (2005), pp. 19–24.
- [22] Maurice V. Wilkes. "Slave Memories and Dynamic Storage Allocation". In: *IEEE Transactions on Electronic Computers* EC-14.2 (1965), pp. 270–271.
- [23] T. Kilburn et al. "One-Level Storage System". In: *IRE Transactions on Electronic Computers* EC-11.2 (1962), pp. 223–235.
- [24] Mark D. Hill and Alan Jay Smith. "Evaluating Associativity in CPU Caches". In: *IEEE Transactions on Computers* 38.12 (1989), pp. 1612–1630.
- [25] David Callahan, Ken Kennedy, and Allan Porterfield. "Software Prefetching". In: *Proceedings of the Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS-IV)*. 1991, pp. 40–52.
- [26] Norman P. Jouppi. "Improving Direct-Mapped Cache Performance by the Addition of a Small Fully-Associative Cache and Prefetch Buffers". In: *Proceedings of the 17th Annual International Symposium on Computer Architecture (ISCA)*. 1990, pp. 364–373.
- [27] Jean-Loup Baer and Wen-Hann Wang. "On the Inclusion Properties for Multi-Level Cache Hierarchies". In: *Proceedings of the 15th Annual International Symposium on Computer Architecture (ISCA)*. 1988, pp. 73–80.
- [28] Robert H. Dennard. "Field-Effect Transistor Memory". U.S. pat. 3387286. International Business Machines Corporation. June 4, 1968.

Index

Symbols

1BP, *see* one-bit predictor
2BC, *see* two-bit predictor
2BP, *see* two-bit predictor

A

aliasing, 98
AMAT, *see* average memory access time
Amdahl's law, 6
architectural register, 31
associativity, 79
average memory access time, 76

B

backward compatibility, 73
benchmarks, 4
BHT, *see* branch history table
bit line, 109
block number, 78
block offset, 78
block size, 77
blocking cache, 104
branch hint, 74
branch history table, 15
branch misprediction, 12
branch misprediction recovery, 50
branch prediction, 12
branch prediction accuracy, 13
branch target address, 12
branch target buffer, 15
broadcast, *see* write result
BTB, *see* branch target buffer

C

cache, 76
cache block, 77
cache hierarchy, 105
cache hit, 76
cache line, 77
cache miss, 76
capacity miss, 101
CAS, *see* column address strobe
CDB, *see* common data bus
CMOVcc, 26
code bloat, 68
column address strobe, 113
column multiplexer, 112
commit, 49, 61
commit pointer, 48
common data bus, 39
compulsory miss, 101
computer architecture, 1
condition code, 26
conditional move, 26

conflict miss, 101
control dependence, 9
correlating prefetcher, 103

D

data dependence, 10
destructive read, 111
dies, 3
digital signal processor, 74
diminishing returns, 6
direct-mapped cache, 79
direction predictor, 15
dirty bit, 83
dispatch, 39
DRAM, *see* dynamic random-access memory
DSP, *see* digital signal processor
dynamic power, 2
dynamic random-access memory, 109

F

false dependences, 10, 31
fast page mode, 113
flat page table, 89
flush, 9
forwarding, 11
front-side bus, 114
FSB, *see* front-side bus
full predication, 28, 74
fully associative cache, 81
function call inlining, 69, 74

G

geometric mean, 5
global branch history, 19
global hit rate, 106
global miss rate, 106
GShare predictor, 19

H

half-miss, 105
hardware prefetching, 103
hazard, 10
hierarchical predictor, 21
history-based predictor, 18
hit rate, 76
hit time, 76

I

ideal processor, 33
if-conversion, 25, 66, 74
ILP, *see* instruction-level parallelism
in-order execution, 35
in-order load-store execution, 59
inclusion property, 107

index, 79
 instruction queue, 38
 instruction scheduling, 64
 instruction-level parallelism, 33
 instructions per cycle, 35
 internal fragmentation, 92
 IPC, *see* instructions per cycle
 iron law, 5
 issue, 39
 issue pointer, 48
 Itanium, 28, 74

L
 L1 cache, 77
 L2 cache, 105
 last-level cache, 105
 latency, 4
 least recently used, 82
 LLC, *see* last-level cache
 load-store queue, 57
 local hit rate, 106
 local miss rate, 106
 locality principle, 75
 loop interchange, 104
 loop unrolling, 66, 72
 LRU, *see* least recently used
 LSQ, *see* load-store queue

M
 memory cell, 109
 memory controller, 115
 memory wall, 2
 meta-predictor, 20
 misprediction penalty, 13
 miss penalty, 76
 miss rate, 76
 miss status handling register, 105
 miss-under-miss support, 104
 misses per 1000 instructions, 107
 Moore's law, 1
 MOVN, 26
 MOVZ, 26
 MPKI, *see* misses per 1000 instructions
 MSHR, *see* miss status handling
 register
 multi-level page table, 90

N
 N-bit history predictor, 18
 NMRU, *see* not most recently used
 non-blocking cache, 104
 northbridge, 114
 not most recently used, 100
 not-taken branch, 12

O
 oldest first, 40
 one-bit predictor, 16
 out-of-order execution, 35, 37, 55

out-of-order load-store execution, 59
 overlap miss, 104

P
 page, 87
 page fault, 88
 page frame, 87
 page offset, 88
 page size, 91
 page table, 87
 page table entry, 89
 page table walk, 93
 pattern history table, 19
 PFN, *see* physical frame number
 phantom exception, 47
 PHT, *see* pattern history table
 physical address, 86
 physical frame number, 89
 physical memory, 86
 physical register, 32
 physically accessed cache, 97
 pipelined cache, 95
 PLRU, *see* pseudo-LRU
 precise exception, 46
 predecoding, 22
 predicate register, 28
 predication, 24
 predict not taken, 13
 prefetch instruction, 102
 prefetching, 102
 private branch history, 19
 pseudo-LRU, 100
 PShare predictor, 19
 PTE, *see* page table entry

R
 random access, 109
 RAS, *see* return address stack
 RAT, *see* register alias table
 read-after-write dependence, 31
 refresh, 111
 register alias table, 32, 51
 register renaming, 32
 reorder buffer, 48
 replacement policy, 81
 reservation station, 38, 49
 retirement, *see* commit
 return address stack, 22
 ROB, *see* reorder buffer
 row address strobe, 113
 row decoder, 112
 RS, *see* reservation station

S
 saturating counter, 17
 sense amplifier, 110
 set-associative cache, 80
 software pipelining, 70
 spatial locality, 75

speedup, 4
 SRAM, *see* static random-access
 memory
 stages, 8
 stale result, 41
 stall, 9
 static power, 3
 static random-access memory, 109
 stop bit, 74
 store-to-load forwarding, 58
 stream buffer, 103
 stride prefetcher, 103
 superscalar, 54, 71

T

tag, 78
 taken branch, 12
 temporal locality, 75
 throughput, 4
 TLB, *see* translation lookaside buffer
 TLB miss, 93
 Tomasulo's algorithm, 38, 48
 tournament predictor, 20
 trace scheduling, 70
 translation lookaside buffer, 93
 tree height reduction, 63, 74
 trench cell, 111
 two-bit predictor, 17

U

unified reservation station, 54

V

valid bit, 78
 very long instruction word, 71
 VIPT, *see* virtually indexed physically
 tagged cache
 virtual address, 87
 virtual address space, 87
 virtual memory, 86
 virtual page number, 88
 virtually accessed cache, 97
 virtually indexed physically tagged
 cache, 97
 VLIW, *see* very long instruction word
 VPN, *see* virtual page number

W

wafers, 3
 way prediction, 99
 word line, 109
 write result, 39
 write-after-read dependence, 31
 write-after-write dependence, 31
 write-allocate policy, 82
 write-back policy, 82
 write-through policy, 82

Y

yield, 3